

# بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

## فصل هشتم: Evaluation in Information Retrieval

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

## فهرست مطالب

8.1 Information retrieval system evaluation

8.2 Standard test collections

8.4 Evaluation of ranked retrieval results

8.5 Assessing relevance

8.5.1 Critiques and justifications of the concept of relevance

8.6 A broader perspective: System quality and user utility

8.6.1 System issues

8.6.2 User utility

8.6.3 Refining a deployed system

8.7 Results snippets

8.8 Chapter 8 Summary

8.9 End of Chapter Exercises

## ۸.۱ چرا بازیابی اطلاعات یک علم "آزمایشگاهی" است؟ 🧪

🔍 آشپزخانه مهندسی: کدام ادویه جواب می‌دهد؟

تا اینجا درس، ما مثل آشپزهایی بودیم که انواع مواد اولیه را یاد گرفتیم: ریشه‌یابی (Stemming)، حذف کلمات توقف (Stop Lists)، وزن‌دهی tf-idf و مدل‌های برداری. اما سوال اصلی اینجا است: «از کجا بفهمیم کدام ترکیب خوشمزه‌ترین غذا را می‌سازد؟»

آیا باید حتماً کلمات را ریشه‌یابی کنیم؟ آیا استفاده از idf همیشه نتیجه را بهتر می‌کند؟ در ریاضیات،  $2 + 2$  همیشه ۴ می‌شود. اما در بازیابی اطلاعات، پاسخ این سوالات بستگی دارد. به همین دلیل است که تأکید می‌شود IR یک رشته کاملاً تجربی (Highly Empirical) است. ما نمی‌توانیم پشت میز بنشینیم و اثبات کنیم روش X بهتر است؛ باید آن را روی داده‌های واقعی تست کنیم.

برای اثبات برتری یک تکنیک جدید، ما نیاز به ارزیابی دقیق و کامل روی مجموعه‌های اسناد نماینده (Representative Document Collections) داریم. دقیقاً همان کاری که تیم‌های مهندسی در شرکت‌های بزرگ انجام می‌دهند.

📺 مثال واقعی: وسواسِ Netflix در ارزیابی تجربی

شاید فکر کنید Netflix فقط فیلم‌ها را به شما پیشنهاد می‌دهد. اما آن‌ها حتی روی تصویر کاور (Thumbnail) فیلم‌ها هم ارزیابی تجربی انجام می‌دهند!

زمانی که سریال *Stranger Things* منتشر شد، نتفلیکس یک تصویر ثابت را به همه نشان نداد. آن‌ها الگوریتم‌های مختلفی را تست کردند که به گروه‌های مختلف کاربران، تصاویر متفاوتی نشان می‌داد (تست A/B). نتیجه؟ آن‌ها متوجه شدند که اگر تصویر شامل چهره‌ای با احساس "ترس" باشد، نرخ کلیک به شدت بالاتر می‌رود.

این یعنی بازیابی اطلاعات در عمل! آن‌ها حدس نزدند؛ آن‌ها ارزیابی کردند. در این فصل یاد می‌گیریم چطور این تست‌ها را طراحی کنیم.

در ادامه این فصل، ما متدولوژی‌های رسمی برای ارزیابی نتایج را بررسی می‌کنیم (بخش ۸.۳ و ۸.۴) و یاد می‌گیریم چطور مجموعه‌های تست قابل اعتماد بسازیم (بخش ۸.۲ و ۸.۵). اما قبل از ورود به ریاضیاتِ ارزیابی، باید یک قدم به عقب برگردیم و به هدف نهایی فکر کنیم.

## 😊 مفهوم "شادی کاربر" (User Happiness)

همه فرمول‌ها و متریک‌هایی که خواهیم خواند، تنها یک هدف دارند: تقریب‌زدنِ مفهومی به نام سودمندی کاربر (User Utility). به زبان ساده‌تر: آیا کاربر خوشحال شد؟

شادی کاربر به فاکتورهای مختلفی بستگی دارد:

- سرعت پاسخ‌دهی (Speed): سیستم چقدر سریع جواب داد؟
- مرتبط بودن نتایج (Relevance): آیا جواب‌ها به درد می‌خوردند؟
- رابط کاربری (UI): آیا ظاهر سیستم تمیز و واضح است؟

🚀 آمازون: جنگ سرعت در برابر کیفیت

شرکت **Amazon** سال‌ها پیش تحقیقی منتشر کرد که نشان می‌داد هر **۱۰۰ میلی‌ثانیه** تاخیر در لود شدن صفحه، باعث کاهش ۱٪ در فروش می‌شود. سرعت مهم است!

اما بیایید منطقی باشیم: اگر آمازون با سرعت نور (۰ ثانیه) نتیجه را برگرداند، اما وقتی شما "کفش نایک" را جستجو می‌کنید به شما "موز" نشان دهد، آیا خوشحال می‌شوید؟ قطعاً نه.

"*Blindingly fast, useless answers do not make a user happy*" (پاسخ‌های کورکننده سریع اما بی‌استفاده، کاربر را خوشحال نمی‌کنند). پس فرض معقول ما این است که **Relevance (ارتباط معنایی)** مهم‌ترین فاکتور است، هرچند تنها فاکتور نیست.

البته گاهی اوقات درک کاربر از کیفیت، با متریک‌های مهندسی ما یکی نیست. طراحی رابط کاربری (UI)، چیدمان و حتی **اسنیپت‌ها (Snippets)** - آن چند خط توضیحی که زیر لینک آبی در گوگل می‌بینید - تأثیر عظیمی بر رضایت کاربر دارند (بخش ۸.۷)، حتی اگر رتبه‌بندی نتایج یکسان باشد.

### 🎨 گوگل و ۴۱ طیف رنگ آبی

برای اینکه بفهمید چقدر مسائلی غیر از "الگوریتم جستجو" روی شادی کاربر تأثیر دارد: **ماريسا مایر** (زمانی که در گوگل بود)، دستور داد **۴۱ طیف مختلف از رنگ آبی** را برای لینک‌های تبلیغاتی تست کنند تا ببینند کاربران روی کدام رنگ بیشتر کلیک می‌کنند.

این ربطی به کیفیت نتایج نداشت، اما مستقیماً روی تعامل کاربر (و درآمد گوگل!) تأثیر داشت. این نشان می‌دهد که "ارزیابی" در دنیای واقعی، ترکیبی پیچیده از الگوریتم، روانشناسی و تجربه کاربری است.

در این فصل، ما روی بخش **الگوریتمی** تمرکز خواهیم کرد: چگونه "ارتباط" (Relevance) را اندازه بگیریم تا مطمئن شویم موتور جستجوی ما، صرف‌نظر از رنگ دکمه‌هایش، محتوای درستی را پیدا می‌کند.

## ۸.۱ آزمایشگاه ارزیابی اطلاعات: چگونه به گوگل نمره دهیم؟ 📄

### 🏠 سه‌پایه مقدس ارزیابی (The Standard Test Collection)

فرض کنید شما مهندس ارشد در **Bing** یا **Spotify** هستید و ادعا می‌کنید الگوریتم جدیدی نوشته‌اید که نتایج بهتری می‌دهد. مدیرتان حرف شما را قبول نمی‌کند مگر اینکه "اثبات" کنید. در دنیای IR، ما نمی‌توانیم به احساسمان اعتماد کنیم. ما به یک "میز تشریح" استاندارد نیاز داریم که از سه جزء حیاتی تشکیل شده است:

1. 📁 **انبار اسناد (Document Collection):** یک مجموعه ثابت از اسناد (مثلاً ۱ میلیون صفحه وب یا ۵۰ هزار مقاله پزشکی).

2. 🗨️ **مجموعه نیازهای اطلاعاتی (Information Needs):** لیستی از سوالات یا دغدغه‌های واقعی کاربران که به زبان ماشین (Query) ترجمه شده‌اند.

3. ✅ **کلید آزمون (Relevance Judgments):** بخش سخت ماجرا! برای هر جفتِ «پرسمان-سند»، یک انسان (یا داور) مشخص کرده که آیا این سند "مرتبط" است یا خیر. به این قضاوت‌ها، **استاندارد طلایی (Gold Standard)** یا **Ground Truth** می‌گوییم.

شاید بپرسید چرا قضاوت "مرتبط بودن" اینقدر پیچیده است؟ چرا فقط نمی‌گردیم ببینیم کلمات پرسمان در متن هست یا نه؟

## 🗨️ تفاوت ظریف و حیاتی: "نیاز اطلاعاتی" در برابر "پرسمان"

اینجا جایی است که سیستم‌های آماتور از حرفه‌ای جدا می‌شوند. کاربری را تصور کنید که کلمه "Python" را در گوگل تایپ می‌کند.

- آیا او یک برنامه‌نویس است که دنبال داکيومنتیشن زبان پایتون می‌گردد؟ 🖥️
- یا یک علاقه‌مند به خزندگان است که می‌خواهد یک مار پایتون خانگی بخرد؟ 🐍

از روی یک کلمه (Query)، سیستم گیج می‌شود. اما کاربر در ذهن خود یک **نیاز اطلاعاتی (Information Need)** شفاف دارد.

**مثال :** فرض کنید نیاز اطلاعاتی کاربر این است: "اطلاعاتی درباره اینکه آیا شراب قرمز در کاهش ریسک حمله قلبی موثرتر از شراب سفید است؟"

کاربر این را به یک پرسمان کلیدواژه‌ای تبدیل می‌کند: wine AND red AND white AND heart AND attack AND effective.

حالا اگر سندی داشته باشیم که شامل تمام این کلمات باشد اما درباره "تأثیر مخرب الکل بر قلب" صحبت کند، از نظر سیستم (تطبیق کلمات) عالی است، اما از نظر کاربر (نیاز اطلاعاتی) **غیرمرتبط (Non-relevant)** است.

بنابراین، در ارزیابی استاندارد، ما سند را بر اساس اینکه آیا **نیاز کاربر را رفع می‌کند** قضاوت می‌کنیم، نه اینکه صرفاً کلماتش را دارد. فعلاً هم فرض می‌کنیم دنیا سیاه و سفید است: سند یا مرتبط است (۱) یا نیست (۰). (هرچند در واقعیت طیفی از خاکستری است).

## 🩹 قانون ۵۰: چرا یک پرسمان کافی نیست؟

اگر الگوریتم شما روی پرسمان "Python" عالی کار کرد، آیا الگوریتم خوبی است؟ خیر. نتایج جستجو نویز زیادی دارند. ممکن است روی یک پرسمان شانسی عالی باشید و روی دیگری فاجعه.

طبق تجربه مهندسی (Rule of thumb)، برای اینکه ارزیابی شما از نظر آماری قابل اتکا باشد، باید میانگین عملکرد را روی

**حداقل ۵۰ نیاز اطلاعاتی مختلف** محاسبه کنید. درست مثل اینکه نمی‌توانید کیفیت یک رستوران را فقط با خوردن یک سیب‌زمینی سرخ‌کرده بسنجید؛ باید منوی کامل را تست کنید!

## ⚠️ تله‌ی مرگبار: تقلب در امتحان (Overfitting)

این مهم‌ترین درس برای کسانی است که می‌خواهند در حوزه Data Science یا AI کار کنند (مثل مسابقات Kaggle). بسیاری از سیستم‌های IR پارامترهایی دارند که قابل تنظیم‌اند (Tuning). یک اشتباه رایج و نابخشودنی این است که پارامترها را طوری تنظیم کنید که روی "مجموعه تست" بهترین جواب را بدهند. این دقیقاً مثل این است که دانش‌آموزی سوالات امتحان نهایی را بدزدد، جواب‌ها را حفظ کند و نمره ۲۰ بگیرد. این دانش‌آموز باسواد نیست، فقط تقلب کرده است!

### راه درست چیست؟

باید داده‌ها را جدا کنید:

- **Development Test Collection (مجموعه توسعه):** پارامترهای سیستم را اینجا تنظیم و دستکاری کنید (چرک‌نویس).

• **Test Collection (مجموعه تست نهایی):** وقتی سیستم نهایی شد، فقط یک بار روی این مجموعه اجرا کنید تا نمره واقعی و بدون سوگیری (Unbiased) مشخص شود.

در بخش‌های بعد، یاد می‌گیریم وقتی این "کلید آزمون" را داریم، دقیقاً چه نمره‌ای (Precision, Recall, etc) به سیستم بدهیم.

## ۸.۲ مجموعه‌های تست استاندارد: زمین بازی برای سنجش کارایی سیستم‌های IR

آیا تا به حال به این فکر کرده‌اید که چطور شرکت‌هایی مثل Microsoft (با Bing) یا تیم‌های پژوهشی دانشگاهی می‌توانند ادعا کنند که الگوریتم جدید جستجوی آن‌ها بهتر از بقیه کار می‌کند؟ پاسخ در اینجا است: مجموعه‌های تست استاندارد.

### نقش مجموعه‌های تست در دنیای واقعی

این مجموعه‌ها، شبیه به «زمین بازی» یا «بنچ‌مارک» های متمرکز هستند که شامل اسناد (Documents)، پرسمان‌ها (Queries/Topics)، و مهم‌تر از همه، قضاوت‌های مرتبط بودن (Relevance Judgments) می‌باشند. این قضاوت‌ها، که توسط انسان‌ها انجام شده‌اند، حکم پاسخ صحیح (Ground Truth) را دارند. وجود این پاسخ صحیح به ما امکان می‌دهد تا عملکرد هر الگوریتم جدید بازیابی اطلاعات را با دقت کمی و قابل تکرار اندازه‌گیری کنیم و بگوییم روش A دقیقاً چقدر از روش B بهتر است.

ما در اینجا به طور خاص بر مجموعه‌های تست برای ارزیابی سیستم‌های بازیابی اطلاعات آزاد (Ad Hoc IR) متمرکز هستیم، اما به چند مجموعه مهم در حوزه طبقه‌بندی متنی (Text Classification) نیز اشاره می‌کنیم.

### • Cranfield (کرنفیلد):

- اهمیت تاریخی: این مجموعه پیشگام، در اواخر دهه ۱۹۵۰ در انگلستان جمع‌آوری شد و اولین مجموعه‌ای بود که اندازه‌گیری‌های کمی دقیق کارایی IR را ممکن ساخت.
- محتوا: ۱۳۹۸ چکیده مقالات ژورنال آیرودینامیک و ۲۲۵ پرسمان.
- ویژگی منحصر به فرد: دارای قضاوت‌های مرتبط بودن کامل و جامع (Exhaustive Relevance Judgments) برای همه جفت‌های (پرسمان، سند).
- کاربرد امروزی: به دلیل اندازه بسیار کوچک، فقط برای آزمایش‌های مقدماتی (Pilot Experiments) مناسب است.

### • TREC (کنفرانس بازیابی متنی):

- پیشگام وب: از سال ۱۹۹۲ توسط NIST (مؤسسه ملی استانداردها و فناوری آمریکا) اجرا شده و بزرگترین بستر تست برای IR است.
- مجموعه اصلی: مشهورترین زیرمجموعه‌ها، مربوط به مسیر TREC Ad Hoc (بین سال‌های ۱۹۹۲ تا ۱۹۹۹) هستند که در مجموع شامل ۱.۸۹ میلیون سند (اغلب مقالات خبری) و قضاوت برای ۴۵۰ نیاز اطلاعاتی (Topics) می‌باشند.
- زیرمجموعه توصیه‌شده: TREC 6-8 که ۱۵۰ نیاز اطلاعاتی بر روی حدود ۵۲۸,۰۰۰ مقاله خبری دارد، اغلب به عنوان بهترین زیرمجموعه برای پژوهش‌های آتی توصیه می‌شود.
- چالش مقیاس و راه‌حل (Pooling): از آنجایی که این مجموعه‌ها بسیار بزرگ هستند، ارزیابی مرتبط بودن همه اسناد غیرممکن است. بنابراین، ارزیابان NIST قضاوت‌های مرتبط بودن را فقط برای اسنادی انجام

دادند که جزو نتایج Top k (مثلاً ۱۰۰۰ سند برتر) سیستم‌های شرکت‌کننده در آن ارزیابی خاص بودند. این روش هوشمندانه به نام Pooling، ارزیابی کارآمد مجموعه‌های عظیم را ممکن می‌سازد.

#### • GOV2 (مجموعه صفحات وب دولتی): 🏛️

- محتوا: شامل ۲۵ میلیون صفحه از وب (محتوای وب‌سایت‌های دولتی آمریکا).
- اهمیت: این مجموعه در زمان خود از هر مجموعه دیگری که در دسترس پژوهشگران بود، به مراتب بزرگی بزرگتر بود و همچنان بزرگترین مجموعه وب است که به راحتی برای اهداف تحقیقاتی در دسترس است.
- نکته کلیدی: با وجود بزرگی، اندازه GOV2 هنوز بیش از دو مرتبه بزرگی (صد برابر) کوچکتر از مجموعه‌های سند ایندکس‌شده توسط موتورهای جستجوی بزرگ وب (مانند Google Search یا Baidu) است.

#### • NTCIR و CLEF (بازیابی میان‌زبانی): 🗣️

تصور کنید شما در UNICEF (صندوق کودکان سازمان ملل) کار می‌کنید و باید گزارشی در مورد واکسیناسیون در آسیای شرقی تهیه کنید. شما پرسمان خود را به زبان انگلیسی وارد می‌کنید، اما سیستم باید بتواند اسناد مرتبط را از مقالات خبری نوشته‌شده به زبان‌های ژاپنی، چینی یا کره‌ای پیدا کند. این وظیفه، هسته بازیابی اطلاعات میان‌زبانی (Cross-Language Information Retrieval) است.

- NTCIR (ژاپن): مجموعه‌ای مشابه TREC که بر روی زبان‌های آسیای شرقی و بازیابی اطلاعات میان‌زبانی متمرکز شده است.
- CLEF (اروپا): بر روی زبان‌های اروپایی و بازیابی اطلاعات میان‌زبانی تمرکز دارد.

#### • Reuters-21578 و RCV1 (طبقه‌بندی متنی): 📊

- حوزه کاربرد: این مجموعه‌ها برای ارزیابی سیستم‌های طبقه‌بندی متنی، جایی که هدف تعیین دسته‌بندی موضوعی سند است (مانند "سیاست"، "ورزش" یا "اقتصاد")، استفاده می‌شوند.
- Reuters-21578: مجموعه‌ای کلاسیک از ۲۱۵۷۸ مقاله خبری.
- RCV1 (Reuters Corpus Volume 1): این مجموعه بسیار بزرگتر، شامل ۸۰۶,۷۹۱ سند است و به دلیل مقیاس و برچسب‌گذاری غنی، پایه بهتری برای پژوهش‌های مدرن (مبتنی بر یادگیری عمیق) در آینده فراهم می‌کند. (به فصل ۴، مراجعه کنید)

#### • Newsgroups 20 (۲۰ گروه خبری): 💬

- محتوا: مجموعه‌ای پرکاربرد که توسط Ken Lang جمع‌آوری شده است. شامل ۱۰۰۰ مقاله از هر یک از ۲۰ گروه خبری Usenet (در مجموع ۱۸۹۴۱ مقاله پس از حذف تکراری‌ها).
- کاربرد: نام گروه خبری (مانند alt.atheism یا comp.graphics) به عنوان دسته‌بندی سند در نظر گرفته می‌شود و به طور گسترده در ارزیابی الگوریتم‌های طبقه‌بندی و خوشه‌بندی استفاده می‌شود.

نکته پایانی: در حوزه پژوهش، مجموعه‌های تست به ما زبان مشترکی برای مقایسه می‌دهند. تفاوت بین Cranfield که ارزیابی کامل دارد با TREC که از تکنیک Pooling استفاده می‌کند، نشان‌دهنده تکامل این حوزه از مقیاس آزمایشگاهی به مقیاس وب است.

## ۸.۳ ارزیابی مجموعه‌های بازیابی بدون رتبه

اثر بخشی سیستم‌های بازیابی اطلاعات (IR) را چطور اندازه‌گیری کنیم؟ برای حالت ساده‌ای که سیستم صرفاً **مجموعه‌ای از اسناد** را برای یک پرسمان بازمی‌گرداند (بدون رتبه‌بندی)، دو معیار اساسی برای سنجش کارایی تعریف می‌شوند: **Precision (دقت)** و **Recall (بازخوانی)**. در بخش‌های بعدی این مفاهیم برای حالت بازیابی رتبه‌دار (Ranked Retrieval) نیز گسترش می‌یابند.

• **Precision (P) - دقت:** کسری از کل اسناد بازیابی شده که واقعاً مرتبط هستند.

$$\text{Precision} = \frac{\text{relevant items retrieved}}{\text{retrieved items}} = P(\text{relevant}|\text{retrieved}) \quad (8.1)$$

• **Recall (R) - بازخوانی:** کسری از کل اسناد مرتبط موجود که توسط سیستم بازیابی شده‌اند.

$$\text{Recall} = \frac{\text{relevant items retrieved}}{\text{relevant items}} = P(\text{retrieved}|\text{relevant}) \quad (8.2)$$

این مفاهیم با بررسی جدول رخداد (Contingency Table) زیر، که تمام حالات ممکن را بر اساس قضاوت واقعی (مرتبط/نامرتب) و تصمیم سیستم (بازیابی شده/بازیابی نشده) نشان می‌دهد، شفاف‌تر می‌شوند:

| واقعاً نامرتب (Nonrelevant) | واقعاً مرتبط (Relevant) |                                        |
|-----------------------------|-------------------------|----------------------------------------|
| fp (False Positives)        | tp (True Positives)     | توسط سیستم بازیابی شد (Retrieved)      |
| tn (True Negatives)         | fn (False Negatives)    | توسط سیستم بازیابی نشد (Not Retrieved) |

فرمول‌ها بر اساس جدول (۸.۴):

$$P = \frac{tp}{tp + fp} \quad \text{and} \quad R = \frac{tp}{tp + fn}$$

## چالش معیار دقت (Accuracy) در بازیابی اطلاعات

یک معیار بدیهی که در یادگیری ماشین برای ارزیابی طبقه‌بندی (Classification) استفاده می‌شود، **دقت (Accuracy)** است: کسری از کل طبقه‌بندی‌های درست که شامل  $(tp + tn)$  می‌شود:

$$\text{Accuracy} = \frac{tp + tn}{tp + fp + fn + tn}$$

### 🚨 چرا Accuracy معیار مناسبی برای IR نیست؟

در تقریباً تمام سناریوهای بازیابی اطلاعات، توزیع داده‌ها به شدت نامتوازن (Skewed) است: معمولاً بیش از ۹۹.۹٪ از اسناد موجود، نسبت به یک پرسمان خاص، نامرتب  $(tn)$  هستند.

**مثال واقعی (پژوهش پزشکی):** یک محقق در **مرکز تحقیقات بیماری‌های خاص در ایران** در حال جستجو در یک مجموعه ۱,۰۰۰,۰۰۰ مقاله پزشکی برای یافتن مقالات مربوط به یک سندرم فوق‌العاده نادر است. فرض کنید تنها ۵ مقاله واقعاً مرتبط هستند.

• اگر سیستمی بسازید که صرفاً تصمیم بگیرد **هیچ مقاله‌ای** مرتبط نیست و هیچ چیزی را بازنگرداند (

$tp = 0, fp = 0$ )، این سیستم به  $tn \approx 999,995$  دست می‌آید.

• **Accuracy** سیستم می‌شود:  $\frac{0+999,995}{1,000,000} \approx 99.9995\%$

این دقت به طور کاذب بالاست، اما سیستم عملاً شکست خورده است چون تمام ۵ مقاله حیاتی (که  $fn = 5$  هستند) را از دست داده و برای کاربر کاملاً بی‌فایده است. کاربران IR همیشه انتظار دارند حداقل مقداری اطلاعات مفید ببینند. بنابراین، Precision و Recall با تمرکز بر روی **مثبت‌های درست ( $tp$ )**، ارزیابی را متمرکزتر و با معنی (meaningful) می‌کنند.

## مبادله (Trade-off) میان Precision و Recall

این دو معیار اغلب در تضاد با هم هستند. شما همیشه می‌توانید با بازیابی تمام اسناد مجموعه، Recall را به ۱۰۰٪ برسانید (اما Precision فاجعه‌بار خواهد بود). به همین دلیل، اینکه کدام معیار مهم‌تر است، بستگی به هدف جستجوگر دارد:

1. **اولویت P (دقت): کاربران وب.** کاربری که در **دیجی‌کالا** به دنبال بهترین راهنمای خرید یک لپ‌تاپ است، می‌خواهد ۱۰ نتیجهٔ اولش همه دقیق و مرتبط باشند. او علاقه‌ای به دیدن هزاران سند مرتبط (Low Recall tolerance) ندارد و صرفاً نتایج دقیق را در بالای صفحه می‌خواهد.

2. **اولویت R (بازخوانی): تحلیلگران و حقوق‌دانان.** یک وکیل یا مشاور حقوقی در **شرکت حقوقی کاردان** که به دنبال هر نوع سابقه یا اشاره‌ای به یک قانون خاص در هزاران سند بایگانی‌شده است، باید حتماً هر مدرک مرتبط را پیدا کند. او حاضر است نتایج زیادی را که نامرتب هستند (Low Precision tolerance) بررسی کند، به شرطی که حتی یک سند حیاتی از دست نرود (High Recall is critical).

## F-measure: میانگین هارمونیک وزن‌دار

برای داشتن یک معیار واحد که Precision و Recall را با هم در نظر بگیرد و رابطه مبادله آن‌ها را منعکس کند، از **F-measure** استفاده می‌شود. این معیار، **میانگین هارمونیک وزن‌دار** این دو مقدار است:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad \text{or} \quad F = \frac{(1 + \beta^2)PR}{\beta^2 P + R} \quad (8.5)$$
$$\text{where } \beta^2 = \frac{1 - \alpha}{\alpha}$$

در رایج‌ترین حالت، از  $F_1$  استفاده می‌شود که در آن Precision و Recall به‌صورت متوازن (Equal Weighting) وزن‌دهی شده‌اند. این به معنی  $\alpha = 1/2$  و  $\beta = 1$  است و فرمول آن به شکل زیر ساده می‌شود:

$$F_1 = \frac{2PR}{P + R} \quad (8.6)$$

توجه داشته باشید که  $\beta < 1$  (مثلاً  $\beta = 0.5$ ) بر **Precision** تأکید می‌کند و  $\beta > 1$  (مثلاً  $\beta = 3$ ) بر **Recall** تأکید می‌کند. انتخاب  $\beta$  باید مطابق با اولویت‌های کاربردی (مثل مثال‌های بالا) باشد.

## نقش حیاتی میانگین هارمونیک

چرا از میانگین هارمونیک به جای میانگین حسابی (Arithmetic Mean) ساده استفاده می‌کنیم؟

### 💡 دلیل علمی: استبداد حداقل (Tyranny of the Minimum)

میانگین حسابی در شرایطی که یکی از دو مقدار (P یا R) نزدیک به صفر باشد، به شدت گمراه‌کننده است. چون همیشه می‌توان Recall را ۱۰۰٪ کرد، میانگین حسابی همیشه به سمت ۵۰٪ میل می‌کند (حتی اگر Precision صفر باشد!).

**میانگین هارمونیک:** این معیار همواره کمتر یا مساوی با میانگین حسابی و هندسی است. ویژگی کلیدی آن این است که **بسیار به حداقل دو مقدار نزدیک است**. به عبارت دیگر، مقدار نهایی F-measure تحت **استبداد مقدار حداقل** قرار دارد.

برای مثال بالا که  $R = 100\%$  و  $P = 0.01\%$  بود:

- میانگین حسابی:  $50.005\%$  (کاملاً گمراه‌کننده)
- میانگین هارمونیک ( $F_1$ ):  $\approx 0.02\%$  (نزدیک به  $0.01\%$  و واقع‌بینانه)

بنابراین، میانگین هارمونیک تضمین می‌کند که یک سیستم برای کسب امتیاز بالا در F-measure، باید در هر دو معیار Precision و Recall، عملکرد مناسبی داشته باشد.

این دو معیار و F-measure، ستون فقرات ارزیابی کارایی در بازیابی اطلاعات هستند و در فصل‌های آینده برای ارزیابی نتایج رتبه‌دار، توسعه خواهند یافت.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt

def calculate_ir_metrics(tp, fp, fn, tn):
    """
    محاسبه معیارهای اصلی ارزیابی سیستم‌های بازیابی اطلاعات

    پارامترها:
    tp: تعداد اسناد مرتبطی که به درستی بازیابی شده‌اند
    fp: تعداد اسناد نامرتبّی که به اشتباه بازیابی شده‌اند
    fn: تعداد اسناد مرتبطی که به اشتباه بازیابی نشده‌اند
    tn: تعداد اسناد نامرتبّی که به درستی بازیابی نشده‌اند

    خروجی:
    precision, recall, accuracy, f1
    دیکشنری شامل مقادیر

    # نسبت اسناد مرتبط در میان اسناد بازیابی شده: precision محاسبه
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0

    # نسبت اسناد مرتبط بازیابی شده در میان کل اسناد مرتبط: recall محاسبه
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0

    # نسبت کل تصمیمات درست به کل تصمیمات: accuracy محاسبه
    accuracy = (tp + tn) / (tp + fp + fn + tn) if (tp + fp + fn + tn) > 0 else 0

    # precision و recall میانگین هارمونیک: F1-score محاسبه
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    return {
        'precision': precision,
        'recall': recall,
        'accuracy': accuracy,
        'f1': f1
    }

def calculate_f_beta(precision, recall, beta):
    """
    محاسبه F-beta score برای مقادیر مختلف beta

    پارامترها:
    precision: مقدار precision
    recall: مقدار recall
    beta: precision (beta > 1: اولویت با recall, beta < 1: اولویت با precision) وزن نسبت به

    خروجی:
    مقدار F-beta score
    """
    beta_squared = beta ** 2
    return (1 + beta_squared) * precision * recall / (beta_squared * precision + recall)

def compare_means(precision, recall):
    """
    "Tyranny of the Minimum" مقایسه میانگین‌های مختلف (حسابی، هندسی، هارمونیک) برای نمایش مفهوم

    پارامترها:
    precision: مقدار precision
    recall: مقدار recall

    خروجی:
    دیکشنری شامل مقادیر میانگین‌های مختلف
    """
    # میانگین حسابی
    arithmetic_mean = (precision + recall) / 2

    # میانگین هندسی
    geometric_mean = np.sqrt(precision * recall)

    # میانگین هارمونیک (F1-score)
    harmonic_mean = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    return {
        'arithmetic': arithmetic_mean,
        'geometric': geometric_mean,
        'harmonic': harmonic_mean
    }

def plot_means_comparison():
    """
    "Tyranny of the Minimum" رسم نمودار مقایسه‌ای بین میانگین‌های مختلف برای نمایش مفهوم

    # precision و recall ایجاد مقادیر مختلف
    precision_values = np.linspace(0.01, 1, 100)
    recall_values = np.ones_like(precision_values) # همیشه 1 است recall فرض می‌کنیم

    # محاسبه میانگین‌ها
    arithmetic_means = (precision_values + recall_values) / 2
```

```
geometric_means = np.sqrt(precision_values * recall_values)
harmonic_means = 2 * precision_values * recall_values / (precision_values + recall_values)

# رسم نمودار
plt.figure(figsize=(10, 6))
plt.plot(precision_values, arithmetic_means, label='Arithmetic Mean', linewidth=2)
plt.plot(precision_values, geometric_means, label='Geometric Mean', linewidth=2)
plt.plot(precision_values, harmonic_means, label='Harmonic Mean (F1)', linewidth=2)

plt.xlabel('Precision')
plt.ylabel('Mean Value')
plt.title('Comparison of Different Means when Recall = 1')
plt.legend()
plt.grid(True)
plt.show()

def plot_f_beta_scores():
    """
    رسم نمودار F-beta scores برای مقادیر مختلف beta
    """
    # ایجاد precision و recall
    precision_values = np.linspace(0.1, 1, 50)
    recall_values = np.linspace(0.1, 1, 50)

    # ایجاد شبکه برای رسم نمودار سه‌بعدی
    P, R = np.meshgrid(precision_values, recall_values)

    # beta برای مقادیر مختلف F-beta محاسبه
    betas = [0.5, 1, 2]
    fig = plt.figure(figsize=(15, 5))

    for i, beta in enumerate(betas):
        ax = fig.add_subplot(1, 3, i+1, projection='3d')
        beta_squared = beta ** 2
        F = (1 + beta_squared) * P * R / (beta_squared * P + R)

        ax.plot_surface(P, R, F, cmap='viridis')
        ax.set_xlabel('Precision')
        ax.set_ylabel('Recall')
        ax.set_zlabel(f'F{beta} Score')
        ax.set_title(f'F{beta} Score Surface')

    plt.tight_layout()
    plt.show()

# مثال 1: سناریوی متعادل (مجموعه 1000 سند با 20 سند مرتبط)
print("=" * 60)
print("مثال 1: سناریوی متعادل")
print("=" * 60)
tp1, fp1, fn1, tn1 = 8, 10, 12, 970 # 12 سند مرتبط از دست رفته 8
metrics1 = calculate_ir_metrics(tp1, fp1, fn1, tn1)
print(f"Precision: {metrics1['precision']:.4f}")
print(f"Recall: {metrics1['recall']:.4f}")
print(f"Accuracy: {metrics1['accuracy']:.4f}")
print(f"F1-Score: {metrics1['f1']:.4f}")

# مقایسه میانگین‌ها برای مثال 1
means1 = compare_means(metrics1['precision'], metrics1['recall'])
print(f"\nمقایسه میانگین‌ها:")
print(f"میانگین حسابی: {means1['arithmetic']:.4f}")
print(f"میانگین هندسی: {means1['geometric']:.4f}")
print(f"میانگین هارمونیک: {means1['harmonic']:.4f}")

# مثال 2: سناریوی نامتوازن (مجموعه 1,000,000 سند با 5 سند مرتبط)
print("\n" + "=" * 60)
print("Accuracy مشکل معیار) مثال 2: سناریوی نامتوازن")
print("=" * 60)
tp2, fp2, fn2, tn2 = 0, 0, 5, 999995 # 5 سندی را بازیابی نکرده
metrics2 = calculate_ir_metrics(tp2, fp2, fn2, tn2)
print(f"Precision: {metrics2['precision']:.4f}")
print(f"Recall: {metrics2['recall']:.4f}")
print(f"Accuracy: {metrics2['accuracy']:.4f}")
print(f"F1-Score: {metrics2['f1']:.4f}")

# مقایسه میانگین‌ها برای مثال 2
means2 = compare_means(metrics2['precision'], metrics2['recall'])
print(f"\nمقایسه میانگین‌ها:")
print(f"میانگین حسابی: {means2['arithmetic']:.4f}")
print(f"میانگین هندسی: {means2['geometric']:.4f}")
print(f"میانگین هارمونیک: {means2['harmonic']:.4f}")

# کاربر حقوقی (Recall مثال 3: سناریوی اولویت با
print("\n" + "=" * 60)
print("(کاربر حقوقی) Recall مثال 3: سناریوی اولویت با")
print("=" * 60)
tp3, fp3, fn3, tn3 = 4, 100, 1, 895 # 1 سند نامرتب بازیابی شده، 100 سند مرتبط بازیابی شده، 4 سند مرتبط بازیابی شده، 2 سند نامرتب بازیابی شده، 12 سند مرتبط از دست رفته 8
metrics3 = calculate_ir_metrics(tp3, fp3, fn3, tn3)
print(f"Precision: {metrics3['precision']:.4f}")
print(f"Recall: {metrics3['recall']:.4f}")
print(f"Accuracy: {metrics3['accuracy']:.4f}")
print(f"F1-Score: {metrics3['f1']:.4f}")

# beta برای مقادیر مختلف F-beta محاسبه
print(f"\nF-beta scores:")
for beta in [0.5, 1, 2]:
    f_beta = calculate_f_beta(metrics3['precision'], metrics3['recall'], beta)
    print(f"F{beta}: {f_beta:.4f}")

# کاربر وب (Precision مثال 4: سناریوی اولویت با
print("\n" + "=" * 60)
print("(کاربر وب) Precision مثال 4: سناریوی اولویت با")
print("=" * 60)
tp4, fp4, fn4, tn4 = 8, 2, 12, 978 # 12 سند مرتبط از دست رفته 8
```

```
metrics4 = calculate_ir_metrics(tp4, fp4, fn4, tn4)
print(f"Precision: {metrics4['precision']:.4f}")
print(f"Recall: {metrics4['recall']:.4f}")
print(f"Accuracy: {metrics4['accuracy']:.4f}")
print(f"F1-Score: {metrics4['f1']:.4f}")

# beta برای مقادیر مختلف F-beta محاسبه
print(f"\nF-beta scores:")
for beta in [0.5, 1, 2]:
    f_beta = calculate_f_beta(metrics4['precision'], metrics4['recall'], beta)
    print(f"F{beta}: {f_beta:.4f}")

# رسم نمودارها
print("\n" + "=" * 60)
print("رسم نمودار مقایسه میانگین‌ها")
print("=" * 60)
plot_means_comparison()

print("\n" + "=" * 60)
print("رسم نمودار F-beta scores")
print("=" * 60)
plot_f_beta_scores()
```

```
=====
مثال 1: سناریوی متعادل
=====
Precision: 0.4444
Recall: 0.4000
Accuracy: 0.9780
F1-Score: 0.4211

مقایسه میانگین‌ها:
میانگین حساسی: 0.4222
میانگین هندسی: 0.4216
میانگین هارمونیک: 0.4211

=====
Accuracy مشکل معیار) مثال 2: سناریوی نامتوازن
=====
Precision: 0.0000
Recall: 0.0000
Accuracy: 1.0000
F1-Score: 0.0000

مقایسه میانگین‌ها:
میانگین حساسی: 0.0000
میانگین هندسی: 0.0000
میانگین هارمونیک: 0.0000

=====
Recall (کاربر حقوقی) مثال 3: سناریوی اولویت با
=====
Precision: 0.0385
Recall: 0.8000
Accuracy: 0.8990
F1-Score: 0.0734

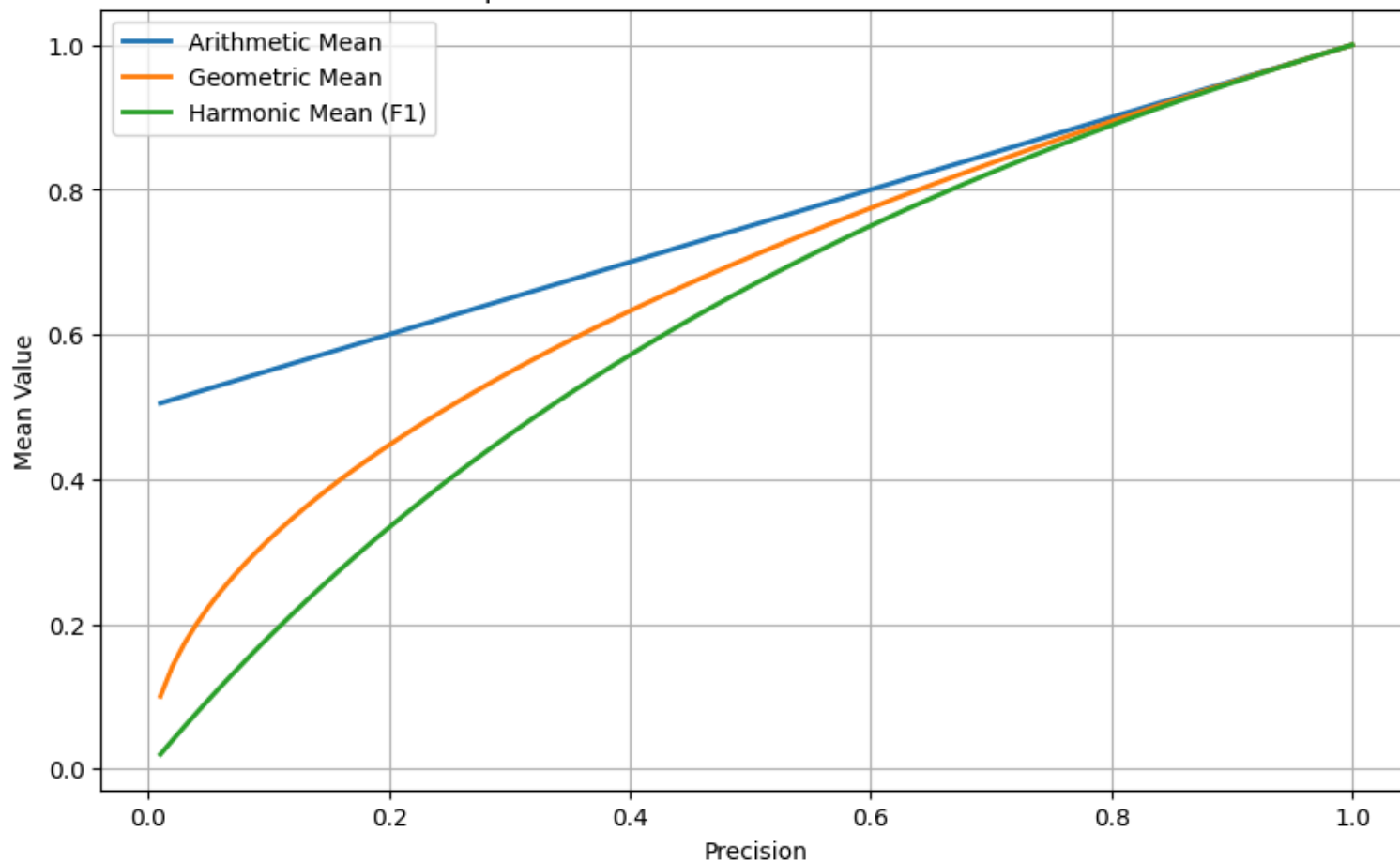
F-beta scores:
F0.5: 0.0475
F1: 0.0734
F2: 0.1613

=====
Precision (کاربر وب) مثال 4: سناریوی اولویت با
=====
Precision: 0.8000
Recall: 0.4000
Accuracy: 0.9860
F1-Score: 0.5333

F-beta scores:
F0.5: 0.6667
F1: 0.5333
F2: 0.4444

=====
رسم نمودار مقایسه میانگین‌ها
=====
```

Comparison of Different Means when Recall = 1

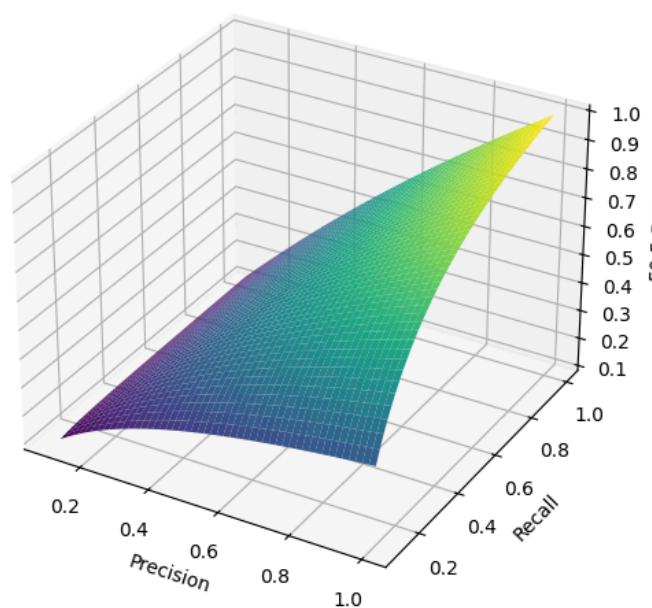


=====

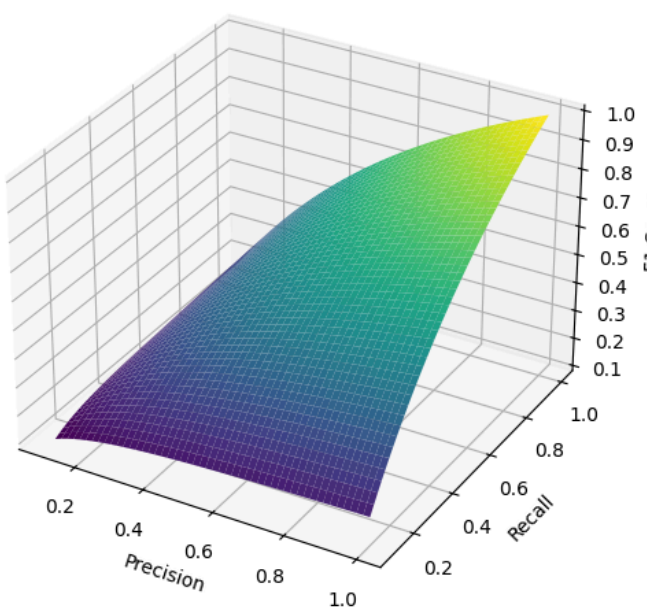
F-beta scores رسم نمودار

=====

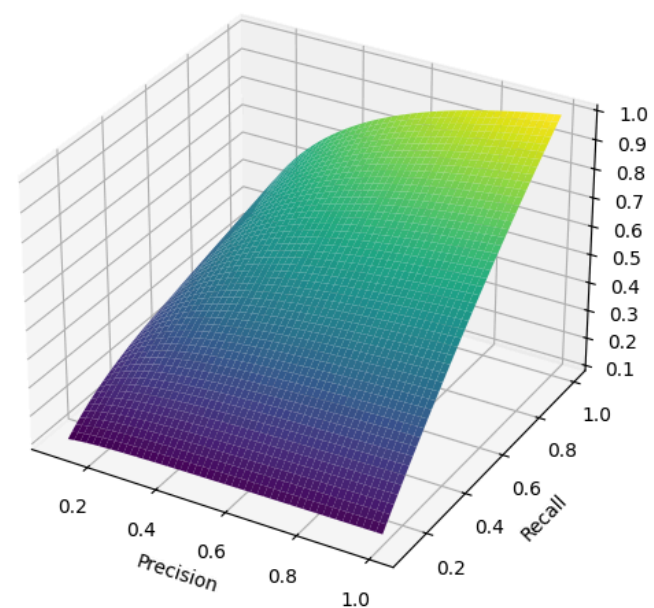
F0.5 Score Surface



F1 Score Surface



F2 Score Surface



## ۸.۴ ارزیابی نتایج بازیابی رتبه‌دار

وقتی در گوگل عبارت «بهترین رستوران‌های ایتالیایی در تهران» را جستجو می‌کنید، نتایج به صورت رتبه‌دار نمایش داده می‌شوند. اما چگونه می‌توانیم بفهمیم که الگوریتم رتبه‌بندی گوگل چقدر خوب عمل می‌کند؟ معیارهای سنتی مانند precision و recall برای مجموعه‌های بدون رتبه طراحی شده‌اند، اما در دنیای واقعی، ما با نتایج رتبه‌دار سر و کار داریم.

### منحنی دقت-بازخوانی (Precision-Recall Curve)

فرض کنید تیم تحقیقاتی در اسنپ در حال ارزیابی الگوریتم جستجوی رستوران‌ها هستند. برای هر پرسمان، می‌توانند precision و recall را در سطوح مختلف  $k$  (تعداد رستوران‌های نمایش داده شده) محاسبه کنند. این منحنی معمولاً شکل دنداندار دارد: اگر رستوران بعدی در لیست نامرتب باشد، recall ثابت می‌ماند ولی precision کاهش می‌یابد؛ اگر مرتبط باشد، هر دو افزایش می‌یابند.

برای حذف نوسانات این منحنی، از دقت درونیابی‌شده (Interpolated Precision) استفاده می‌شود:

$$p_{interp}(r) = \max_{r' \geq r} p(r')$$

این فرمول نشان می‌دهد که کاربران حاضرند چند رستوران بیشتر را ببینند اگر این کار درصد رستوران‌های مرتبط در نتایج را افزایش دهد.

نمونه‌ای از ۱۱ سطح interpolated precision برای ارزیابی الگوریتم جستجوی اسنپ:

| Interpolated Precision | Recall |
|------------------------|--------|
| 1.00                   | 0.0    |
| 0.67                   | 0.1    |
| 0.63                   | 0.2    |
| 0.55                   | 0.3    |
| 0.45                   | 0.4    |
| 0.41                   | 0.5    |
| 0.36                   | 0.6    |
| 0.29                   | 0.7    |
| 0.13                   | 0.8    |
| 0.10                   | 0.9    |
| 0.08                   | 1.0    |

### میانگین میانگین دقت (Mean Average Precision - MAP)

در مسابقات TREC (مهم‌ترین رویداد ارزیابی سیستم‌های بازیابی اطلاعات)، معیار MAP به عنوان استاندارد طلایی برای ارزیابی سیستم‌ها شناخته می‌شود. MAP میانگین precision در نقاطی که سند مرتبط بازیابی شده است را برای هر پرسمان محاسبه می‌کند و سپس میانگین آن‌ها را روی کل پرسمان‌ها می‌گیرد:

$$MAP(Q) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \left( \frac{1}{m_j} \sum_{k=1}^{m_j} Precision(R_{jk}) \right)$$

برای مثال، وقتی **دیجی‌کالا** الگوریتم جستجوی محصولات خود را ارزیابی می‌کند، MAP نشان می‌دهد که به طور متوسط، چقدر محصولات مرتبط در رتبه‌های بالاتر قرار گرفته‌اند. این معیار به خصوص برای کاربرانی که به دنبال خرید محصولات خاص هستند، بسیار مهم است.

### دقت در k نتیجه اول (Precision@k)

برای کاربران عادی، مهم‌ترین بخش نتایج جستجو، صفحه اول است. **Precision@k** دقت را در میان k سند اول اندازه‌گیری می‌کند. برای مثال، **Precision@10** نشان می‌دهد که چند درصد از 10 نتیجه اول جستجو مرتبط هستند. این معیار برای شرکت‌هایی مانند **گوگل** و **یاهو** که اکثر کاربرانشان هرگز به صفحه دوم نتایج نمی‌روند، بسیار حیاتی است. با این حال، این معیار ناپایدار است و به تعداد کل اسناد مرتبط بستگی دارد.

### R-Precision

یک جایگزین برای Precision@10، **R-Precision** است که دقت را در میان |Rel| سند اول اندازه‌گیری می‌کند، که در آن |Rel| تعداد اسناد مرتبط شناخته‌شده است:

$$R-Precision = \frac{r}{|Rel|} \quad \text{where } r = \text{number of relevant documents in top } |Rel|$$

این معیار که توسط محققان **موسسه تحقیقاتی مایکروسافت** معرفی شد، مشکل وابستگی به تعداد اسناد مرتبط را حل می‌کند. برای مثال، اگر برای پرسمان «بهترین دوربین‌های حرفه‌ای» 20 سند مرتبط وجود داشته باشد، R-Precision دقت را در 20 نتیجه اول اندازه‌گیری می‌کند.

### منحنی ROC (ROC Curve)

منحنی ROC نرخ مثبت واقعی (recall) را در برابر نرخ مثبت کاذب (1 - specificity) رسم می‌کند. در بازیابی اطلاعات کلاسیک، specificity تقریباً همیشه نزدیک ۱ است و نمودار ROC فشرده می‌شود. اما در سیستم‌های مدرن مانند **فیلتر اسپم گوگل**، این معیار بسیار مفید است.

برای مثال، وقتی جیمیل در حال یادگیری تشخیص ایمیل‌های اسپم است، منحنی ROC نشان می‌دهد که چگونه الگوریتم با تغییر آستانه تشخیص، بین تشخیص صحیح ایمیل‌های اسپم و علامت‌گذاری اشتباه ایمیل‌های مهم تعادل برقرار می‌کند.

### NDCG (Normalized Discounted Cumulative Gain)

در بسیاری از سیستم‌های مدرن، مرتبط بودن یک سند یک مفهوم دودویی (مرتبط/نامرتبط) نیست، بلکه دارای درجات مختلفی است. برای مثال، در **نتفلیکس**، یک فیلم می‌تواند «کاملاً مطابق سلیقه»، «تا حدی مطابق» یا «اصلاً مطابق» سلیقه کاربر باشد.

NDCG برای چنین سناریوهایی طراحی شده است:

$$NDCG(Q, k) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \frac{1}{Z_{kj}} \sum_{m=1}^k \frac{2^{R(j,m)} - 1}{\log_2(1 + m)}$$

که در آن  $Z_{kj}$  عامل نرمال‌سازی است تا NDCG کامل برابر ۱ شود. این معیار که توسط محققان **یاهو** و **گوگل** توسعه داده شد، امروزه استاندارد طلایی برای ارزیابی سیستم‌های توصیه‌گر است.

برای مثال، وقتی **اسپاتیفای** در حال ارزیابی الگوریتم توصیه موسیقی خود است، NDCG نشان می‌دهد که چقدر موسیقی‌های کاملاً مرتبط در رتبه‌های بالاتر قرار گرفته‌اند. این معیار به طور خاص برای کاربرانی است که به دنبال کشف موسیقی‌های جدید و متناسب با سلیقه خود هستند.

In [8]:

```
import numpy as np
import math

# =====
# تعریف توابع کمکی برای محاسبه معیارهای بازیابی رتبه‌دار
# =====

def calculate_dcg(relevance_scores):
    """
    محاسبه Cumulative Discounted Gain (DCG).
    این تابع، مرتبط بودن نتایج (امتیازدهی شده) را در رتبه‌های مختلف جمع می‌کند و به رتبه‌های پایین‌تر، تخفیف می‌دهد.
    DCG: Sum_{m=1}^{k} (2^{R(m)} - 1) / log2(1 + m) فرمول
    """
    dcg_value = 0.0
    for m, score in enumerate(relevance_scores):
        # است (m+1) از 0 شروع می‌شود، پس رتبه واقعی m است.
        # را دارد Log2(1 + m)، استفاده می‌کنیم چون صورت m+2 در اینجا از
        # Log2(1 + rank) که rank می‌شود
        # rank = m + 1 رتبه را از 0 نشان می‌دهد، پس m، در کد
        rank = m + 1
        gain = 2**score - 1
        # Discounting): تنزیل از Log2(1 + rank) استفاده
        discount = math.log2(1 + rank)
        dcg_value += gain / discount
    return dcg_value

def calculate_ndcg(retrieved_scores, k):
    """
    نتیجه اول k در Normalized Discounted Cumulative Gain (NDCG) محاسبه
    یک معیار استاندارد برای ارزیابی نتایج با مرتبط بودن درجه‌بندی شده است
    """
    # 1. برای نتایج واقعی بازیابی شده DCG محاسبه
    dcg_val = calculate_dcg(retrieved_scores[:k])

    # 2. بهترین حالت ممکن با مرتب‌سازی نتایج بر اساس مرتبط بودن - Ideal DCG (IDCG) محاسبه
    ideal_scores = sorted(retrieved_scores, reverse=True)
    ideal_dcg_val = calculate_dcg(ideal_scores[:k])
```

```
# 3. نرمال‌سازی: DCG / IDCG
if ideal_dcg_val == 0:
    return 0.0
return dcg_val / ideal_dcg_val

def calculate_average_precision(retrieved_labels):
    """
    برای یک پرسمان Average Precision (AP) محاسبه.
    میانگین دقت‌ها را در تمام نقاطی که یک سند مرتبط بازیابی شده است، محاسبه می‌کند AP.
    """

    relevant_count = 0
    precision_sum = 0.0
    total_relevant_docs = retrieved_labels.count('R')

    if total_relevant_docs == 0:
        return 0.0

    for i, label in enumerate(retrieved_labels):
        rank = i + 1
        if label == 'R':
            relevant_count += 1
            # Precision در این رتبه (R_{jk})
            precision_at_rank = relevant_count / rank
            precision_sum += precision_at_rank

    # AP = (مجموع دقت در نقاط مرتبط) / (تعداد کل اسناد مرتبط)
    return precision_sum / total_relevant_docs

# =====
# داده‌های نمونه برای ارزیابی
# =====

# (Non-relevant) برای نامرتبط 'N' و (Relevant) برای مرتبط 'R': نتایج بازیابی شده
# فرض کنید الگوریتم 10 نتیجه اول را برگردانده است.
retrieved_labels_q1 = ['R', 'N', 'R', 'N', 'N', 'R', 'N', 'N', 'R', 'N']
retrieved_labels_q2 = ['N', 'R', 'R', 'N', 'N', 'N', 'N', 'R', 'N', 'N']
all_queries_labels = [retrieved_labels_q1, retrieved_labels_q2]

# کاملاً مرتبط، 0 = نامرتبط = 3 (برای NDCG): مرتبط بودن درجه‌بندی شده
relevance_scores = [3, 0, 2, 0, 1, 0, 0, 0, 2, 1]
# رتبه‌هایی که در آنها سند مرتبط وجود دارد (برای مثال اول): 1, 3, 6, 9
# در مثال دوم: 2, 3, 8
relevant_ranks_q1 = [1, 3, 6, 9]
total_relevant_q1 = len(relevant_ranks_q1) # 4 1 پرسمان برای مرتبط

# =====
# ۱. Precision@k (نتیجه اول k دقت در)
# =====
print("--- 1. Precision@k (نتیجه اول k دقت در) ---")
k_5 = 5
k_10 = 10

# مرتبط 2 => R, N, R, N, N (q1): نتایج مرتبط در 5 نتیجه اول: P@5 محاسبه
p_at_5 = retrieved_labels_q1[:k_5].count('R') / k_5
print(f"Precision@5 (q1): {p_at_5:.4f}، از 5 نتیجه اول {retrieved_labels_q1[:k_5].count('R')} مورد مرتبط است")

# مرتبط 4 (q1): نتایج مرتبط در 10 نتیجه اول: P@10 محاسبه
p_at_10 = retrieved_labels_q1[:k_10].count('R') / k_10
print(f"Precision@10 (q1): {p_at_10:.4f}، از 10 نتیجه اول {retrieved_labels_q1[:k_10].count('R')} مورد مرتبط است")

# =====
# ۲. R-Precision
# =====
print("\n--- 2. R-Precision (دقت در تعداد کل اسناد مرتبط) ---")

# است (تعداد کل اسناد مرتبط برای پرسمان 1) = 4 |Rel| در اینجا.
r_precision_q1 = retrieved_labels_q1[:total_relevant_q1].count('R') / total_relevant_q1
print(f"R-Precision (q1, R=|Rel|=4): {r_precision_q1:.4f}، از 4 نتیجه اول {retrieved_labels_q1[:total_relevant_q1].count('R')} مورد مرتبط است")

# =====
# میانگین میانگین دقت - Mean Average Precision (MAP)
# =====
print("\n--- 3. Mean Average Precision (MAP) ---")

# برای پرسمان 1 AP محاسبه
ap_q1 = calculate_average_precision(retrieved_labels_q1)
print(f"Average Precision (AP) for Query 1: {ap_q1:.4f}")

# برای پرسمان 2 AP محاسبه
ap_q2 = calculate_average_precision(retrieved_labels_q2)
print(f"Average Precision (AP) for Query 2: {ap_q2:.4f}")

# هاست AP میانگین این MAP.
map_score = (ap_q1 + ap_q2) / len(all_queries_labels)
print(f"Mean Average Precision (MAP) for 2 Queries: {map_score:.4f}")

# =====
# ۴. NDCG@k (Normalized Discounted Cumulative Gain)
# =====
print("\n--- 4. NDCG@k (با استفاده از مرتبط بودن درجه‌بندی شده) ---")
k_ndcg = 10 # 10 تا رتبه

# امتیازات مرتبط بودن: [3, 0, 2, 0, 1, 0, 0, 0, 2, 1]
print(f"Relevance Scores (امتیاز مرتبط بودن): {relevance_scores}")

# DCG محاسبه
dcg_value = calculate_dcg(relevance_scores[:k_ndcg])
print(f"DCG@{k_ndcg}: {dcg_value:.4f}")

# NDCG محاسبه
```

```
ndcg_score = calculate_ndcg(relevance_scores, k_ndcg)
print(f"NDCG@{k_ndcg}: {ndcg_score:.4f} (نزدیکتر باشد، بهتر است)")

# =====
# ۵. Interpolated Precision (دقت درونیابی شده)
# =====
# در یک منحنی هستند Precision/Recall داده‌های زیر، نمونه‌ای از نقاط.
print("\n--- 5. Interpolated Precision (دقت درونیابی شده) ---")
recall_points = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
# فرض کنید مقادیر دقت در این نقاط بازخوانی به دست آمده‌اند
precision_at_recall = [0.8, 0.75, 0.7, 0.5, 0.4, 0.6, 0.3, 0.15, 0.1, 0.05]

interpolated_precision_values = []
# p_interp(r) = max_{r' >= r} p(r')
for i, r in enumerate(recall_points):
    # از نقطه کنونی تا انتهای لیست، حداکثر دقت را پیدا کن
    max_precision = max(precision_at_recall[i:])
    interpolated_precision_values.append(max_precision)

print("Interpolated Precision at Standard Recall Points:")
for r, p_interp, p_actual in zip(recall_points, interpolated_precision_values, precision_at_recall):
    print(f"Recall {r:.1f} | Actual P: {p_actual:.4f} | Interpolated P: {p_interp:.4f}")

print("\n(دقت درونیابی شده در واقع حداکثر دقت برای هر سطح بازخوانی کنونی و تمام سطوح بازخوانی بالاتر است)")
```

```
--- 1. Precision@k (نتیجه اول k دقت در) ---
Precision@5 (q1): 0.4000 (از 5 نتیجه اول، 2 مورد مرتبط است)
Precision@10 (q1): 0.4000 (از 10 نتیجه اول، 4 مورد مرتبط است)

--- 2. R-Precision (دقت در تعداد کل اسناد مرتبط) ---
R-Precision (q1, R=|Rel|=4): 0.5000 (از 4 نتیجه اول، 2 مورد مرتبط است)

--- 3. Mean Average Precision (MAP) ---
Average Precision (AP) for Query 1: 0.6528
Average Precision (AP) for Query 2: 0.5139
Mean Average Precision (MAP) for 2 Queries: 0.5833

--- 4. NDCG@k (یا استفاده از مرتبط بودن درجه‌بندی شده) ---
Relevance Scores (امتیاز مرتبط بودن): [1, 2, 0, 0, 0, 1, 0, 2, 0, 3]
DCG@10: 10.0790
NDCG@10: 0.8991 (مقدار نرمال شده‌ای که به 1 نزدیکتر باشد، بهتر است)
```

```
--- 5. Interpolated Precision (دقت درونیابی شده) ---
Interpolated Precision at Standard Recall Points:
Recall 0.1 | Actual P: 0.8000 | Interpolated P: 0.8000
Recall 0.2 | Actual P: 0.7500 | Interpolated P: 0.7500
Recall 0.3 | Actual P: 0.7000 | Interpolated P: 0.7000
Recall 0.4 | Actual P: 0.5000 | Interpolated P: 0.6000
Recall 0.5 | Actual P: 0.4000 | Interpolated P: 0.6000
Recall 0.6 | Actual P: 0.6000 | Interpolated P: 0.6000
Recall 0.7 | Actual P: 0.3000 | Interpolated P: 0.3000
Recall 0.8 | Actual P: 0.1500 | Interpolated P: 0.1500
Recall 0.9 | Actual P: 0.1000 | Interpolated P: 0.1000
Recall 1.0 | Actual P: 0.0500 | Interpolated P: 0.0500
```

(دقت درونیابی شده در واقع حداکثر دقت برای هر سطح بازخوانی کنونی و تمام سطوح بازخوانی بالاتر است)

## ۸.۵ ارزیابی مرتبط بودن

تصور کنید تیمی در **گوگل** در حال بهبود الگوریتم جستجوی پزشکی خود هستند. برای ارزیابی معتبر این سیستم، نیازهای اطلاعاتی باید با مجموعهٔ اسناد تست هم‌راستا و متناسب با کاربرد پیش‌بینی‌شدهٔ سیستم باشند. طراحی این نیازها معمولاً توسط متخصصان حوزه انجام می‌شود. استفاده از ترکیب تصادفی واژه‌ها به‌عنوان پرسمان، معمولاً منجر به نتایج غیرواقعی و نامعتبر می‌شود.

### مثال واقعی: طراحی پرسمان‌های پزشکی 🔍

در یک پروژه تحقیقاتی در **مایکروسافت ریسرچ**، متخصصان پزشکی پرسمان‌های واقعی بیماران را برای ارزیابی سیستم جستجوی اطلاعات پزشکی طراحی کردند. به جای استفاده از ترکیبات تصادفی مانند "تب سرد قرمز"، از پرسمان‌های واقعی مانند "علائم سندرم رینود در بیماران دیابتی" استفاده کردند که با نیازهای واقعی کاربران مطابقت داشت.

پس از تعریف نیازهای اطلاعاتی، باید ارزیابی مرتبط بودن اسناد نسبت به هر نیاز انجام شود. این فرآیند انسانی، پرهزینه و زمان‌بر است. در مجموعه‌های کوچک مانند Cranfield، قضاوت کامل برای هر زوج پرسمان–سند انجام شده است. اما در مجموعه‌های بزرگ، معمولاً فقط برای زیرمجموعه‌ای از اسناد ارزیابی انجام می‌شود.

روش رایج برای انتخاب این زیرمجموعه، **Pooling** است: اسناد انتخاب‌شده شامل k سند برتر از چند سیستم مختلف، نتایج جستجوی بولی، یا اسناد انتخاب‌شده توسط جست‌وجوگران خبره هستند.

در مسابقات سالانه (Text REtrieval Conference) **TREC**، که توسط NIST سازمان ملی استانداردها و فناوری آمریکا برگزار می‌شود، از روش pooling استفاده می‌شود. هر تیم شرکت‌کننده ۱۰۰۰ سند برتر خود را برای هر پرسمان ارائه می‌دهد. سپس ارزیابان فقط این اسناد را از نظر مرتبط بودن بررسی می‌کنند. این روش باعث صرفه‌جویی چشمگیر در زمان و هزینه می‌شود، زیرا در مجموعه‌های بزرگ مانند GOV2 با ۲۵ میلیون سند، بررسی تمام اسناد غیرممکن است.

## چالش ذهنیت در قضاوت مرتبط بودن

ارزیابی انسانی ذاتاً متغیر و فردی است. انسان‌ها دستگاهی نیستند که قضاوت قطعی و یکنواخت ارائه دهند. اما موفقیت یک سیستم IR دقیقاً در توانایی آن برای پاسخ‌گویی به همین نیازهای فردی نهفته است.

## مثال حقوقی: تفاوت در قضاوت

در یک مطالعه در **شرکت حقوقی کاردان**، دو وکیل متخصص اسناد یک پرونده حقوقی را از نظر مرتبط بودن با یک پرسمان خاص ارزیابی کردند. یک وکیل سندی را کاملاً مرتبط دانست زیرا حاوی استناد به یک قانون خاص بود، در حالی که وکیل دیگر همان سند را تنها تا حدی مرتبط ارزیابی کرد زیرا معتقد بود که استناد به قانون در زمینه متفاوتی صورت گرفته است. این مثال نشان می‌دهد که حتی متخصصان یک حوزه ممکن است در مورد مرتبط بودن یک سند اختلاف نظر داشته باشند.

با این حال، بررسی میزان توافق بین ارزیاب‌ها اهمیت دارد. معیار رایج در علوم اجتماعی برای سنجش توافق، **آمار کاپا (Kappa Statistic)** است که نرخ توافق واقعی را با نرخ توافق تصادفی اصلاح می‌کند:

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

که در آن:

- $P(A)$ : نسبت دفعاتی که دو ارزیاب توافق داشتند
- $P(E)$ : نرخ توافق مورد انتظار بر اساس احتمال تصادفی (با استفاده از توزیع‌های حاشیه‌ای)

جدول زیر نمونه‌ای از محاسبه کاپا را نشان می‌دهد:

| Judge 2     | Yes | No | Total |
|-------------|-----|----|-------|
| Judge 1 Yes | 300 | 20 | 320   |
| Judge 1 No  | 10  | 70 | 80    |
| Total       | 310 | 90 | 400   |

محاسبات:

- $P(A) = (300 + 70)/400 = 0.925$
- $P(\text{nonrelevant}) = (80 + 90)/800 = 0.2125$
- $P(\text{relevant}) = (320 + 310)/800 = 0.7878$
- $P(E) = 0.2125^2 + 0.7878^2 = 0.665$
- $\kappa = (0.925 - 0.665)/(1 - 0.665) = 0.776$

- بالای ۰.۸ ← توافق خوب
- بین ۰.۶۷ تا ۰.۸ ← توافق متوسط
- زیر ۰.۶۷ ← داده مشکوک برای ارزیابی

### 📁 مطالعه موردی: ارزیابی مرتبط بودن در پزشکی

در یک مطالعه انجام شده در **مرکز تحقیقاتی Mayo Clinic**، دو متخصص ارزیاب اسناد پزشکی را برای ۱۰۰ پرسمان مختلف قضاوت کردند. میانگین آمار کاپا بین آن‌ها ۰.۷۲ بود که در محدوده "توافق متوسط" قرار می‌گیرد. این یافته نشان می‌دهد که حتی متخصصان پزشکی نیز ممکن است در مورد مرتبط بودن یک سند پزشکی اختلاف نظر داشته باشند، که این موضوع طراحی سیستم‌های ارزیابی اطلاعات پزشکی را چالش‌برانگیزتر می‌کند.

در TREC و مجموعه‌های پزشکی، توافق معمولاً در محدوده «متوسط» قرار دارد. با وجود تفاوت در قضاوت ارزیاب‌ها، رتبه‌بندی نسبی سیستم‌ها معمولاً پایدار باقی می‌ماند – حتی اگر ارزیابی بر اساس نظر یکی از ارزیاب‌ها انجام شود.

### 🔍 نکته کلیدی پژوهشی

محققان در **دانشگاه کلمبیا** آزمایشی انجام دادند که در آن نتایج ارزیابی را بر اساس قضاوت‌های دو ارزیاب مختلف مقایسه کردند. اگرچه انتخاب ارزیاب می‌توانست تفاوت مطلق در امتیازات را ایجاد کند، اما تأثیر کمی بر رتبه‌بندی نسبی سیستم‌های مختلف داشت. این یافته اهمیت تمرکز بر بهبود نسبی سیستم‌ها به جای تمرکز بر امتیازات مطلق را نشان می‌دهد.

## ۸.۵.۱ نقد و توجیه مفهوم مرتبط بودن

وقتی مهندسان **گوگل** الگوریتم جدیدی برای جستجو طراحی می‌کنند، چگونه می‌توانند با اطمینان بگویند که این الگوریتم بهتر از نسخه قبلی عمل می‌کند؟ پاسخ در ارزیابی‌های رسمی است. مزیت اصلی ارزیابی سیستم‌های IR با استفاده از مدل استاندارد مرتبط/نامرتب این است که بستری ثابت برای مقایسه سیستم‌ها و پارامترها فراهم می‌کند. این روش بسیار ارزان‌تر از مطالعات کاربری است و امکان بهینه‌سازی خودکار با یادگیری ماشین را فراهم می‌کند.

### 🔍 مثال واقعی: بهینه‌سازی در میکروسافت

تیم جستجوی **بینگ** در میکروسافت از ارزیابی‌های رسمی برای بهینه‌سازی الگوریتم‌های خود استفاده می‌کند. آنها با استفاده از مجموعه‌های تست استاندارد مانند TREC، می‌توانند تأثیر تغییرات پارامترهای مختلف را به صورت کمی اندازه‌گیری کنند. این رویکرد به آنها اجازه می‌دهد تا با استفاده از یادگیری ماشین، وزن‌های مختلف در معادله امتیازدهی را به صورت خودکار تنظیم کنند، به جای اینکه این کار را به صورت دستی انجام دهند.

البته، اگر معیار رسمی با نیاز واقعی کاربران هم‌راستا نباشد، بهبود آن معیار لزوماً منجر به رضایت کاربر نمی‌شود. در عمل، معیارهای رسمی ساده‌سازی شده‌اند، اما به اندازه کافی خوب هستند. بسیاری از تکنیک‌های موفق مانند نرمال‌سازی طول سند و تنظیم وزن‌ها با یادگیری ماشین، در همین چارچوب توسعه یافته‌اند.

با این حال، این مدل دارای کاستی‌های مفهومی است:

- **استقلال اسناد:** مرتبط بودن یک سند مستقل از سایر اسناد فرض می‌شود. در واقعیت، کاربران مجموعه‌ای از اسناد را بررسی می‌کنند و ارزش یک سند ممکن است به اسناد دیگری که قبلاً دیده‌اند بستگی داشته باشد.
- **قضاوت دودویی:** قضاوت‌ها دودویی هستند (مرتبط یا نامرتبط) و ظرافت ندارند. در واقعیت، یک سند ممکن است "کاملاً مرتبط"، "تا حدی مرتبط" یا "فقط در بخش کوچکی مرتبط" باشد.
- **ذات عینی قضاوت:** مرتبط بودن به صورت تصمیمی مطلق و عینی در نظر گرفته می‌شود، در حالی که در واقعیت قضاوتی ذهنی است که بین افراد مختلف متفاوت است.
- **ثبات نیاز اطلاعاتی:** نیاز اطلاعاتی کاربر در طول جست‌وجو ثابت فرض می‌شود، در حالی که در واقعیت، کاربران با دیدن نتایج، نیاز خود را اصلاح می‌کنند.
- **وابستگی به حوزه:** نتایج به شدت به مجموعه اسناد و پرسمان‌ها وابسته‌اند و ممکن است قابل تعمیم نباشند. سیستمی که در حوزه پزشکی خوب عمل می‌کند، لزوماً در حوزه حقوقی موفق نخواهد بود.

#### 💡 مثال: تفاوت در قضاوت در آمازون

در یک مطالعه در آمازون، دو ارزیاب محصول "دوربین عکاسی حرفه‌ای" را برای پرسمان "بهترین دوربین برای عکاسی ورزشی" ارزیابی کردند. یک ارزیاب دوربینی را که قابلیت فیلم‌برداری با فریم‌ریت بالا داشت کاملاً مرتبط دانست، در حالی که ارزیاب دیگر همان دوربین را تنها تا حدی مرتبط ارزیابی کرد زیرا معتقد بود برای عکاسی ورزشی، سرگشودگی لنز مهم‌تر است. این مثال نشان می‌دهد که حتی متخصصان یک حوزه ممکن است در مورد مرتبط بودن یک سند اختلاف نظر داشته باشند.

برخی از این مشکلات قابل اصلاح‌اند. مثلاً در INEX و برخی مسیرهای TREC و NTCIR، از ارزیابی ترتیبی (ordinal) با چند سطح مرتبط بودن استفاده شده است که اسناد را به دسته‌های "کاملاً مرتبط"، "تا حدی مرتبط" و "نامرتبط" تقسیم می‌کند.

### مرتبط بودن حاشیه‌ای (Marginal Relevance)

یکی از مشکلات واضح با ارزیابی مبتنی بر مرتبط بودن، تمایز بین مرتبط بودن و مرتبط بودن حاشیه‌ای است: آیا یک سند همچنان دارای ارزش متمایز است پس از اینکه کاربر اسناد دیگری را بررسی کرده است؟ حتی اگر سندی مرتبط باشد، ممکن است اطلاعات آن تکراری با اسناد قبلی باشد.

#### 📰 مثال: تکراری بودن خبرها در گوگل

وقتی در گوگل "نتیجه انتخابات ریاست جمهوری آمریکا" را جستجو می‌کنید، ممکن است ۱۰ نتیجه اول همگی از منابع مختلف (BBC، CNN، نیویورک تایمز و ...) تقریباً همان اطلاعات را گزارش کنند. اگرچه هر کدام از این اسناد به صورت جداگانه مرتبط هستند، اما پس از دیدن اولین سند، ارزش حاشیه‌ای ۹ سند بعدی بسیار کم است. در چنین شرایطی، مرتبط بودن حاشیه‌ای معیار بهتری برای سنجش سودمندی واقعی برای کاربر است.

این موضوع در وب بسیار رایج است (اسناد تکراری یا خلاصه‌های مشابه). بنابراین، معیار بهتر، مرتبط بودن حاشیه‌ای است که تنوع و تازگی را نیز لحاظ می‌کند. حداکثر کردن مرتبط بودن حاشیه‌ای مستلزم بازگرداندن اسنادی است که تنوع و نوآوری را نشان دهند.

برای سنجش مرتبط بودن حاشیه‌ای، می‌توان از «حقایق متمایز» یا «موجودیت‌های منحصربه‌فرد» به عنوان واحد ارزیابی استفاده کرد. این روش شاید مستقیماً سودمندی واقعی را برای کاربر اندازه‌گیری کند، اما ساخت مجموعه تست را دشوارتر می‌کند.

در **نتفلیکس**، وقتی کاربر به دنبال فیلم‌های اکشن می‌گردد، سیستم نه تنها فیلم‌های مرتبط را پیشنهاد می‌دهد، بلکه تلاش می‌کند تنوع را نیز حفظ کند. برای مثال، اگر کاربر چند فیلم ابرقهرمانی دیده باشد، نتفلیکس ممکن است فیلم‌های اکشن با تم‌های متفاوت (مانند تعقیب و گریز، جنگی یا علمی-تخیلی) را پیشنهاد دهد تا تجربه کاربر را غنی‌تر کند. این رویکرد، تلاشی برای حداکثر کردن مرتبط بودن حاشیه‌ای است.

## ۸.۶ دید گسترده‌تر: کیفیت سیستم و مطلوبیت کاربر

تا کنون معیارهای رسمی مانند Precision و Recall را برای ارزیابی سیستم‌های بازیابی اطلاعات بررسی کردیم، اما این معیارها فاصله‌ای با نهایت علاقه ما دارند: **چقدر هر کاربر از نتایجی که سیستم برای هر نیاز اطلاعاتی او ارائه می‌دهد، رضایت دارد؟**

### 🔍 مطالعات کاربری در گوگل

گوگل سالانه میلیون‌ها دلار برای مطالعات کاربری هزینه می‌کند. این مطالعات شامل معیارهای کمی مانند **زمان تکمیل یک وظیفه** (مثلاً پیدا کردن اطلاعات و خرید یک محصول) و معیارهای کیفی مانند **امتیاز رضایت** از موتور جستجو و نظرات کاربران در مورد رابط کاربری است. این داده‌ها به گوگل کمک می‌کند تا الگوریتم‌های خود را بهبود بخشد.

در این بخش، به جنبه‌های دیگر سیستم که امکان ارزیابی کمی دارند و مسئله مطلوبیت کاربر را بررسی می‌کنیم.

### مسائل سیستم

معیارهای عملی زیادی برای ارزیابی یک سیستم بازیابی اطلاعات فراتر از کیفیت بازیابی وجود دارد:

- **سرعت ایندکس‌گذاری:** سیستم چقدر سریع اسناد را ایندکس می‌کند؟ (مثلاً چند سند در ساعت برای توزیع مشخصی از طول اسناد)

#### 📺 مثال: آمازون وب سرویسز

آمازون وب سرویسز (AWS) برای سرویس CloudSearch خود، سرعت ایندکس‌گذاری را به عنوان یک معیار کلیدی بازیابی می‌کند. برای مشتریانی که با حجم عظیمی از داده‌ها کار می‌کنند، توانایی ایندکس کردن میلیون‌ها سند در روز یک مزیت رقابتی مهم است.

- **سرعت جستجو:** تأخیر (latency) سیستم به عنوان تابعی از اندازه ایندکس چقدر است؟

#### ⚡ مثال: بینگ

بینگ به طور مداوم تأخیر جستجو را در سراسر جهان اندازه‌گیری می‌کند. هر بهبود ۱۰۰ میلی‌ثانیه‌ای در تأخیر جستجو می‌تواند منجر به افزایش قابل توجهی در تعداد جستجوهای روزانه شود، زیرا کاربران ترجیح می‌دهند نتایج را سریع‌تر دریافت کنند.

- **زبان پرسمان:** زبان پرسمان چقدر گویا و قدرتمند است؟ سیستم چقدر با پرسمان‌های پیچیده کار می‌کند؟

#### 🔍 مثال: Elasticsearch

Elasticsearch که توسط بسیاری از شرکت‌ها برای جستجوی داخلی استفاده می‌شود، به دلیل زبان پرسمان غنی خود که شامل کوئری‌های پیچیده، فیلترهای تو در تو و تجزیه و تحلیل متن است، محبوب است. این قابلیت به کاربران امکان می‌دهد تا جستجوهای دقیق‌تری انجام دهند.

- **اندازه مجموعه اسناد:** مجموعه اسناد چقدر بزرگ است و آیا اطلاعات در طیف گسترده‌ای از موضوعات توزیع شده است؟

#### مثال: ویکی‌پدیا

ویکی‌پدیا نه تنها به دلیل حجم عظیم اسناد خود (میلیون‌ها مقاله)، بلکه به دلیل پوشش گسترده موضوعات از علم تا فرهنگ عامه، یک منبع اطلاعاتی ارزشمند است. این پوشش گسترده باعث می‌شود کاربران بتوانند به طیف وسیعی از نیازهای اطلاعاتی خود پاسخ دهند.

همه این معیارها به جز زبان پرسمان، به راحتی قابل اندازه‌گیری هستند: می‌توانیم سرعت یا اندازه را کمی کنیم. انواع مختلف چک‌لیست‌های ویژگی می‌توانند گویایی زبان پرسمان را نیمه‌دقیق کنند.

## ۸.۶.۲ سودمندی کاربر

آنچه واقعاً به دنبال آن هستیم، راهی برای سنجش میزان رضایت کلی کاربران است، بر اساس مرتبط بودن نتایج، سرعت سیستم و رابط کاربری. بخشی از این فرآیند، درک توزیع افرادی است که می‌خواهیم آن‌ها را راضی نگه داریم، که این موضوع کاملاً به بستر مورد استفاده بستگی دارد.

#### مثال: گوگل و وفاداری کاربران

برای یک موتور جستجوی وب مانند گوگل، کاربران راضی کسانی هستند که آنچه را می‌خواهند پیدا می‌کنند. یک معیار غیرمستقیم برای سنجش این رضایت، نرخ بازگشت کاربران به همان موتور جستجو است. گوگل این معیار را به دقت اندازه‌گیری می‌کند، زیرا کاربرانی که به طور مداوم از گوگل استفاده می‌کنند، معمولاً نتایج را مفید می‌یابند. این معیار حتی مؤثرتر می‌شود اگر بتوانیم میزان استفاده کاربران از موتورهای جستجوی دیگر را نیز اندازه‌گیری کنیم.

اما کاربران مدرن موتورهای جستجوی وب فقط کاربران نهایی نیستند؛ **تبلیغ‌دهندگان** نیز کاربران این سیستم‌ها هستند. آن‌ها زمانی راضی هستند که مشتریان روی تبلیغاتشان کلیک کرده و سپس خرید انجام دهند.

#### مثال: دیجی‌کالا و نرخ تبدیل

در یک وب‌سایت تجارت الکترونیک مانند دیجی‌کالا، کاربر احتمالاً به دنبال خرید چیزی است. بنابراین، می‌توانیم معیارهایی مانند **زمان تا خرید** یا **درصد جستجوکنندگانی که به خریدار تبدیل می‌شوند** را اندازه‌گیری کنیم. دیجی‌کالا این معیارها را به دقت رصد می‌کند تا بفهمد کدام الگوریتم‌های جستجو به بیشترین فروش منجر می‌شوند.

#### مثال: جستجوی داخلی در شرکت‌ها

برای یک موتور جستجوی داخلی در یک سازمان (شرکت، دولت یا دانشگاه)، معیار مرتبط احتمالاً **بهره‌وری کاربر** است: کاربران چقدر زمان برای پیدا کردن اطلاعاتی که نیاز دارند صرف می‌کنند؟ در مایکروسافت، سیستم جستجوی داخلی بر اساس این

معیار بهینه‌سازی شده است تا کارمندان بتوانند سریع‌تر به اسناد مورد نیاز خود دسترسی پیدا کنند و زمان کمتری برای جستجو صرف کنند.

در حالت کلی، باید تصمیم بگیریم که آیا در تلاش برای بهینه‌سازی، رضایت کاربر نهایی یا صاحب وب‌سایت تجارت الکترونیک را در نظر داریم. معمولاً، صاحب کسب‌وکار است که به ما پول می‌پردازد.

## چالش‌های اندازه‌گیری رضایت کاربر

رضایت کاربر اندازه‌گیری مشکل و گریزپذیری است و این یکی از دلایلی است که روش‌شناسی استاندارد از نماینده مرتبط بودن نتایج جستجو استفاده می‌کند. روش مستقیم استاندارد برای دستیابی به رضایت کاربر، اجرای مطالعات کاربری است که در آن افراد در وظایف شرکت می‌کنند، معمولاً معیارهای مختلفی اندازه‌گیری می‌شود، از شرکت‌کنندگان مشاهده می‌شود و از تکنیک‌های مصاحبه قوم‌نگاری برای کسب اطلاعات کیفی در مورد رضایت استفاده می‌شود.

### مثال: مطالعات کاربری در نتلیکس

نتلیکس به طور منظم مطالعات کاربری انجام می‌دهد تا رضایت کاربران از سیستم توصیه فیلم را ارزیابی کند. در این مطالعات، کاربران وظایفی مانند پیدا کردن فیلم‌های جدید برای تماشا را انجام می‌دهند و زمان تکمیل وظیفه، رضایت از نتایج و تعامل با رابط کاربری اندازه‌گیری می‌شود. این مطالعات به نتلیکس کمک می‌کند تا الگوریتم‌های توصیه خود را بهبود بخشد.

مطالعات کاربری در طراحی سیستم بسیار مفید هستند، اما انجام آن‌ها زمان‌بر و پرهزینه است. همچنین انجام خوب این مطالعات دشوار است و برای طراحی مطالعات و تفسیر نتایج به تخصص نیاز است. ما در اینجا جزئیات تست قابلیت استفاده انسانی را مورد بحث قرار نخواهیم داد.

## ۸.۶.۳ بهبود سیستم‌های مستقر

تصور کنید تیمی در **گوگل** در حال بهبود الگوریتم جستجوی خود است. با میلیون‌ها کاربر در سراسر جهان، چگونه می‌توانند با اطمینان بگویند که تغییر پیشنهادی واقعاً بهتر عمل می‌کند؟ اینجاست که آزمایش‌های زنده (Live Testing) وارد عمل می‌شوند.

### مثال واقعی: تست A/B در گوگل

گوگل به طور مداوم تغییرات کوچکی در الگوریتم جستجوی خود آزمایش می‌کند. برای مثال، آنها ممکن است وزن داده شده به عنوان صفحه (title tag) را در الگوریتم رتبه‌بندی تغییر دهند. سپس ۱٪ از کاربران به صورت تصادفی به نسخه جدید هدایت می‌شوند، در حالی که ۹۹٪ باقی‌مانده از نسخه اصلی استفاده می‌کنند. با اندازه‌گیری نرخ کلیک روی نتایج برتر، می‌توانند با اطمینان بگویند که آیا این تغییر مثبت بوده است یا خیر.

## تست A/B: روش استاندارد صنعت

رایج‌ترین روش آزمایش، **تست A/B** است - اصطلاحی برگرفته از صنعت تبلیغات که در حال حاضر در تمام شرکت‌های فناوری بزرگ استفاده می‌شود. در این روش، فقط یک پارامتر بین سیستم فعلی و نسخه پیشنهادی تغییر می‌کند، و درصد کوچکی از کاربران (معمولاً ۱ تا ۱۰٪) به صورت تصادفی به نسخه جدید هدایت می‌شوند.

آمازون از تست A/B برای بهینه‌سازی صفحه نتایج جستجوی خود استفاده می‌کند. برای مثال، آنها ممکن است ترتیب نمایش محصولات را تغییر دهند: در نسخه A، محصولات بر اساس محبوبیت مرتب می‌شوند، در حالی که در نسخه B، محصولات بر اساس امتیاز کاربران مرتب می‌شوند. سپس نرخ تبدیل (درصد کاربرانی که خرید انجام می‌دهند) در هر دو نسخه اندازه‌گیری می‌شود. اگر نسخه B نرخ تبدیل بالاتری داشته باشد، به تدریج برای همه کاربران اجرا می‌شود.

در این نوع آزمایش‌ها، معیارهای کلیدی معمولاً شامل موارد زیر است:

- **نرخ کلیک روی نتیجه اول:** چه درصدی از کاربران روی اولین نتیجه کلیک می‌کنند؟
- **نرخ کلیک روی هر نتیجه در صفحه اول:** توزیع کلیک‌ها در صفحه اول نتایج چگونه است؟
- **زمان ماندن در صفحه:** کاربران چقدر طولانی در صفحه نتایج باقی می‌مانند؟
- **نرخ تبدیل:** چه درصدی از کاربران اقدام مورد نظر (مانند خرید یا ثبت‌نام) را انجام می‌دهند؟

این روش به تحلیل لاگ کلیک (Clickthrough Log Analysis) معروف است:

$$\text{Clickthrough Rate} = \frac{\text{Number of Clicks}}{\text{Number of Impressions}} \times 100\%$$

## مزایا و محدودیت‌های تست A/B

پایه تست A/B اجرای مجموعه‌ای از آزمایش‌های تک متغیره است - هر بار فقط یک پارامتر تغییر می‌کند. این باعث می‌شود اثر هر تغییر به وضوح قابل مشاهده باشد. با داشتن تعداد زیاد کاربر، حتی تغییرات بسیار کوچک نیز قابل اندازه‌گیری هستند.

## 🎬 مثال: نتفلیکس و آزمایش‌های چندمتغیره

نتفلیکس علاوه بر تست‌های A/B، از آزمایش‌های چند متغیره نیز استفاده می‌کند. برای مثال، آنها ممکن است همزمان چند پارامتر را در الگوریتم توصیه فیلم تغییر دهند: اندازه تصاویر، توضیحات فیلم، و دکمه‌های فراخوان. سپس با استفاده از تحلیل آماری چندمتغیره مانند رگرسیون خطی چندگانه، تأثیر هر تغییر را به صورت جداگانه و ترکیبی ارزیابی می‌کنند.

در اصل، می‌توان با تغییر همزمان چند پارامتر به صورت تصادفی و تحلیل آماری چندمتغیره، قدرت تحلیلی بیشتری به دست آورد. اما در عمل، تست A/B محبوب‌تر است، زیرا:

- **پیاده‌سازی آسان:** نیازی به زیرساختار پیچیده ندارد
- **تفسیر ساده:** نتایج به وضوح قابل درک هستند
- **قابل توضیح برای مدیریت:** می‌توان به سادگی به مدیران توضیح داد که چرا یک تغییر مثبت یا منفی بوده است

## 📊 مثال: تصمیم‌گیری داده‌محور در فیسبوک

فیسبوک از تست A/B برای بهبود الگوریتم نمایش پست‌ها در فید خبری استفاده می‌کند. با آزمایش تغییرات مختلف و اندازه‌گیری تعامل کاربران، آنها می‌توانند تصمیمات داده‌محور بگیرند. برای مثال، اگر تست نشان دهد که نمایش ویدیوها در بالای فید نرخ تعامل را افزایش می‌دهد، این تغییر به صورت دائمی اجرا می‌شود.

در مجموع، تست A/B ابزاری مؤثر برای بهبود تدریجی سیستم‌های IR در محیط واقعی است که به شرکت‌ها اجازه می‌دهد تا با ریسک کم، تغییرات مؤثری در سیستم‌های خود ایجاد کنند.

## ۸.۷ بریده‌های نتایج (Results Snippets)

وقتی در گوگل عبارت «اقتصاد پاپوا گینه نو» را جستجو می‌کنید، چه چیزی در صفحه نتایج می‌بینید؟ علاوه بر عنوان و لینک، یک متن کوتاه نمایش داده می‌شود که به شما کمک می‌کند بفهمید این صفحه به پرسمان شما مرتبط است یا نه. این متن کوتاه **بریده (snippet)** نام دارد و هدف آن کمک به کاربر برای تصمیم‌گیری در مورد مرتبط بودن سند است.

### مثال واقعی: گوگل و بریده‌های پویا

وقتی شما عبارت «اقتصاد پاپوا گینه نو» را جستجو می‌کنید، گوگل بریده‌هایی را نمایش می‌دهد که عبارت جستجو شده را در متن برجسته می‌کند. این بریده‌ها معمولاً از قسمتی از متن اصلی استخراج می‌شوند و به کاربر کمک می‌کنند تا بفهمند سند حاوی اطلاعات مورد نظر آن‌هاست یا خیر.

### انواع بریده‌ها: ایستا در برابر پویا

دو نوع اصلی بریده وجود دارد: **ایستا (Static)** و **پویا (Dynamic)**. بریده‌های ایستا همیشه یکسان هستند، در حالی که بریده‌های پویا بر اساس پرسمان کاربر سفارشی می‌شوند.

**بریده‌های ایستا:** معمولاً شامل بخشی از سند یا فراداده‌های مرتبط با سند هستند. ساده‌ترین شکل بریده، دو جمله اول یا ۵۰ کلمه اول سند است. به جای بخش‌های سند، می‌توان از فراداده‌های مرتبط با سند مانند عنوان، نویسنده یا توضیحات متا استفاده کرد.

### مثال: بریده‌های ایستا در کتابخانه‌های دیجیتال

در سیستم‌های کتابخانه دیجیتال مانند **پروژه گوتنبرگ**، بریده‌های ایستا معمولاً شامل اطلاعات کتاب‌شناختی مانند عنوان، نویسنده، سال انتشار و خلاصه‌ای از محتوای کتاب است. این اطلاعات در زمان ایندکس‌گذاری استخراج و ذخیره می‌شوند تا در هنگام نمایش نتایج جستجو، سریعاً در دسترس باشند.

**بریده‌های پویا:** تلاش می‌کنند توضیح دهند چرا سند خاصی برای پرسمان فعلی بازایی شده است. این بریده‌ها معمولاً یک یا چند «پنجره» از سند را نمایش می‌دهند که حاوی کلمات کلیدی پرسمان هستند. به این بریده‌ها «کلمات کلیدی در بافت (Keyword-in-Context)» یا KWIC نیز گفته می‌شود.

### مثال: بریده‌های پویا در بینگ

بینگ از تکنیک‌های پردازش زبان طبیعی برای تولید بریده‌های پویا استفاده می‌کند. وقتی شما عبارتی را جستجو می‌کنید، بینگ کلمات کلیدی را در متن برجسته می‌کند و تلاش می‌کند پنجره‌هایی از متن را نمایش دهد که به طور خاص به پرسمان شما پاسخ می‌دهند. این بریده‌ها معمولاً خواناتر و مفیدتر هستند زیرا زمینه را حفظ می‌کنند.

### چالش‌های فنی در تولید بریده‌ها

تولید بریده‌های پویا چالش‌های فنی خاصی دارد. اگر سیستم فقط ایندکس موقعیتی دارد، نمی‌تواند به راحتی بافت اطراف برخوردهای جستجو را بازسازی کند. این یکی از دلایل استفاده از بریده‌های ایستا است. راه حل استاندارد در دنیای دیسک‌های بزرگ و ارزان، ذخیره محلی تمام اسناد در زمان ایندکس‌گذاری است.

### مثال: بهینه‌سازی در گوگل ⚡

گوگل برای بهینه‌سازی تولید بریده‌ها، فقط پیشنوندهای ثابتی از اسناد را ذخیره می‌کند (مثلاً ۱۰,۰۰۰ کاراکتر). برای اکثر اسناد کوتاه، کل سند ذخیره می‌شود، اما ذخیره محلی عظیم برای اسناد بالقوه بزرگ هدر نمی‌رود. بریده‌های اسنادی که طولشان از اندازه پیشنهاد تجاوزی کند، تنها بر اساس مواد در پیشنهاد خواهد بود که به طور کلی یک منطقه مفید برای جستجوی خلاصه سند است.

اگر سندی از زمان آخرین پردازش توسط ایندکس‌گر به‌روز شده باشد، این تغییرات نه در کش و نه در ایندکس خواهند بود. در این شرایط، نه ایندکس و نه بریده به درستی محتوای فعلی سند را منعکس می‌کنند، اما تفاوت‌های بین بریده و محتوای واقعی سند برای کاربر نهایی آشکارتر خواهد بود.

### نکته کلیدی 📌

بریده‌های خوب باید ویژگی‌های زیر را داشته باشند: (۱) حداکثر اطلاعات مفید در مورد بحث کلمات کلیدی در سند، (۲) به اندازه کافی خود-contained برای خوانایی آسان، و (۳) به اندازه کافی کوتاه برای قرار گرفتن در محدودیت‌های فضای معمول برای بریده‌ها.

تولید بریده‌ها باید سریع باشد زیرا سیستم معمولاً برای هر پرسمان تعداد زیادی بریده تولید می‌کند. به جای ذخیره کل سند، معمولاً فقط پیشنوندهای ثابت و با اندازه سخاوند از سند ذخیره می‌شود.

## جمع‌بندی فصل ۸: ارزیابی در بازیابی اطلاعات 📊

در این فصل:

- معیارهای کلاسیک ارزیابی مانند accuracy، precision، recall، F-measure و برای بازیابی بدون رتبه بررسی کردیم.
- مفاهیم ارزیابی رتبه‌دار مانند MAP، Precision@k، R-Precision، NDCG و منحنی‌های precision-recall را تحلیل کردیم.
- مفهوم مرتبط بودن و چالش‌های ارزیابی انسانی را با ابزارهایی مانند آمار کاپا (kappa statistic) بررسی کردیم.
- نقدهایی بر مدل مرتبط بودن دودویی و پیشنهادهایی برای ارزیابی ترتیبی و مرتبط بودن حاشیه‌ای ارائه شد.
- ابعاد سیستمی ارزیابی مانند سرعت ایندکس‌سازی، مقیاس‌پذیری، و زبان پرسمان را معرفی کردیم.
- روش‌های سنجش سودمندی کاربر و مطالعات کاربری را مرور کردیم.
- مفهوم تست A/B برای بهبود سیستم‌های مستقر و تحلیل رفتار کاربر را توضیح دادیم.
- نقش قطعه‌های نتایج (snippets) در تجربه کاربر و طراحی آن‌ها با روش‌های static و dynamic را بررسی کردیم.

این فصل پایه ارزیابی کمی و کیفی سیستم‌های IR را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- بازخورد ضمنی و یادگیری از رفتار کاربر (فصل ۹)
- مدل‌های احتمالاتی و زبان‌محور برای رتبه‌بندی (فصل ۱۱)
- یادگیری رتبه‌بندی و بهینه‌سازی سیستم‌های IR (فصل ۱۵)

آشنا خواهیم شد – که همگی بر پایهٔ ارزیابی دقیق و طراحی تجربهٔ کاربر بنا شده‌اند.

## End of Chapter Exercises

### مسئله 8.1: ارزیابی سیستم‌های بازیابی (IR system evaluation)

برای ارزیابی موثر یک سیستم بازیابی اطلاعات بدون رتبه‌بندی ( **Unranked** )، دو معیار اصلی مورد استفاده قرار می‌گیرند. این دو معیار را نام برده و فرمول محاسبه هر یک را بر اساس تعداد اسناد مرتبط و بازیابی شده بنویسید.

### مسئله 8.2: مجموعه‌های آزمون استاندارد (Standard test collections)

یک مجموعه تست استاندارد برای ارزیابی الگوریتم‌های بازیابی اطلاعات از چه اجزای حیاتی تشکیل شده است؟ توضیح دهید چرا غیبت هر یک از این اجزا، ارزیابی را ناممکن می‌سازد.

### مسئله 8.3: ارزیابی نتایج رتبه‌بندی شده (Evaluation of ranked retrieval results)

مفهوم "رتبه‌بندی" **Ranking** چه چالشی به معیارهای سنتی ارزیابی اضافه می‌کند و چرا معیارهایی که فقط بر اساس مجموعه‌های بدون رتبه کار می‌کنند، در اینجا کافی نیستند؟

### مسئله 8.4: چالش‌های قضاوت ارتباط (Assessing relevance)

ارتباط **Relevance** یک مفهوم ذهنی است. دو دلیل اصلی بیاورید که چرا توافق دو انسان بر سر مرتبط بودن یک سند با یک پرسمان ممکن است متفاوت باشد.

### مسئله 8.5: نقد مفهوم ارتباط (Critiques and justifications of the concept of relevance)

مفهوم «ارتباط حاشیه‌ای» **Marginal Relevance** را تعریف کنید و توضیح دهید چرا صرفاً بررسی ارتباط هر سند به صورت مستقل از سایر اسناد، ممکن است نتایج مطلوبی برای کاربر نداشته باشد.

### مسئله 8.6: کیفیت سیستم در برابر مطلوبیت (System quality and user utility)

تفاوت اساسی بین **System Quality** و **User Utility** در چیست؟ یک مثال بزنید که در آن کیفیت سیستم بالا باشد اما مطلوبیت کاربر پایین باشد.

### مسئله 8.7: مسائل سیستمی (System issues)

در ارزیابی یک سیستم مستقر، علاوه بر دقت نتایج، چه فاکتورهای سیستمی دیگری (حداقل دو مورد) برای بررسی عملکرد کلی موتور جستجو ارزیابی می‌شوند؟

### مسئله 8.8: خلاصه نتایج جستجو (Results snippets)

تفاوت بین اسنیپت‌های ایستا ( **Static** ) و پویا ( **Dynamic** ) در چیست؟ موتورهای جستجوی امروزی بیشتر تمایل به استفاده از کدام نوع دارند و چرا؟

### مسئله 8.9: محاسبه معیار F1 (F1 Measure Calculation)

یک موتور جستجو برای یک پرسمان، 100 سند بازیابی کرده است که 40 تای آنها مرتبط هستند. اگر کل اسناد مرتبط در مجموعه 60 عدد باشد، مقدار **Precision** ، **Recall** و **F1** را محاسبه کنید.

### مسئله 8.10: ارزیابی در نقاط مشخص (Precision at k)

مفهوم **Precision@K** را تعریف کنید. چرا در موتورهای جستجوی وب، معیار **P@10** بسیار مهم‌تر از **Recall** کل سیستم ارزیابی می‌شود؟

### مسئله 8.11: نمودار دقت-بازیابی (Precision-Recall curve)

روند تغییرات در یک نمودار استاندارد **Precision-Recall** چگونه است؟ چرا معمولاً با افزایش **Recall** ، شاهد افت **Precision** در نمودار هستیم؟

### مسئله 8.12: میانگین دقت متوسط (Mean Average Precision)

الگوریتم ما برای یک پرسمان، اسناد زیر را به ترتیب بازیابی کرده است ( **R** =مرتبط، **N** =نامرتبط):

R, N, R, R, N, N, R

اگر کل اسناد مرتبط برای این پرسمان 5 عدد باشد، **Average Precision (AP)** را برای این پرسمان به دست آورید.

### مسئله 8.13: مطلوبیت کاربر در عمل (User utility practical assessment)

برای سنجش مطلوبیت کاربر بدون نیاز به قضاوت‌های انسانی مستقیم ( **Explicit Relevance Feedback** )، از چه روش‌های جایگزین و سیگنال‌های ضمنی ( **Implicit Signals** ) می‌توان استفاده کرد؟ سه مورد را نام ببرید.

### مسئله 8.14: بهینه‌سازی سیستم مستقر (Refining a deployed system)

در یک موتور جستجوی در حال کار، چگونه از **Click Logs** برای بهبود مستمر الگوریتم رتبه‌بندی استفاده می‌شود؟ چالش **Position Bias** در این لاگ‌ها چیست؟

### مسئله 8.15: تولید اسنیپت‌های پویا (Dynamic Snippet Generation)

چه الگوریتم‌ها یا تکنیک‌های متنی برای تولید یک اسنیپت پویای باکیفیت استفاده می‌شود تا کلمات کلیدی پرسمان کاربر به بهترین شکل در بستر ( **Context** ) جمله نمایش داده شود؟

### مسئله 8.16: مجموعه تست کرنفیلد (Cranfield test collection)

نقش تاریخی مجموعه تست **Cranfield** در توسعه علم ارزیابی بازیابی اطلاعات چه بود و چرا امروزه برای ارزیابی موتورهای جستجوی وب از مجموعه‌هایی مانند **TREC** به جای آن استفاده می‌شود؟

### مسئله 8.17: محاسبه معیار R-Precision (R-Precision Calculation)

معیار **R-Precision** را تعریف کنید. اگر برای پرسمانی کلاً 12 سند مرتبط وجود داشته باشد و در 12 سند اول بازیابی شده توسط سیستم، 4 سند مرتبط حضور داشته باشد، مقدار **R-Precision** چقدر است؟

### مسئله 8.18: توافق داوران در ارتباط (Inter-judge agreement)

برای ساخت یک مجموعه تست جدید، اگر دو داور روی مرتبط بودن اسناد اختلاف نظر زیادی داشته باشند، چه راهکارهای استاندارد برای رسیدن به یک **Ground Truth** نهایی پیشنهاد می‌کنید؟

### مسئله 8.19: محاسبه کاپای کوهن برای ارزیابان (Cohen's Kappa for assessors) ★

شما 100 دانشجو را به عنوان ارزیاب برای ساخت یک دادگان جستجو به کار گرفته‌اید. برای یک نمونه‌ی 50 سندی، دو دانشجو (ارزیاب 1 و ارزیاب 2) نتایج زیر را ثبت کرده‌اند:

- موافقت روی مرتبط بودن: 15 سند
- موافقت روی نامرتب بودن: 20 سند
- ارزیاب 1 مرتبط و ارزیاب 2 نامرتب: 10 سند
- ارزیاب 1 نامرتب و ارزیاب 2 مرتبط: 5 سند

آماره کاپا (**Kappa**) را محاسبه کنید و تحلیل کنید که آیا این سطح از توافق برای یک مجموعه تست استاندارد قابل قبول است یا خیر.

### مسئله 8.20: سناریوی محاسبه NDCG (NDCG Calculation Scenario) ★

یک موتور جستجوی سازمانی ارتباط اسناد را با مقادیر  $\{0, 1, 2, 3\}$  می‌سنجد. برای پرسمان خاصی، 5 سند اول بازیابی شده دارای نمرات **[2, 0, 3, 1, 3]** هستند. اگر بدانیم بهترین رتبه‌بندی ممکن (Ideal) برای این پرسمان **[3, 3, 2, 1, 0]** است، مقدار **NDCG@5** را محاسبه کنید.

### مسئله 8.21: طراحی سیستم ارزیابی A/B (A/B Testing Design) ★

یک تغییر اساسی در الگوریتم وزن‌دهی **tf-idf** موتور جستجوی دانشگاه داده‌اید و می‌خواهید آن را تست کنید. یک تست **A/B** طراحی کنید و به طور دقیق مشخص کنید ترافیک کاربران چگونه تقسیم می‌شود و چه متریک‌هایی برای تعیین برنده نهایی این تست (با در نظر گرفتن **User Utility**) پایش خواهند شد؟

### مسئله 8.22: الگوریتم ارزیاب خودکار (Autograder Algorithm) ★

شما باید یک اسکریپت پایتون بنویسید که خروجی سیستم دانشجویان را به طور خودکار نمره‌دهی کند. تابعی بنویسید که **MAP** را برای مجموعه‌ای از پرسمان‌ها محاسبه کند. دقت کنید که مخرج کسر در محاسبه **AP**، باید تعداد کل اسناد مرتبط موجود در

### مسئله 8.23: بهینه‌سازی سیستم مستقر بر اساس رفتار (Refining a deployed system based on behavior) ★

پس از استقرار موتور جستجو، متوجه می‌شوید که کاربران در پرسمان‌های طولانی (Long-tail queries) مرتباً به صفحات دوم و سوم می‌روند اما در نهایت روی هیچ نتیجه‌ای کلیک نمی‌کنند (Abandonment). با توجه به مفاهیم ارزیابی کیفیت سیستم، مشکل را در کدام بخش از معماری جستجو (Indexing, Scoring, یا Snippet Generation) ردیابی می‌کنید؟ دلیل خود را شرح دهید.

### مسئله 8.24: چالش نتایج رتبه‌بندی با اطلاعات ناقص (Ranked retrieval evaluation with pooling) ★

در ساخت مجموعه‌های بزرگ مانند TREC، بررسی تمامی میلیون‌ها سند برای یک پرسمان غیرممکن است. تکنیک Pooling چگونه این مشکل را حل می‌کند؟ از نظر ریاضی استدلال کنید که آیا اسناد مرتبطی که در فرآیند Pooling جا می‌مانند، تاثیر مخربی روی مقایسه نسبی بین دو موتور جستجوی A و B در معیار MAP می‌گذارند یا خیر؟

### مسئله 8.25: تحلیل خطای اسنیپت‌ها در موتور جستجو (Snippet error analysis) ★

موتور جستجوی شما برای فایل‌های متنی طولانی (مانند مقالات) اسنیپت‌هایی تولید می‌کند که فاقد کلمات کلیدی کاربر هستند، در نتیجه کاربر روی نتایج کلیک نمی‌کند با وجود اینکه سند کاملاً مرتبط است. یک الگوریتم Sliding Window پیشنهاد دهید که با اسکن کردن سند، متراکم‌ترین بخش شامل عبارات پرسمان را برای نمایش در اسنیپت استخراج کند.

### مسئله 8.26: مطلوبیت کاربر و مدل‌های کلیک (User utility and Click models) ★

برای مدل‌سازی مطلوبیت کاربر، دو مدل رفتار کاربر داریم: «مدل آبشاری» (Cascade Model) و «مدل ناوبری» (Navigational Model). تفاوت فرض‌های این دو مدل در نحوه اسکن کردن نتایج جستجو توسط کاربر چیست؟ اگر قصد ارزیابی سیستمی را داشته باشید که مختص جستجوی کدهای برنامه‌نویسی است، کدام مدل رفتاری به واقعیت نزدیک‌تر است؟

### مسئله 8.27: سناریوی تغییر معماری بر اساس لاگ‌ها (Architecture shift based on logs) ★

تحلیل لاگ‌های جستجو نشان می‌دهد که در 50 درصد مواقع، کاربران پرسمان خود را پس از دیدن نتایج اولیه اصلاح می‌کنند (Query Reformulation). این موضوع چه نقصی را در سیستم ارزیابی فعلی شما (که فقط مبتنی بر پرسمان‌های تک‌مرحله‌ای است) نشان می‌دهد و چگونه مفاهیم ارزیابی جلسه (Session-based evaluation) را برای حل این مشکل پیاده‌سازی می‌کنید؟

### مسئله 8.28: طراحی جامع فریم‌ورک ارزیابی (Comprehensive Evaluation Framework Design) ★

شما مدیر ارزیابی یک پروژه ملی موتور جستجو هستید. باید یک پروتکل طراحی کنید که از سه فاز تشکیل شده باشد:

- ارزیابی آفلاین (با مجموعه تست)
- ارزیابی آنلاین (با A/B تست)

برای هر فاز دقیقاً یک متریک کلیدی (Key Metric) انتخاب کنید و دلیل این انتخاب را به طور تحلیلی شرح دهید.

## پاسخنامه مسئله 8.1: ارزیابی سیستم‌های بازیابی (IR System Evaluation)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، هدف ارزیابی عملکرد یک سیستم بازیابی اطلاعات (Information Retrieval) از نوع بدون رتبه‌بندی (Unranked) است. در سیستم‌های بدون رتبه‌بندی (مانند مدل بازیابی بولین)، سیستم برای یک پرسمان مشخص، تنها مجموعه‌ای از اسناد را برمی‌گرداند و هیچ‌گونه ترتیب یا امتیازی برای میزان ارتباط آن‌ها قائل نمی‌شود. خواسته مسئله، معرفی دو معیار اصلی ارزیابی در این سیستم‌ها و ارائه فرمول ریاضی آن‌ها بر پایه تعداد اسناد مرتبط و بازیابی‌شده است.

### بخش 2) مفاهیم کلیدی

برای ارزیابی کیفیت خروجی در سیستم‌های بدون رتبه‌بندی، مجموعه کل اسناد به چهار دسته تقسیم می‌شوند که پایه و اساس معیارهای ارزیابی را شکل می‌دهند:

**اسناد مرتبط و بازیابی شده (True Positives - TP):** اسنادی که کاربر به آن‌ها نیاز داشته و سیستم نیز آن‌ها را به درستی پیدا کرده است.

**اسناد نامرتب و بازیابی شده (False Positives - FP):** اسنادی که سیستم به اشتباه آن‌ها را آورده است، در حالی که کاربر نیازی به آن‌ها نداشته است (هرزنامه‌ها یا نویز).

**اسناد مرتبط و بازیابی نشده (False Negatives - FN):** اسنادی که کاربر به آن‌ها نیاز داشته اما سیستم در پیدا کردن آن‌ها ناکام مانده است (ریزش).

**اسناد نامرتب و بازیابی نشده (True Negatives - TN):** اسنادی که مرتبط نبوده‌اند و سیستم هم به درستی آن‌ها را نادیده گرفته است.

### بخش 3) معرفی معیارهای اصلی و نمادگذاری ریاضی

در سیستم‌های ارزیابی بدون رتبه‌بندی، دو معیار جهان‌شمول و استاندارد برای سنجش کارایی سیستم وجود دارد:

1. **دقت (Precision):** نشان می‌دهد چه کسری از اسناد بازیابی شده توسط سیستم، واقعاً با نیاز کاربر مرتبط هستند.

2. **فراخوانی (Recall):** نشان می‌دهد چه کسری از کل اسناد مرتبط موجود در پایگاه داده، توسط سیستم با موفقیت بازیابی شده‌اند.

برای نوشتن فرمول‌ها، ابتدا متغیرهای زیر را تعریف می‌کنیم:

**Retrieved:** کل اسناد بازیابی شده توسط سیستم.

**Relevant:** کل اسناد مرتبط با پرسمان کاربر در کل پایگاه داده.

**$Retrieved \cap Relevant$ :** اشتراک این دو مجموعه؛ یعنی اسنادی که هم بازیابی شده‌اند و هم مرتبط هستند.

### بخش 4) تدوین فرمول‌های ریاضی

## الف) فرمول دقت (Precision):

دقت، نسبت اسناد مرتبطِ بازیابی شده به کل اسناد بازیابی شده است. این معیار نشان‌دهنده میزان «خلوص» خروجی سیستم است.

$$Precision = \frac{|Retrieved \cap Relevant|}{|Retrieved|}$$

## ب) فرمول فراخوانی (Recall):

فراخوانی، نسبت اسناد مرتبطِ بازیابی شده به کل اسناد مرتبط موجود است. این معیار نشان‌دهنده میزان «جامعیت» و توانایی سیستم در یافتن تمام جواب‌های ممکن است.

$$Recall = \frac{|Retrieved \cap Relevant|}{|Relevant|}$$

## بخش 5) پاسخ نهایی

دو معیار اصلی برای ارزیابی سیستم‌های بازیابی اطلاعات بدون رتبه‌بندی عبارتند از:

$$1. Precision = \frac{|Retrieved \cap Relevant|}{|Retrieved|}$$

$$2. Recall = \frac{|Retrieved \cap Relevant|}{|Relevant|}$$

### نکته مهم:

به صورت تجربی در سیستم‌های بازیابی اطلاعات، همواره یک تقابل (**Trade-off**) میان «دقت» و «فراخوانی» وجود دارد. اگر سیستم را طوری تنظیم کنید که اسناد بسیار زیادی را برگرداند تا هیچ سند مرتبطی از قلم نیفتد (افزایش **Recall**)، ناگزیر اسناد نامرتبب زیادی نیز وارد خروجی می‌شوند که باعث افت دقت (**Precision**) می‌گردد. برعکس، اگر سیستم را بسیار سخت‌گیر کنید تا فقط اسناد کاملاً مطمئن را برگرداند (افزایش **Precision**)، ممکن است بسیاری از اسناد مرتبط را از دست بدهید (افت **Recall**). برای ایجاد تعادل بین این دو معیار، در مباحث پیشرفته‌تر از معیار ترکیبی به نام **F-Measure** استفاده می‌شود.

## پاسخنامه مسئله 8.2: مجموعه‌های آزمون استاندارد (Standard Test Collections)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، هدف شناخت زیرساخت‌های لازم برای ارزیابی کمی و علمی الگوریتم‌های بازیابی اطلاعات است. برای اینکه بتوانیم ادعا کنیم یک سیستم بازیابی بهتر از سیستم دیگر عمل می‌کند، به یک محیط آزمایشگاهی استاندارد نیاز داریم که در

علم بازیابی اطلاعات به آن «مجموعه آزمون» (**Test Collection**) گفته می‌شود. خواسته مسئله، شناسایی اجزای این مجموعه و تحلیل علت وابستگی ارزیابی به هر یک از آنهاست.

## بخش 2) مفاهیم کلیدی

ارزیابی در سیستم‌های بازیابی اطلاعات بر اساس مقایسه خروجی سیستم با یک «حقیقت پایه» (**Ground Truth**) انجام می‌شود. مجموعه‌های آزمون استاندارد این حقیقت پایه را فراهم می‌کنند تا مهندسان و پژوهشگران بتوانند الگوریتم‌های مختلف را در شرایط یکسان، پایدار و عادلانه بسنجند. این رویکرد که به ارزیابی مبتنی بر مجموعه (**Cranfield Paradigm**) معروف است، سنگ‌بنای توسعه سیستم‌های جستجو به شمار می‌رود.

## بخش 3) حل گام‌به‌گام

### گام اول: معرفی اجزای حیاتی مجموعه آزمون

یک مجموعه آزمون استاندارد در بازیابی اطلاعات، به صورت کلاسیک از 3 جزء جدایی‌ناپذیر تشکیل شده است:

1. مجموعه اسناد (Document Collection / Corpus)

2. مجموعه‌ای از نیازهای اطلاعاتی یا پرسمان‌ها (Information Needs / Queries)

3. قضاوت‌های ارتباط (Relevance Judgments / Qrels)

### گام دوم: بررسی علت ضروری بودن هر جزء (اثبات وابستگی)

اکنون نقش هر جزء و نتیجه غیبت آن را به لحاظ منطقی تحلیل می‌کنیم:

#### جزء اول: مجموعه اسناد (Corpus)

این جزء، پایگاه داده‌ای شامل متون یا موجودیت‌هایی است که سیستم باید آن‌ها را پردازش کرده و در آن‌ها جستجو کند.

نتیجه غیبت: بدون وجود اسناد، فضای جستجویی وجود نخواهد داشت. سیستم‌های بازیابی برای ساختاردهی، فشرده‌سازی اطلاعات و نمایه‌سازی (**Indexing**) به داده‌های خام نیاز دارند. آزمایش عملکرد یک الگوریتم جستجو در یک فضای خالی، به لحاظ منطقی بی‌معنا و ناممکن است.

#### جزء دوم: مجموعه پرسمان‌ها (Queries)

این بخش نمایانگر نیازهای اطلاعاتی کاربران فرضی است که به زبان سیستم (مثلاً کلمات کلیدی یا جملات طبیعی) ترجمه شده است و سیستم باید به آن‌ها پاسخ دهد.

نتیجه غیبت: بدون پرسمان، هیچ ورودی یا محرکی برای سیستم وجود ندارد تا خروجی تولید کند. الگوریتم‌های بازیابی بر اساس تطابق پرسمان با اسناد کار می‌کنند و در صورت نبود پرسمان، سیستم در حالت سکون باقی مانده و ارزیابی آن ممکن نیست.

#### جزء سوم: قضاوت‌های ارتباط (Relevance Judgments)

مجموعه‌ای از داده‌هاست که معمولاً به صورت دستی توسط انسان‌های متخصص ارزیابی و تهیه می‌شود. این جزء مشخص می‌کند برای هر پرسمان خاص، کدام اسناد در مجموعه اسناد، «مرتبط» و کدام «نامرتبط» هستند.

نتیجه غیبت: این بخش، هسته مرکزی و حیاتی ارزیابی است. اگر ما (به عنوان ارزیاب) از پیش ندانیم کدام اسناد واقعاً با پرسمان ارتباط دارند، محاسبه معیارهای ارزیابی مانند دقت (**Precision**) و فراخوانی (**Recall**) کاملاً غیرممکن

می‌شود؛ زیرا هیچ مبنای صحیحی برای تشخیص درست یا اشتباه بودن اسنادی که سیستم بازگردانده است، در اختیار نخواهیم داشت.

## بخش 4) پاسخ نهایی

یک مجموعه آزمون استاندارد در ارزیابی سیستم‌های بازیابی اطلاعات، الزاماً از 3 جزء اساسی تشکیل می‌شود که غیبت هر یک روند ارزیابی را مختل می‌کند:

1. **مجموعه اسناد:** محیط جستجو را فراهم می‌کند؛ بدون آن داده‌ای برای پردازش، نمایه‌سازی و بازیابی وجود ندارد.

2. **مجموعه پرسمان‌ها:** نیاز اطلاعاتی کاربر را شبیه‌سازی می‌کند؛ بدون آن سیستم ورودی و درخواستی برای پاسخگویی و جستجو دریافت نمی‌کند.

3. **قضاوت‌های ارتباط:** حقیقت پایه (**Ground Truth**) را شکل می‌دهد؛ بدون آن هیچ معیار مرجعی برای سنجش صحت اسناد بازیابی‌شده و محاسبه ریاضی دقت و فراخوانی وجود نخواهد داشت.

### نکته:

باید توجه داشته باشید که گران‌ترین و چالش‌برانگیزترین بخش در ساخت یک مجموعه آزمون، تهیه «قضاوت‌های ارتباط» است. این فرآیند نیازمند صرف زمان و هزینه فراوان توسط نیروی انسانی متخصص برای مطالعه اسناد و تصمیم‌گیری درباره میزان ارتباط آن‌هاست. به همین دلیل، در پروژه‌های آکادمیک و سیستم‌های تجاری، به جای ساخت مجموعه از پایه، معمولاً از مجموعه‌های آزمون استاندارد و از پیش آماده‌شده مانند **TREC (Text REtrieval Conference)** استفاده می‌شود که توسط موسسات معتبر جهانی توسعه یافته‌اند. این کار ضمن صرفه‌جویی در زمان، امکان مقایسه دقیق الگوریتم‌های طراحی‌شده توسط شما را با دستاوردهای سایر پژوهشگران فراهم می‌کند.

## پاسخنامه مسئله 8.3: ارزیابی نتایج رتبه‌بندی شده (Evaluation of ranked retrieval results)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، با تغییر پارادایم از سیستم‌های «مبتنی بر مجموعه» (**Set-based**) به سیستم‌های «مبتنی بر رتبه‌بندی» (**Ranked**) مواجه هستیم. هدف مسئله، درک نارسایی معیارهای کلاسیک (مانند آنچه در سیستم‌های بدون رتبه استفاده می‌شود) در مواجهه با خروجی‌های مرتب‌شده، و شناسایی چالش جدیدی است که مفهوم «رتبه» به فرآیند ارزیابی اضافه می‌کند.

### بخش 2) مفاهیم کلیدی

**بازیابی بدون رتبه‌بندی (Unranked Retrieval):** خروجی سیستم تنها یک مجموعه بی‌نظم از اسناد است. در این حالت فقط «بودن» یا «نبودن» سند مرتبط در خروجی اهمیت دارد.

**بازیابی رتبه‌بندی شده (Ranked Retrieval):** خروجی سیستم یک لیست مرتب‌شده بر اساس میزان ارتباط (**Relevance Score**) است. در اینجا، رفتار کاربر نقش مهمی ایفا می‌کند؛ کاربران معمولاً فقط به چند نتیجه اول نگاه می‌کنند و اسنادی که در رتبه‌های پایین‌تر قرار می‌گیرند، عملاً ارزش و شانس دیده شدن بسیار کمتری دارند.

### بخش 3) حل گام‌به‌گام

## گام اول: تحلیل رفتار معیارهای سنتی

معیارهایی مانند دقت (**Precision**) و فراخوانی (**Recall**) بر روی کل مجموعه اسناد بازبایی شده به صورت یکپارچه محاسبه می‌شوند. در این معیارها، خروجی سیستم به عنوان یک «کیسه اسناد» (**Bag of Documents**) در نظر گرفته می‌شود که در آن هیچ ترتیبی وجود ندارد و جایگاه اسناد فاقد ارزش محاسباتی است.

## گام دوم: بررسی یک مثال نقض برای درک نارسایی معیارهای سنتی

فرض کنید کاربری پرسمانی را جستجو می‌کند و سیستم 100 سند را به عنوان خروجی برمی‌گرداند که از بین آن‌ها، تنها 10 سند واقعاً با نیاز کاربر مرتبط هستند.

**حالت الف (سیستم ایده‌آل):** سیستم به قدری هوشمند است که این 10 سند مرتبط را دقیقاً در رتبه‌های 1 تا 10 (ابتدای لیست) قرار می‌دهد.

**حالت ب (سیستم ضعیف):** سیستم این 10 سند مرتبط را در رتبه‌های 91 تا 100 (انتهای لیست) پنهان می‌کند.

اگر از معیارهای سنتی استفاده کنیم، در هر دو حالت فرمول دقت مقدار 0.1 (یعنی 10 تقسیم بر 100) و فرمول فراخوانی نیز مقدار کاملاً یکسانی را نشان می‌دهد. سیستم ارزیاب این دو حالت را برابر می‌داند، در حالی که برای کاربر، «حالت الف» یک تجربه جستجوی عالی و «حالت ب» یک تجربه کاملاً شکست‌خورده است.

## گام سوم: استخراج چالش اصلی

چالشی که رتبه‌بندی ایجاد می‌کند، لزوم ارزش‌گذاری بر اساس جایگاه (**Positional Value**) است. سیستمی که اسناد مرتبط را در رتبه‌های بالاتر قرار می‌دهد، باید امتیاز بیشتری در ارزیابی دریافت کند. معیارهای سنتی نسبت به این موضوع کور هستند و قابلیت درک مکان یا رتبه سند را در لیست خروجی ندارند.

## بخش 4) پاسخ نهایی

مفهوم "رتبه‌بندی"، چالش «وابستگی ارزش سند به جایگاه آن» را به ارزیابی اضافه می‌کند. معیارهای سنتی (مانند دقت و فراخوانی پایه) در اینجا کافی نیستند؛ زیرا خروجی را به عنوان یک مجموعه بدون ترتیب ارزیابی می‌کنند و بین سیستمی که اسناد مرتبط را در ابتدای لیست (در معرض دید کاربر) نمایش می‌دهد با سیستمی که آن‌ها را در انتهای لیست قرار می‌دهد، هیچ تفاوتی قائل نمی‌شوند. در یک سیستم رتبه‌بندی شده، معیار ارزیابی باید قادر باشد به یافتن زود هنگام اسناد مرتبط پاداش دهد.

### نکته :

برای حل این مشکل و ارزیابی دقیق سیستم‌های رتبه‌بندی شده، علم بازبایی اطلاعات معیارهای جدیدی را معرفی کرده است که به "رتبه" حساس هستند. یکی از ساده‌ترین راهکارها، محاسبه معیار در یک نقطه برش مشخص است که به آن **Precision@K** (دقت در K سند اول) می‌گویند (مثلاً محاسبه دقت فقط برای 10 نتیجه اول جستجو یا **P@10**). برای ارزیابی‌های جامع‌تر و دقیق‌تر که کل لیست را با در نظر گرفتن وزن رتبه‌ها بررسی می‌کنند، از معیارهای پیشرفته‌تری مانند میانگین میانگین دقت (**MAP**) و بهره تجمعی تخفیف‌یافته نرمال‌شده (**NDCG**) استفاده می‌شود. این معیارها جایگاه هر سند مرتبط را به عنوان یک ضریب کاهنده (**Discount**) در محاسبه نهایی دخالت می‌دهند؛ به این معنا که هرچه سند مرتبط در رتبه پایین‌تری ظاهر شود، تاثیر کمتری در امتیاز نهایی سیستم خواهد داشت.

## پاسخنامه مسئله 8.4: چالش‌های قضاوت ارتباط (**Assessing relevance**)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما با یکی از بنیادین‌ترین و چالش‌برانگیزترین مفاهیم در ارزیابی سیستم‌های بازیابی اطلاعات، یعنی «قضاوت ارتباط» روبرو هستیم. همان‌طور که پیش‌تر در مسئله مجموعه آزمون‌ها بررسی کردیم، تعیینِ مرتبط بودن یا نبودن اسناد بر عهده انسان است. خواسته مسئله این است که چرا این قضاوت انسانی یک فرآیند عینی و قطعی نیست و دو ارزیاب ممکن است در مواجهه با یک پرسمان و یک سند یکسان، به نتایج متفاوتی (عدم توافق) برسند.

## بخش 2) مفاهیم کلیدی

«ارتباط» در ذات خود یک ویژگی ریاضی یا فیزیکی نیست که بتوان آن را به صورت مطلق محاسبه کرد، بلکه یک تجربه شناختی و ذهنی **(Subjective)** است. در ایجاد یک مجموعه آزمون، میزان سازگاری و هم‌رأی بودن ارزیاب‌ها **(Inter-Annotator Agreement)** بسیار حیاتی است. عدم توافق بین ارزیاب‌ها، قابلیت اتکای مجموعه آزمون را کاهش می‌دهد. برای درک دلایل این اختلاف نظر، باید به ماهیت زبان، نیاز کاربر و پیش‌زمینه ذهنی ارزیاب توجه کنیم.

## بخش 3) حل گام‌به‌گام

### گام اول: تحلیل تفاوت در پیشینه دانش و تخصص ارزیاب‌ها

اولین دلیل اصلی، تفاوت در سطح دانش و تخصص ارزیاب‌هاست. ارزیاب‌ها ماشین نیستند و هر کدام با دانش و تجربه قبلی خود متن را تفسیر می‌کنند.

**سناریو:** فرض کنید پرسمان در مورد «تاثیرات تورم بر بازار مسکن» است. یک ارزیاب که متخصص اقتصاد است، ممکن

است یک مقاله پیچیده و پر از اصطلاحات تخصصی را بسیار «مرتبط» و مفید تشخیص دهد. در مقابل، یک ارزیاب با

دانش عمومی، همان مقاله را غیرقابل فهم ارزیابی کرده و آن را «نامرتبط» یا کم‌ارزش برای کاربر عادی تلقی کند.

### گام دوم: تحلیل ابهام در نیاز اطلاعاتی (ابهام پرسمان)

دلیل دوم به ماهیت خود پرسمان‌ها برمی‌گردد. پرسمان‌ها معمولاً بسیار کوتاه هستند و تمام ابعاد نیاز ذهنی کاربر را بیان نمی‌کنند (به این حالت نقص در بیان یا **Query Formulation Problem** می‌گویند).

**سناریو:** پرسمانی به شکل تک‌کلمه‌ای مانند "جاوا" (**Java**) را در نظر بگیرید. یک ارزیاب ممکن است نیاز اطلاعاتی نهفته

در این پرسمان را مرتبط با «زبان برنامه‌نویسی جاوا» تصور کند و اسناد برنامه‌نویسی را مرتبط بداند. ارزیاب دیگر ممکن

است تصور کند هدف کاربر اطلاعاتی درباره «جزیره جاوا» یا «قهوه جاوا» بوده است و بر همین اساس قضاوت متفاوتی

انجام دهد. هر ارزیاب، قصدِ پنهان کاربر را به شکل متفاوتی تفسیر می‌کند.

## بخش 4) پاسخ نهایی

ارتباط یک مفهوم عینی نیست و به دلیل دخالت ذهنیت انسان، ممکن است دو ارزیاب درباره یک سند هم‌نظر نباشند. دو دلیل اصلی این عدم توافق عبارتند از:

**1. تفاوت در دانش پس‌زمینه و تخصص:** تفسیر ارزش و کاربردی بودن یک سند، به شدت به سطح دانش فرد ارزیاب

بستگی دارد. یک سند تخصصی ممکن است برای یک ارزیابِ متخصص کاملاً مرتبط و ارزشمند باشد، اما از دید یک

ارزیاب عامی، به دلیل پیچیدگی بالا، نامرتبط و غیرمفید ارزیابی شود.

**2. ابهام در تفسیر پرسمان (نیاز اطلاعاتی نامشخص):** از آنجا که پرسمان‌ها (به ویژه پرسمان‌های کوتاه) اغلب ناتوان از

بیان کامل نیاز واقعی کاربر هستند، ارزیاب‌ها مجبورند نیت پنهان در پشت پرسمان را حدس بزنند. برداشت‌های

متفاوت ارزیاب‌ها از مقصود واقعی کاربر (مثلاً کلمات چندمعنایی)، منجر به قضاوت‌های متفاوتی درباره ارتباط اسناد خواهد شد.

#### نکته:

برای کنترل این ذهنیت‌گرایی و افزایش اعتبار مجموعه‌های آزمون، متخصصان از معیاری آماری به نام ضریب کاپای کوهن (**Cohen's Kappa Coefficient**) استفاده می‌کنند. این معیار میزان توافق بین دو ارزیاب را، با حذف احتمال «توافق‌های تصادفی»، محاسبه می‌کند. اگر مقدار کاپا بسیار پایین باشد، نشان می‌دهد که تعریف هدف یا دستورالعمل‌های ارزیابی (**Annotation Guidelines**) دارای نقص است و باید بازنویسی شود تا ارزیاب‌ها به درک مشترکی از مسئله برسند.

## پاسخنامه مسئله 8.5: نقد مفهوم ارتباط

### (Critiques and justifications of the concept of relevance)

#### بخش 1) تحلیل اولیه مسئله

در این مسئله، یکی از فرضیات کلاسیک و بنیادین سیستم‌های بازیابی اطلاعات، یعنی «استقلال اسناد از یکدیگر»، مورد نقد و بررسی قرار می‌گیرد. در مدل‌های سنتی، سیستم فرض می‌کند میزان ارتباط یک سند با پرسمان، هیچ وابستگی به سایر اسنادی که به کاربر نشان داده می‌شود ندارد. خواسته مسئله این است که مفهوم جدیدی به نام «ارتباط حاشیه‌ای» را که در نقطه مقابل این فرض قرار دارد تعریف کنیم و دلیل ناکارآمدی بررسی مستقل اسناد را از دیدگاه تجربه کاربر (**User Experience**) تحلیل نماییم.

#### بخش 2) مفاهیم کلیدی

**ارتباط مستقل (Independent Relevance):** امتیازی که سیستم به تنهایی به یک سند اختصاص می‌دهد، فارغ از

اینکه چه اسناد دیگری در لیست خروجی وجود دارند.

**تنوع و تازگی (Diversity and Novelty):** نیاز کاربر به دریافت جنبه‌های مختلف یک موضوع و اطلاعات جدید، به جای

دریافت اطلاعات تکراری.

**افزونگی اطلاعات (Information Redundancy):** وضعیتی که در آن چندین سند مختلف، محتوای اطلاعاتی دقیقاً

یکسانی را ارائه می‌دهند.

#### بخش 3) حل گام‌به‌گام

##### گام اول: بررسی سناریوی ارزیابی مستقل و درک مشکل آن

فرض کنید کاربری پرسمان «نتایج انتخابات ریاست جمهوری» را جستجو می‌کند. اگر سیستم اسناد را صرفاً به صورت مستقل ارزیابی کند، ممکن است 10 خبر اولی که به کاربر نشان می‌دهد، همگی متن دقیقاً یکسانی باشند که از خبرگزاری‌های مختلف کپی و بازنشر (**Syndicate**) شده‌اند. در ارزیابی سنتی، هر 10 سند کاملاً «مرتبط» هستند و سیستم امتیاز کامل می‌گیرد؛ اما کاربر پس از خواندن سند اول، هیچ اطلاعات جدیدی از 9 سند بعدی به دست نمی‌آورد و به شدت از سیستم ناراضی خواهد بود.

##### گام دوم: تعریف ارتباط حاشیه‌ای (Marginal Relevance)

برای حل مشکل گام اول، مفهوم «ارتباط حاشیه‌ای» معرفی می‌شود. ارتباط حاشیه‌ای به معنای «میزان اطلاعات جدید و مفیدی است که یک سند، با در نظر گرفتن اسنادی که پیش‌تر توسط کاربر دیده شده، به او ارائه می‌دهد».

در سناریوی انتخابات، سند اول بالاترین ارتباط حاشیه‌ای را دارد، اما اسناد دوم تا دهم (با وجود اینکه ذاتاً به موضوع مرتبط‌اند)، چون هیچ حرف تازه‌ای برای گفتن ندارند، ارزش حاشیه‌ای آن‌ها برابر با «صفر» خواهد بود.

## گام سوم: تحلیل دلیل عدم مطلوبیت ارزیابی مستقل

بررسی مستقل اسناد به 2 دلیل عمده نتایج مطلوبی برای کاربر به همراه ندارد:

**1. تولید افزونگی (Redundancy):** سیستم وقت و انرژی کاربر را با نمایش محتوای تکراری هدر می‌دهد. کاربر به جای

خواندن یک مفهوم تکراری در چندین قالب، ترجیح می‌دهد اطلاعات جدید کسب کند.

**2. عدم پوشش ابهام (Lack of Diversity):** بسیاری از پرسمان‌ها مبهم یا چندوجهی هستند (مانند پرسمان "اپل" که

می‌تواند میوه یا شرکت فناوری باشد). اگر سیستم فقط به صورت مستقل عمل کند، ممکن است تمام نتایج را به یک

وجه خاص (مثلاً فقط گوشی‌های آیفون) اختصاص دهد. بررسی حاشیه‌ای سیستم را مجبور می‌کند برای جلب رضایت

کاربر، تنوع (Diversity) را در نتایج لحاظ کند تا شانس پوشش دادن نیت واقعی کاربر افزایش یابد.

## بخش 4) پاسخ نهایی

**تعریف ارتباط حاشیه‌ای (Marginal Relevance):** سودمندی و ارزش اطلاعاتی افزوده‌ای است که یک سند جدید، علاوه بر آنچه

کاربر از اسناد قبلی موجود در لیست نتایج به دست آورده است، ارائه می‌دهد. به عبارت ساده‌تر، یک سند زمانی ارتباط حاشیه‌ای

بالایی دارد که حاوی اطلاعاتی «مرتبط» اما «جدید و غیرتکراری» باشد.

**دلیل ناکارآمدی ارزیابی مستقل:** ارزیابی صرف و مستقل اسناد بدون توجه به سایر نتایج، منجر به پدیده «افزونگی اطلاعات»

می‌شود. در این حالت، سیستم ممکن است چندین سند بسیار مرتبط اما با محتوای کاملاً تکراری را به کاربر نشان دهد. کاربر

پس از مطالعه سند اول، نیازی به اسناد تکراری بعدی ندارد؛ در نتیجه، لیست خروجی فاقد «تنوع» و «تازگی» لازم بوده و

نمی‌تواند تجربه جستجوی کارآمد و مطلوبی را برای کاربر رقم بزند.

### نکته:

برای پیاده‌سازی مفهوم ارتباط حاشیه‌ای در الگوریتم‌های موتورهای جستجو، از تکنیکی بسیار معروف به نام **MMR (Maximal Marginal Relevance)** استفاده می‌شود. الگوریتم MMR در زمان چینش لیست نتایج، برای هر سند 2 فاکتور را به صورت همزمان در یک تابع ریاضی محاسبه می‌کند:

**1.** میزان شباهت سند به پرسمان کاربر (برای تضمین مرتبط بودن)

**2.** میزان عدم شباهت (فاصله) سند با اسنادی که در رتبه‌های بالاتر قرار گرفته‌اند (برای تضمین تازگی و تنوع)

بدین ترتیب، سیستم سندی را در رتبه بعدی قرار می‌دهد که ضمن مرتبط بودن با موضوع، کمترین هم‌پوشانی معنایی را با نتایج دیده‌شده داشته باشد. این یکی از کلیدی‌ترین مباحث در طراحی موتورهای جستجوی مدرن است.

## پاسخنامه مسئله 8.6: کیفیت سیستم در برابر مطلوبیت (System quality and user utility)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، 2 دیدگاه متفاوت برای ارزیابی موفقیت یک سیستم بازایی اطلاعات (IR) مورد بررسی قرار می‌گیرد: دیدگاه

ماشین‌محور (الگوریتمی) و دیدگاه انسان‌محور (کاربردی). هدف این است که نشان دهیم عملکرد بی‌نقص یک الگوریتم از نظر

ریاضی و آماری، لزوماً به معنای حل مشکل کاربر در دنیای واقعی نیست. درک این تفاوت برای طراحی سیستم‌هایی که علاوه بر دقت فنی، کاربردی نیز باشند، بسیار حیاتی است.

## بخش 2) مفاهیم کلیدی

**کیفیت سیستم (System Quality):** یک معیار کاملاً «عینی» (**Objective**) و قابل اندازه‌گیری است که عملکرد فنی و الگوریتمی سیستم را می‌سنجد. این کیفیت با معیارهای استاندارد ریاضی مانند دقت (**Precision**)، فراخوانی (**Recall**)، سرعت پاسخ‌گویی، و توان عملیاتی (**Throughput**) ارزیابی می‌شود.

**مطلوبیت کاربر (User Utility):** یک معیار کاملاً «ذهنی» (**Subjective**) و وابسته به شرایط است که میزان رضایت کاربر و موفقیت او در برطرف کردن نیاز اطلاعاتی‌اش را می‌سنجد. مطلوبیت به عواملی مانند سطح دانش کاربر، زمان موجود، زبان، و قالب ارائه اطلاعات بستگی دارد.

## بخش 3) حل گام‌به‌گام

### گام اول: تبیین تفاوت اساسی

تفاوت بنیادین این 2 مفهوم در «نقطه تمرکز» آن‌هاست. کیفیت سیستم بر این سوال تمرکز دارد که: «آیا سیستم کار خود را بر اساس قواعد تعریف‌شده درست انجام داده است؟» در حالی که مطلوبیت کاربر بر این سوال تمرکز دارد که: «آیا خروجی سیستم توانسته است مشکل یا وظیفه (**Task**) کاربر را حل کند؟» کیفیت سیستم در محیط ایزوله و آزمایشگاهی قابل سنجش است، اما مطلوبیت تنها در دنیای واقعی و در تعامل با انسان معنا پیدا می‌کند.

### گام دوم: طراحی یک سناریو برای مثال نقض

برای نشان دادن تقابل این 2 مفهوم، باید شرایطی را تصور کنیم که سیستم وظیفه تطبیق متن را به بی‌نقص‌ترین شکل ممکن انجام می‌دهد، اما خروجی برای کاربر بی‌فایده است.

**مثال:** فرض کنید یک دانش‌آموز دبیرستانی برای انجام تکالیف مدرسه خود، عبارت «چگونه یک مدار الکترونیکی ساده بسازیم؟» را جستجو می‌کند.

**بررسی کیفیت سیستم (بالا):** سیستم در کسری از ثانیه، مجموعه‌ای از مقالات سطح بالای دانشگاهی و ژورنال‌های تخصصی مهندسی برق (مثلاً مقالاتی در مورد طراحی مدارهای مجتمع با استفاده از ترانزیستورهای پیشرفته) را بازیابی می‌کند. از نظر الگوریتمی، این اسناد **100%** با کلمات کلیدی پرسمان مرتبط هستند؛ بنابراین سیستم امتیاز بسیار بالایی در معیارهای **Precision** و **Recall** کسب می‌کند.

**بررسی مطلوبیت کاربر (پایین):** دانش‌آموز دبیرستانی متوجه هیچ‌یک از فرمول‌ها و اصطلاحات تخصصی این مقالات نمی‌شود. با وجود اینکه اطلاعات کاملاً مرتبط و دقیق هستند، اما چون با سطح دانش و نیاز واقعی او (یک راهنمای ساده و گام‌به‌گام) هم‌خوانی ندارند، او نمی‌تواند از آن‌ها استفاده کند. در نتیجه، ارزش کاربردی و مطلوبیت این خروجی برای او در حد صفر است.

## بخش 4) پاسخ نهایی

تفاوت اساسی بین این 2 مفهوم در این است که «کیفیت سیستم» به عملکرد عینی، فنی و الگوریتمی موتور جستجو (مانند میزان دقت، فراخوانی و سرعت) اشاره دارد، در حالی که «مطلوبیت کاربر» به ارزش ذهنی، کاربردی و میزان رضایت انسان از نتایج برای حل یک مشکل واقعی در دنیای فیزیکی اشاره می‌کند.

**مثال:** کاربری که فقط به زبان فارسی مسلط است، به دنبال دستور پخت یک غذا می‌گردد. سیستم بازیابی، دقیق‌ترین و معتبرترین دستور پخت جهان را پیدا کرده و به او نشان می‌دهد (کیفیت سیستم = بسیار بالا)، اما متن این دستور پخت به زبان چینی نوشته شده است. از آنجا که کاربر قادر به خواندن و استفاده از آن نیست، نیاز اطلاعاتی او برطرف نشده و نتیجه برای او کاملاً بی‌فایده خواهد بود (مطلوبیت کاربر = بسیار پایین).

#### نکته:

در طراحی سیستم‌های بازیابی اطلاعات نوین (**Modern IR Systems**)، مهندسان برای پر کردن شکاف میان «کیفیت سیستم» و «مطلوبیت کاربر»، از سیگنال‌های رفتاری (**Implicit Feedback**) استفاده می‌کنند. موتورهای جستجویی مانند گوگل، تنها به تطبیق کلمات و محتوا اکتفا نمی‌کنند؛ بلکه با تحلیل نرخ کلیک (**CTR**)، مدت زمان ماندگاری کاربر در یک صفحه (**Dwell Time**) و نرخ پرش (**Bounce Rate**)، تلاش می‌کنند متوجه شوند آیا سندی که از نظر سیستم باکیفیت ارزیابی شده، توانسته است مطلوبیت لازم را برای کاربران به همراه داشته باشد یا خیر. در صورت پایین بودن مطلوبیت، الگوریتم‌های یادگیری ماشین رتبه آن سند را به مرور کاهش می‌دهند.

## پاسخنامه مسئله 8.7: مسائل سیستمی (System issues)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، تمرکز ارزیابی از معیارهای کیفیت محتوایی خروجی (مانند دقت و فراخوانی) به سمت ویژگی‌های مهندسی و عملکردی سیستم (**System Performance / Efficiency**) در یک محیط عملیاتی و مستقر (**Deployed**) معطوف شده است. یک موتور جستجوی موفق در دنیای واقعی، علاوه بر تولید نتایج دقیق، باید الزامات سخت‌افزاری، نرم‌افزاری و زیرساختی را نیز برآورده کند. خواسته مسئله، معرفی حداقل 2 فاکتور سیستمی کلیدی در این زمینه است.

### بخش 2) مفاهیم کلیدی

معیارهای سیستمی معمولاً معیارهای غیرعملیاتی (**Non-functional requirements**) هستند که تجربه تعاملی کاربر با سیستم و هزینه‌های نگهداری زیرساخت را تعیین می‌کنند. یک سیستم بازیابی ممکن است از نظر الگوریتمی عالی باشد، اما اگر پاسخ‌دهی آن بیش از حد طول بکشد یا هزینه ذخیره‌سازی داده‌های آن سرسام‌آور باشد، در دنیای واقعی با شکست مواجه خواهد شد. در مهندسی سیستم‌های بازیابی، کارایی با معیارهای زمانی و مکانی سنجیده می‌شود.

### بخش 3) حل گام‌به‌گام

برای ارزیابی کلی عملکرد یک موتور جستجو، فاکتورهای سیستمی متعددی سنجیده می‌شوند که در ادامه به تشریح 3 مورد از حیاتی‌ترین آن‌ها می‌پردازیم:

#### گام اول: بررسی فاکتور اول؛ زمان پاسخ‌دهی و نرخ پردازش پرسمان (Query Latency & Throughput)

این فاکتور مشخص می‌کند که سیستم با چه سرعتی به درخواست‌های کاربران پاسخ می‌دهد. این فاکتور به 2 صورت کمی ارزیابی می‌شود:

**تاخیر پرسمان (Query Latency):** مدت زمانی که طول می‌کشد تا پس از فشردن دکمه جستجو توسط کاربر، لیست

نتایج آماده و نمایش داده شود (معمولاً بر حسب میلی‌ثانیه سنجیده می‌شود). برای کاربران مدرن، تاخیر بیش از

چندصد میلی‌ثانیه غیرقابل قبول است.

**نرخ پردازش پرسمان (Throughput):** تعداد پرسمانی که سیستم می‌تواند در یک واحد زمانی (مثلاً یک ثانیه) تحت بار ترافیکی شدید بدون افت کیفیت پردازش کند که اصطلاحاً به آن **QPS (Queries Per Second)** می‌گویند.

**گام دوم: بررسی فاکتور دوم؛ اندازه نمایه و بار حافظه (Index Size & Storage Space)**  
موتورهای جستجو برای سرعت بخشیدن به فرآیند جستجو، از ساختار داده‌ای به نام «نمایه معکوس» (**Inverted Index**) استفاده می‌کنند.

اندازه این نمایه بر روی دیسک و به‌ویژه میزان فضایی که از حافظه موقت (**RAM**) اشغال می‌کند، یک فاکتور سیستمی بسیار حیاتی است.

اگر اندازه نمایه بیش از حد بزرگ باشد، بخش عمده‌ای از آن باید روی دیسک ذخیره شود که این امر به دلیل سرعت پایین خواندن از دیسک (**Disk I/O**)، سرعت جستجو را به شدت کاهش می‌دهد. بنابراین، ارزیابی میزان بهینه بودن و فشرده‌سازی نمایه یک فاکتور اساسی است.

**گام سوم: بررسی فاکتور سوم؛ سرعت ساخت و به‌روزرسانی نمایه (Indexing Speed)**  
پایگاه داده اسناد در دنیای واقعی ثابت نیست؛ اسناد جدید به طور مداوم اضافه، حذف یا ویرایش می‌شوند (مانند صفحات وب یا خبرگزاری‌ها).

این فاکتور می‌سنجد که سیستم با چه سرعتی می‌تواند اسناد جدید را پردازش کرده، توکن‌بندی کند و به نمایه معکوس موجود اضافه نماید تا در نتایج جستجو قابل مشاهده باشند. سرعت نمایه سازی بالا، نشان‌دهنده پویایی و به‌روز بودن موتور جستجو است.

## بخش 4) پاسخ نهایی

علاوه بر دقت نتایج، فاکتورهای سیستمی زیر برای ارزیابی عملکرد کلی یک موتور جستجو به کار می‌روند:

**1. سرعت و نرخ پردازش پرسمان (Query Efficiency):** شامل «تاخیر پاسخ‌دهی» (مدت زمان تولید نتایج بر حسب میلی‌ثانیه) و «نرخ پردازش» (تعداد پرسمان‌های پشتیبانی‌شده در ثانیه یا **QPS**).

**2. اندازه نمایه و بار حافظه (Storage & Memory Efficiency):** میزان فضای دیسک و حافظه **RAM** اشغال‌شده توسط نمایه معکوس که مستقیماً بر هزینه‌های زیرساخت و سرعت دسترسی به داده‌ها تأثیرگذار است.

**3. سرعت نمایه سازی (Indexing Speed):** توانایی و سرعت سیستم در پردازش اسناد جدید و به‌روزرسانی ساختار نمایه برای همگام‌شدن با تغییرات داده‌ها.

### نکته:

توجه داشته باشید که میان معیارهای سیستمی و معیارهای کیفیت بازیابی، یک رابطه الکلنگی یا تقابل (**Trade-off**) برقرار است. به عنوان مثال، برای افزایش سرعت پاسخ‌دهی سیستم (فاکتور سیستمی اول)، مهندسان ناچارند نمایه معکوس را با استفاده از روش‌هایی مانند کدگذاری بایت متغیر (**Variable Byte Encoding**) یا کدگذاری گاما (**γ-encoding**) فشرده کنند. فرآیند فشرده‌سازی، اندازه نمایه را کاهش می‌دهد (فاکتور سیستمی دوم) تا در حافظه **RAM** جا شود، اما از طرف دیگر، باز کردن این فشرده‌سازی (**Decompression**) در زمان بازخوانی پرسمان، به توان پردازشی **CPU** نیاز دارد. هنر یک مهندس بازیابی اطلاعات، ایجاد تعادل بهینه میان این فاکتورهای متضاد بر اساس محدودیت‌های پروژه است.

## پاسخنامه مسئله 8.8: خلاصه نتایج جستجو (Results snippets)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، تمرکز بر روی نحوه نمایش نتایج در صفحه نتایج موتور جستجو (SERP) است. پس از آنکه سیستم بازیابی اسناد را رتبه‌بندی کرد، باید پیش‌نمایش یا خلاصه‌ای کوتاه از هر سند را به کاربر ارائه دهد تا او را در انتخاب سند مناسب یاری کند. این قطعات متنی کوتاه «اسنیپت» (Snippet) نامیده می‌شوند. خواسته مسئله، مقایسه 2 روش اصلی تولید این خلاصه‌ها (رویکرد ایستا در برابر پویا) و تحلیل منطق موتورهای جستجوی مدرن در انتخاب میان این 2 است.

### بخش 2) مفاهیم کلیدی

**اسنیپت (Snippet):** قطعه متن کوتاهی که در صفحه نتایج جستجو، زیر عنوان و پیوند (لینک) هر سند نمایش داده می‌شود. هدف آن ارائه تصویری اجمالی از محتوای سند است.

**اسنیپت ایستا (Static Snippet):** خلاصه‌ای از پیش تعیین‌شده و ثابت که مستقل از پرسمان کاربر است.

**اسنیپت پویا یا وابسته به پرسمان (Dynamic / Query-biased Snippet):** خلاصه‌ای که در لحظه جستجو و منحصرأ بر اساس کلمات کلیدی پرسمان کاربر ساخته می‌شود.

### بخش 3) حل گام‌به‌گام

#### گام اول: تحلیل مکانیزم اسنیپت‌های ایستا

اسنیپت‌های ایستا معمولاً در مرحله نمایه‌سازی (Indexing) تولید و در پایگاه داده ذخیره می‌شوند. سیستم برای ساخت آن‌ها عموماً به سراغ متادیتاهای سند (مانند تگ `<meta name="description">` در HTML) یا چند خط اول متن سند می‌رود. ویژگی بارز آن‌ها «ثبات» است؛ یعنی اگر 2 کاربر مختلف با 2 پرسمان کاملاً متفاوت به 1 سند واحد در نتایج جستجو برسند، هر 2 دقیقاً 1 متن خلاصه یکسان را زیر آن لینک مشاهده خواهند کرد.

#### گام دوم: تحلیل مکانیزم اسنیپت‌های پویا

اسنیپت‌های پویا در زمان پردازش پرسمان (Query Time) تولید می‌شوند. در این روش، سیستم متن سند را اسکن می‌کند تا پنجره‌ای از کلمات (معمولاً 1 یا 2 جمله) را بیابد که حاوی بیشترین کلمات کلیدی پرسمان کاربر باشد. سپس کلمات جستجو شده را در متن خلاصه پررنگ (Bold) می‌کند. در این حالت، 1 سند واحد می‌تواند برای پرسمان‌های مختلف، خلاصه‌های کاملاً متفاوتی را به نمایش بگذارد تا دقیقاً نشان دهد چرا این سند برای آن پرسمان خاص، بازیابی شده است.

#### گام سوم: بررسی رویکرد موتورهای جستجوی امروزی

موتورهای جستجوی مدرن، تمایل قاطعی به استفاده از اسنیپت‌های پویا دارند (هرچند گاهی در صورت نبود متن مناسب، به اسنیپت‌های ایستا به عنوان راهکار پشتیبان یا Fallback تکیه می‌کنند).

دلیل این انتخاب، ارتقای شدید «مطلوبیت کاربر» است. اسنیپت پویا بستر معنایی (Context) کلمات جستجو شده را به کاربر نشان می‌دهد. کاربر بدون نیاز به باز کردن لینک و خواندن کل متن طولانی سند، می‌تواند قضاوت کند که آیا نویسنده، کلمات کلیدی را در همان مفهومی که مد نظر او بوده به کار برده است یا خیر. این امر باعث می‌شود ارزیابی ارتباط توسط کاربر سریع‌تر و دقیق‌تر انجام شود.

### بخش 4) پاسخ نهایی

**تفاوت اساسی:** اسنیپت ایستا همواره ثابت است و معمولاً از توضیحات متادیتای سند استخراج می‌شود و هیچ وابستگی به کلماتی که کاربر جستجو کرده است ندارد. در مقابل، اسنیپت پویا در لحظه و بر اساس پرسمان ساخته می‌شود و بخش‌هایی از متن سند را که حاوی کلمات کلیدی جستجو شده هستند، استخراج کرده و به عنوان خلاصه نمایش می‌دهد.

**گرایش سیستم‌های امروزی:** موتورهای جستجوی مدرن عمدتاً از اسنیپت‌های پویا (**Query-biased**) استفاده می‌کنند.

**علت ترجیح:** اسنیپت پویا به کاربر اجازه می‌دهد تا بستر استفاده از کلمات جستجو شده در داخل متن را ببیند. این امر به کاربر کمک می‌کند تا پیش از کلیک روی لینک، ارزیابی بسیار دقیق‌تری از میزان ارتباط سند با نیاز اطلاعاتی خود داشته باشد؛ در نتیجه تجربه جستجوی کارآمدتری رقم خورده و از کلیک‌های بی‌فایده جلوگیری می‌شود.

#### نکته:

از منظر معماری سیستم (**System Architecture**)، تولید اسنیپت‌های پویا 1 چالش مهم مهندسی ایجاد می‌کند. برای اسنیپت ایستا، سیستم خلاصه را 1 بار تولید و ذخیره می‌کند که هزینه پردازشی بسیار کمی در زمان جستجو دارد. اما تولید اسنیپت پویا بار پردازشی سنگینی به سیستم تحمیل می‌کند؛ زیرا سیستم باید در کسری از ثانیه متن اصلی اسناد برترِ بازیابی‌شده را از دیسک یا حافظه پنهان فراخوانی کرده، جملات را تحلیل کند و خلاصه را بسازد. به همین دلیل، در طراحی موتورهای جستجوی مقیاس‌پذیر، تکنیک‌های پیشرفته‌ای برای ذخیره‌سازی موقت (**Caching**) متن اسناد طراحی می‌شود تا تولید اسنیپت پویا به فاکتورهای سیستمی (مانند تاخیر پرسمان) آسیب نرساند.

## پاسخنامه مسئله 8.9: محاسبه معیار F1 (F1 Measure Calculation)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، داده‌های خام حاصل از جستجوی یک پرسمان در اختیار ما قرار گرفته و هدف، محاسبه کمی 3 معیار بنیادین ارزیابی سیستم‌های بازیابی اطلاعات (**Precision**، **Recall** و **F-Measure**) است. داده‌های مسئله شامل تعداد کل اسناد بازیابی شده، تعداد اسناد مرتبط موجود در خروجی و تعداد کل اسناد مرتبط در پایگاه داده است. این مسئله به دانشجویان کمک می‌کند تا نحوه پیوند خوردن مقادیر عددی با فرمول‌های تئوری را به صورت عملی درک کنند.

### بخش 2) مفاهیم کلیدی

**دقت (Precision):** کسر اسناد مرتبط در میان کل اسناد بازیابی شده.

**فراخوانی (Recall):** کسر اسناد مرتبط بازیابی شده نسبت به کل اسناد مرتبط موجود در مجموعه.

**معیار اف (F<sub>1</sub> Measure):** میانگین همساز (**Harmonic Mean**) بین دقت و فراخوانی. از آنجا که سیستم‌ها معمولاً با

تقابل (**Trade-off**) بین دقت و فراخوانی مواجه هستند، معیار **F<sub>1</sub>** یک عدد واحد ارائه می‌دهد که نشان‌دهنده تعادل عملکرد سیستم است.

### بخش 3) حل گام‌به‌گام

**گام اول: استخراج و نمادگذاری داده‌های مسئله**

ابتدا داده‌های صورت سؤال را به زبان متغیرهای استاندارد بازیابی اطلاعات ترجمه می‌کنیم:

**تعداد کل اسناد بازیابی شده توسط موتور جستجو:  $|\text{Retrieved}| = 100$**

تعداد اسناد مرتبط در میان اسناد بازیابی شده (اشتراک اسناد مرتبط و بازیابی شده):

$$|\text{Retrieved} \cap \text{Relevant}| = 40$$

تعداد کل اسناد مرتبط موجود در مجموعه:  $|\text{Relevant}| = 60$

گام دوم: محاسبه معیار دقت (Precision)

بر اساس فرمول، دقت برابر است با نسبت اسناد مرتبط بازیابی شده به کل اسناد بازیابی شده:

$$\text{Precision} = \frac{|\text{Retrieved} \cap \text{Relevant}|}{|\text{Retrieved}|} = \frac{40}{100} = 0.4$$

گام سوم: محاسبه معیار فراخوانی (Recall)

فراخوانی برابر است با نسبت اسناد مرتبط بازیابی شده به کل اسناد مرتبط موجود در پایگاه داده:

$$\text{Recall} = \frac{|\text{Retrieved} \cap \text{Relevant}|}{|\text{Relevant}|} = \frac{40}{60} = \frac{2}{3} \approx 0.667$$

(تفسیر: سیستم موفق شده است حدود **66.7** درصد از کل اسناد مرتبط موجود را پیدا کند.)

گام چهارم: محاسبه معیار  $F_1$

برای محاسبه  $F_1$ ، از فرمول میانگین همساز استفاده می‌کنیم. برای جلوگیری از خطای گرد کردن، بهتر است از مقادیر کسری در

محاسبات استفاده کنیم ( $P = 4/10$  و  $R = 2/3$ ):

$$F_1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times \frac{4}{10} \times \frac{2}{3}}{\frac{4}{10} + \frac{2}{3}} = \frac{\frac{16}{30}}{\frac{32}{30}} = 0.5$$

بخش 4) پاسخ نهایی

بر اساس داده‌های مسئله، مقادیر معیارهای ارزیابی به شرح زیر است:

دقت (Precision): 0.4

فراخوانی (Recall): 0.667 (یا کسر 2/3)

معیار اف ( $F_1$ ): 0.5

نکته مهم:

شاید برای شما سؤال پیش بیاید که چرا برای ترکیب دقت و فراخوانی از میانگین حسابی (Arithmetic Mean) ساده استفاده نمی‌کنیم و به سراغ میانگین همساز (Harmonic Mean) می‌رویم. دلیل این امر در رفتار ریاضی میانگین همساز نهفته است. اگر از میانگین حسابی استفاده کنیم، سیستمی که 100% اسناد را برمی‌گرداند (در نتیجه  $\text{Recall}=1$  و  $\text{Precision}$  بسیار پایین، مثلاً 0.001 است) نمره قبولی بسیار بالایی (حدود 0.5) می‌گیرد که گمراه‌کننده است. اما میانگین همساز به مقادیر کوچک‌تر به شدت حساس است و آن‌ها را جریمه می‌کند. معیار  $F_1$  تنها زمانی مقدار بالایی به خود می‌گیرد که هر 2 معیار دقت و فراخوانی مقادیر

متناسب و بالایی داشته باشند. در این مسئله، مشاهده می‌کنید که عدد نهایی  $F_1$  به مقدار کمتر ( $P=0.4$ ) متمایل‌تر است تا مقدار بیشتر ( $R=0.667$ ).

## پاسخنامه مسئله 8.10: ارزیابی در نقاط مشخص (Precision at k)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، به یکی از پرکاربردترین معیارهای ارزیابی در سیستم‌های بازیابی رتبه‌بندی شده (**Ranked Retrieval**)، به‌ویژه موتورهای جستجوی وب، می‌پردازیم. هدف مسئله، شناخت معیار **Precision@K** و تحلیل تفاوت رفتار کاربران در محیط وب نسبت به سیستم‌های بازیابی سنتی است که باعث می‌شود محاسبه فراخوانی (**Recall**) در این محیط‌ها اهمیت خود را از دست بدهد.

### بخش 2) مفاهیم کلیدی

**نقطه برش (Cut-off Point):** محدود کردن ارزیابی به تعداد مشخصی از نتایج برتر (مثلاً **K** سند اول) به جای بررسی کل لیست نتایج.

**رفتار کاربر وب:** تمایل شدید کاربران در اینترنت به یافتن پاسخ در صفحه اول نتایج و نادیده گرفتن صفحات بعدی.

**حجم داده در وب:** گستردگی بی‌نهایت صفحات وب که محاسبه دقیق مجموع کل اسناد مرتبط در جهان را ناممکن می‌سازد.

### بخش 3) حل گام‌به‌گام

#### گام اول: تعریف معیار Precision@K

در سیستم‌های رتبه‌بندی شده، کاربر کل خروجی را بررسی نمی‌کند. معیار «دقت در نقطه **K**» مشخص می‌کند که از بین **K** سند اولی که سیستم در بالاترین رتبه‌ها به کاربر نشان داده است، چه نسبتی از آن‌ها واقعاً با پرسمان مرتبط هستند.

$$Precision@K = \frac{|Relevant Documents in Top K|}{K}$$

#### گام دوم: بررسی جایگاه $P@10$ در موتورهای جستجوی وب

در موتورهای جستجوی وب، به طور سنتی در هر صفحه از نتایج (**SERP**)، **10** پیوند نمایش داده می‌شود. بنابراین، بررسی معیار **P@10** در واقع به معنای محاسبه «دقت خروجی در صفحه اول نتایج» است.

#### گام سوم: تحلیل دلایل برتری $P@10$ بر Recall در محیط وب

در سیستم‌های سنتی پایگاه داده، یافتن تمامی اسناد مرتبط (**Recall** بالا) اهمیت دارد، اما در وب این‌گونه نیست. این تفاوت ناشی از **2** دلیل عمده است:

**1. محدودیت توجه و رضایت کاربر (User Attention):** مطالعات رفتار کاربر نشان می‌دهد که اکثریت قریب به اتفاق

کاربران وب، هرگز به صفحه دوم نتایج مراجعه نمی‌کنند. اگر سیستمی تمام اسناد مرتبط جهان را پیدا کند اما آن‌ها را

در رتبه‌های 50 به بعد نمایش دهد (فراخوانی 100%)، برای کاربر وب کاملاً بی‌فایده است. کاربر وب عموماً به دنبال یافتن همه پاسخ‌های موجود نیست، بلکه فقط به چند پاسخ دقیق و سریع در همان صفحه اول نیاز دارد.

**2. ناممکن بودن محاسبه فراخوانی در مقیاس وب:** برای محاسبه فرمول فراخوانی (**Recall**)، ما باید تعداد کل اسناد مرتبط موجود در کل پایگاه داده (مخرج کسر) را بدانیم. در محیطی مانند وب با میلیاردها صفحه، هیچ‌کس نمی‌تواند به طور قطع بگوید دقیقاً چند صفحه مرتبط با یک پرسمان خاص وجود دارد؛ در نتیجه، محاسبه ریاضی و دقیق **Recall** در وب عملاً غیرممکن است.

## بخش 4) پاسخ نهایی

**تعریف  $P@K$ :** این معیار، میزان «دقت» سیستم جستجو را تنها در **K** نتیجه برتر اندازه‌گیری می‌کند. به عبارت دیگر، این معیار نشان می‌دهد چه کسری از **K** سند اول بازایی شده، با نیاز کاربر مرتبط هستند.

**دلیل اهمیت  $P@10$  نسبت به Recall در وب:** معیار  **$P@10$**  عملکرد موتور جستجو را دقیقاً در «صفحه اول نتایج» می‌سنجد که کاملاً منطبق بر رفتار و حوصله کاربر وب است. در مقابل، معیار **Recall** در جستجوی وب کارایی ندارد؛ زیرا اولاً محاسبه آن به دلیل نامشخص بودن تعداد کل اسناد مرتبط در شبکه اینترنت غیرممکن است و ثانیاً کاربران وب نیازی به دیدن «تمام» اسناد مرتبط ندارند، بلکه یافتن چند نتیجه کاملاً دقیق در همان نتایج اولیه برای آن‌ها کفایت می‌کند.

### نکته:

اگرچه معیار  **$P@K$**  بسیار ملموس و محاسبه آن ساده است، اما **1** نقطه‌ضعف محاسباتی دارد: این معیار به جایگاه دقیق اسناد مرتبط در داخل همان **K** سند اهمیتی نمی‌دهد. به عنوان مثال در محاسبه  **$P@10$** ، اگر تنها سند مرتبط در رتبه اول ظاهر شود یا در رتبه دهم، در هر **2** حالت نمره ارزیابی برابر با **0.1** خواهد بود. برای رفع این مشکل و حساس کردن ارزیابی به رتبه دقیق هر سند در مقیاس‌های وب، در مهندسی بازایی اطلاعات مدرن از معیارهای پیشرفته‌تری استفاده می‌شود. برای پرسمان‌هایی که فقط **1** پاسخ دقیق دارند از میانگین متقابل رتبه (**MRR**) و برای پرسمان‌های عمومی‌تر از معیار **NDCG** استفاده می‌گردد تا اسنادی که در رتبه‌های بالاتر (مانند رتبه **1** و **2**) ظاهر می‌شوند، امتیاز بیشتری برای سیستم به همراه داشته باشند.

## پاسخنامه مسئله 8.11: نمودار دقت-بازایی (Precision-Recall curve)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، به بررسی یکی از مهم‌ترین ابزارهای بصری در علم بازایی اطلاعات، یعنی نمودار دقت-فراخوانی (**Precision-Recall Curve**) می‌پردازیم. هدف این است که رفتار گرافیکی این **2** معیار ارزیابی را در کنار یکدیگر تحلیل کنیم و منطق نهفته در پس رابطه معکوس آن‌ها را در سیستم‌های مبتنی بر رتبه‌بندی (**Ranked Retrieval**) تشریح نماییم.

### بخش 2) مفاهیم کلیدی

**نمودار P-R:** یک فضای مختصات دوبعدی است که در آن محور افقی (**X**) نشان‌دهنده مقدار فراخوانی (**Recall**) و محور عمودی (**Y**) نشان‌دهنده مقدار دقت (**Precision**) است. هر نقطه روی این نمودار، عملکرد سیستم را در یک نقطه برش (**Cut-off**) خاص از لیست نتایج نشان می‌دهد.

**تقابل (Trade-off):** یک اصل در طراحی الگوریتم‌های بازیابی است که نشان می‌دهد به حداکثر رساندن همزمان **2**

معیار دقت و فراخوانی عملاً غیرممکن است و بهبود یکی، به بهای تضعیف دیگری تمام می‌شود.

### بخش 3) حل گام به گام

#### گام اول: تحلیل روند ظاهری نمودار

روند تغییرات در یک نمودار استاندارد دقت-فراخوانی، از نظر کلی نزولی (**Downward Trend**) است. نمودار معمولاً از سمت چپ بالا (جایی که سیستم دقت بسیار بالا اما فراخوانی پایینی دارد) آغاز شده و به سمت راست پایین (فراخوانی بالا اما دقت پایین) حرکت می‌کند. به دلیل ماهیت گسسته بودن جایگاه اسناد در لیست رتبه‌بندی، شکل خام این نمودار به صورت یک خط صاف نیست، بلکه دارای نوسان و حالتی دندانه‌دار (**Zig-zag**) است.

#### گام دوم: تحلیل فرآیند حرکت در طول لیست نتایج

برای درک علت این رفتار، باید نحوه کارکرد موتورهای جستجو را بررسی کنیم. الگوریتم‌های رتبه‌بندی، اسنادی را که بیشترین شباهت را به پرسمان دارند (و بیشترین اطمینان به مرتبط بودن آن‌ها وجود دارد) در صدر لیست قرار می‌دهند.

هنگامی که کاربر تنها چند سند اول را بررسی می‌کند، به احتمال بسیار زیاد اسناد کاملاً مرتبطی را می‌بیند (دقت نزدیک به **100%** است). اما چون کاربر هنوز اسناد مرتبط موجود در ادامه‌ی لیست را ندیده است، میزان فراخوانی در این نقطه بسیار پایین است.

#### گام سوم: تحلیل ریاضی و منطقی افت دقت

برای اینکه معیار فراخوانی (**Recall**) افزایش یابد، سیستم باید اسناد بیشتری را از رتبه‌های پایین‌تر لیست واکنشی کند تا هیچ سند مرتبطی از قلم نیفتد. با پایین رفتن در لیست نتایج:

**1.** سیستم از منطقه «اطمینان بالا» خارج شده و شروع به حدس و گمان می‌کند.

**2.** برای پیدا کردن تعداد کمی سند مرتبط باقی‌مانده، سیستم ناچار است حجم عظیمی از اسناد مبهم و نامرتب (نویز) را نیز وارد خروجی کند.

**3.** از نظر ریاضی، در فرمول دقت:

$$Precision = \frac{|Retrieved \cap Relevant|}{|Retrieved|}$$

مخرج کسر (تعداد کل اسناد واکنشی‌شده) با سرعت بسیار بیشتری نسبت به صورت کسر (تعداد اسناد مرتبط جدید) رشد می‌کند. این امر لاجرم باعث کاهش سریع مقدار کسر و افت شدید دقت می‌شود.

### بخش 4) پاسخ نهایی

**روند تغییرات نمودار:** یک روند کلی نزولی و دندانه‌دار است که از مختصات (دقت بالا، فراخوانی پایین) در چپ شروع شده و به مختصات (دقت پایین، فراخوانی بالا) در راست ختم می‌گردد.

**علت افت Precision با افزایش Recall:** با حرکت به سمت رتبه‌های پایین‌تر لیست برای استخراج تمام اسناد مرتبط (افزایش **Recall**)، سیستم مجبور است اسناد نامرتب بسیار بیشتری را بازیابی کند. این ورود پر حجم اسناد نامرتب به لیست خروجی، خلوص نتایج را از بین برده و منجر به افت ناگزیر معیار **Precision** می‌شود.

از آنجا که نمودار خام **P-R** به دلیل وجود دندان‌ها (نوسانات محلی) خوانایی مناسبی برای مقایسه الگوریتم‌ها ندارد، در ارزیابی‌های رسمی و استاندارد دانشگاهی از تکنیکی به نام «دقت درونیابی‌شده» (**Interpolated Precision**) استفاده می‌شود. در این روش، به جای نمایش نوسانات، در هر سطح از فراخوانی، بیشترین مقدار دقتی که در آن نقطه یا نقاط پس از آن به دست آمده است جایگزین می‌شود. این کار باعث می‌شود دندان‌های نمودار هموار شده و یک منحنی نزولی یکدست (**Monotonically Decreasing**) به دست آید که مقایسه بصری بین عملکرد **2** سیستم جستجوی متفاوت را بسیار شفاف و علمی می‌سازد. رایج‌ترین استاندارد در این زمینه، محاسبه دقت در **11** نقطه فراخوانی (از **0.0** تا **1.0** با گام‌های **0.1**) است که به آن **11-point interpolated average precision** می‌گویند.

## پاسخنامه مسئله 8.12: میانگین دقت متوسط (Mean Average Precision)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما با یک لیست رتبه‌بندی شده از خروجی‌های موتور جستجو برای یک پرسمان خاص مواجه هستیم و هدف، محاسبه معیار «متوسط دقت» یا **AP (Average Precision)** است. این مسئله به منظور درک نحوه ارزش‌گذاری جایگاه (رتبه) اسناد مرتبط در لیست نتایج طراحی شده است و به وضوح نشان می‌دهد که چگونه سیستم برای نیافتن تمام اسناد مرتبط جریمه می‌شود (زیرا تنها **4** سند از **5** سند مرتبط بازیابی شده‌اند).

### بخش 2) مفاهیم کلیدی

**متوسط دقت (Average Precision):** یک معیار ارزیابی تک‌مقداری برای یک پرسمان است که هم دقت (**Precision**)

و هم فراخوانی (**Recall**) را همزمان در نظر می‌گیرد و به شدت به رتبه اسناد مرتبط حساس است.

**منطق محاسبه:** در این روش، ما فقط در رتبه‌هایی که یک سند مرتبط (**R**) یافت می‌شود، مقدار «دقت در آن نقطه»

**(Precision@K)** را محاسبه می‌کنیم. در نهایت، مجموع این دقت‌ها را بر کل اسناد مرتبط موجود در پایگاه داده (نه

فقط اسناد بازیابی شده) تقسیم می‌کنیم.

### بخش 3) حل گام‌به‌گام

#### گام اول: فرمول‌بندی ریاضی

فرمول استاندارد برای محاسبه **AP** به صورت زیر است:

$$AP = \frac{1}{|\text{Relevant}|} \sum_{k=1}^n \left( P@k \times rel(k) \right)$$

که در آن:

**|Relevant|**: تعداد کل اسناد مرتبط موجود برای آن پرسمان (طبق صورت سؤال برابر با **5** است).

**n**: تعداد کل اسناد بازیابی شده در لیست.

**P@k**: میزان دقت تا رتبه **k**.

**rel(k)**: یک تابع نشانگر است؛ اگر سند در رتبه **k** مرتبط باشد مقدار آن **1** و اگر نامرتبط باشد مقدار آن **0** است. (این یعنی اسناد نامرتبط در مجموع نهایی محاسبه نمی‌شوند).

### گام دوم: بررسی رتبه به رتبه لیست بازیابی شده

لیست داده شده به این صورت است: **R, N, R, R, N, N, R**.

ما رتبه‌ها (**k**) را از **1** تا **7** شماره‌گذاری کرده و **P@k** را فقط برای رتبه‌های حاوی اسناد مرتبط (**R**) محاسبه می‌کنیم:

**رتبه 1 (R)**: اولین سند مرتبط یافت شد.

$$P@1 = \frac{1}{1} = 1.0$$

**رتبه 2 (N)**: نامرتبط است (**rel(2) = 0**). محاسبه نمی‌شود.

**رتبه 3 (R)**: دومین سند مرتبط یافت شد.

$$P@3 = \frac{2}{3} \approx 0.67$$

**رتبه 4 (R)**: سومین سند مرتبط یافت شد.

$$P@4 = \frac{3}{4} = 0.75$$

**رتبه 5 (N)**: نامرتبط است پس محاسبه نمی‌شود.

**رتبه 6 (N)**: نامرتبط است پس محاسبه نمی‌شود.

**رتبه 7 (R)**: چهارمین سند مرتبط یافت شد.

$$P@7 = \frac{4}{7} \approx 0.57$$

### گام سوم: محاسبه نهایی AP

اکنون مقادیر دقت به دست آمده را با هم جمع کرده و بر تعداد کل اسناد مرتبط (یعنی **5**) تقسیم می‌کنیم. سند مرتبط پنجم که توسط سیستم پیدا نشده است، به صورت یک رتبه بی‌نهایت با دقت صفر فرض می‌شود.

$$AP = \frac{1}{5} (1.0 + 0.67 + 0.75 + 0.57)$$

برای دقت بیشتر در محاسبات ریاضی:

$$\begin{aligned}AP &= \frac{1}{5} \left( 1 + \frac{2}{3} + \frac{3}{4} + \frac{4}{7} \right) \\&= \frac{1}{5} \left( \frac{84 + 56 + 63 + 48}{84} \right) \\&= \frac{1}{5} \left( \frac{251}{84} \right) \approx 0.597\end{aligned}$$

#### بخش 4) پاسخ نهایی

مقدار **Average Precision (AP)** برای این لیست رتبه‌بندی شده و بر اساس 5 سند مرتبط موجود، به شرح زیر محاسبه می‌گردد:

$$AP \approx 0.597$$

#### نکته:

یک اشتباه بسیار رایج در میان دانشجویان هنگام محاسبه **AP** این است که در مرحله آخر، مجموع دقت‌ها را بر تعداد اسناد مرتبط «بازیابی شده» (در این مثال 4) تقسیم می‌کنند. دقت کنید که مخرج کسر در فرمول **AP**، همیشه تعداد کل اسناد مرتبط برای آن پرسمان در کل مجموعه (**Relevant**) است (در اینجا 5). این کار باعث می‌شود سیستمی که نتوانسته تمامی اسناد مرتبط را بازیابی کند، با کاهش امتیاز مواجه شده و اصطلاحاً به دلیل «فراخوانی ناقص» (**Low Recall**) جریمه شود. همچنین به خاطر داشته باشید که اگر میانگین مقادیر **AP** را برای مجموعه‌ای از پرسمان‌های مختلف (مثلاً 50 پرسمان یک مجموعه آزمون) محاسبه کنیم، به معیار جامع‌تر و مشهورتری به نام **MAP (Mean Average Precision)** دست می‌یابیم که استاندارد طلایی ارزیابی در سیستم‌های بازیابی است.

### پاسخنامه مسئله 8.13: مطلوبیت کاربر در عمل (User utility practical assessment)

#### بخش 1) تحلیل اولیه مسئله

در این مسئله، رویکرد ما از ارزیابی‌های آزمایشگاهی (مانند مجموعه‌های آزمون و قضاوت‌های انسانی که بسیار پرهزینه و زمان‌بر هستند) به سمت ارزیابی در محیط عملیاتی و واقعی تغییر می‌کند. چالش اصلی این است که در دنیای واقعی، کاربران عموماً تمایلی ندارند به صورت مستقیم و صریح (**Explicit**) به ما بگویند که آیا نتایج برایشان مفید بوده است یا خیر (مثلاً با پر کردن فرم نظرسنجی یا امتیازدهی ستاره‌دار). بنابراین، سیستم باید بتواند از روی «رفتار طبیعی» کاربر، میزان مطلوبیت و موفقیت خود را استنتاج کند. خواسته مسئله، معرفی 3 نمونه از این سیگنال‌های رفتاری است.

#### بخش 2) مفاهیم کلیدی

**بازخورد صریح (Explicit Feedback):** زمانی که کاربر به صورت آگاهانه و مستقیم میزان رضایت خود را اعلام می‌کند.

**بازخورد ضمنی (Implicit Feedback):** استخراج اطلاعات ارزیابی از لاگ‌ها (**Logs**) و ردپای تعاملات کاربر با سیستم.

این روش مقیاس‌پذیر و ارزان است، اما از آنجا که رفتار انسان می‌تواند دلایل متعددی داشته باشد، داده‌های آن دارای «نویز» (**Noise**) هستند و نیازمند تحلیل آماری دقیق می‌باشند.

## بخش 3) حل گام به گام

برای سنجش مطلوبیت بدون پرسش مستقیم از کاربر، موتورهای جستجو رفتار او را در صفحه نتایج (SERP) رصد می‌کنند. 3 سیگنال بسیار قدرتمند در این زمینه عبارتند از:

### گام اول: تحلیل رفتار کلیک (Click-Through Data)

ساده‌ترین و در دسترس‌ترین سیگنال، بررسی لینک‌هایی است که کاربر روی آن‌ها کلیک می‌کند.

اگر سیستمی سندی را در رتبه اول نمایش دهد، اما درصد زیادی از کاربران روی سند رتبه سوم کلیک کنند، این یک سیگنال ضمنی قوی است که نشان می‌دهد مطلوبیت سند سوم برای این پرسمان خاص بالاتر است. نرخ کلیک (CTR) نشان‌دهنده جذابیت اسنیپت و عنوان سند برای کاربر است.

### گام دوم: تحلیل زمان ماندگاری (Dwell Time) و پدیده پرش (Pogo-sticking)

صرفِ کلیک کردن روی یک لینک، مطلوبیت را تضمین نمی‌کند (ممکن است کاربر فریب یک عنوان جذاب را خورده باشد). برای تایید مطلوبیت، سیستم مدت زمانی که کاربر در صفحه مقصد می‌ماند تا زمانی که دوباره به موتور جستجو برمی‌گردد را اندازه‌گیری می‌کند.

**ماندگاری طولانی (Long Dwell Time):** اگر کاربر روی سندی کلیک کند و دقایق زیادی را در آن سایت سپری کند یا اصلاً به صفحه جستجو برنگردد، نشان‌دهنده مطلوبیت بالای آن سند است.

**پرش (Pogo-sticking):** اگر کاربر روی لینک کلیک کند و بلافاصله (مثلاً زیر 5 ثانیه) دکمه **Back** را بزند و به صفحه نتایج برگردد، نشان می‌دهد که محتوای سند با نیاز او هم‌خوانی نداشته و مطلوبیت صفر بوده است.

### گام سوم: تحلیل توالی جستجو و بازنویسی پرسمان (Query Reformulation)

سیستم‌ها رفتار کاربر را در قالب «نشست‌های جستجو» (Search Sessions) تحلیل می‌کنند.

اگر کاربر پرسمانی را جستجو کند، نتایج را ببیند، روی هیچ لینکی کلیک نکند و بلافاصله کلمات جستجوی خود را تغییر دهد (مثلاً واژگان مترادف را به کار ببرد یا پرسمان را طولانی‌تر کند)، این یک سیگنال قطعی از شکست سیستم در ارائه نتایج مطلوب در تلاش اول است. بازنویسی مکرر پرسمان نشان‌دهنده سردرگمی کاربر و پایین بودن کیفیت خروجی سیستم می‌باشد.

## بخش 4) پاسخ نهایی

برای سنجش مطلوبیت سیستم از دیدگاه کاربر بدون نیاز به قضاوت‌های صریح انسانی، از سیگنال‌های ضمنی زیر استفاده می‌شود:

1. **داده‌های کلیک (Click Behavior & CTR):** رصد اینکه کاربر کدام اسناد را برای مشاهده انتخاب می‌کند.

2. **زمان ماندگاری و پرش (Dwell Time & Pogo-sticking):** اندازه‌گیری مدت زمانی که کاربر پس از کلیک، در یک سند سپری می‌کند پیش از آنکه برای یافتن نتیجه بهتر به صفحه جستجو بازگردد.

3. **بازنویسی پرسمان (Query Reformulation):** بررسی اینکه آیا کاربر پس از مشاهده نتایج اولیه، مجبور به تغییر کلمات کلیدی و جستجوی مجدد شده است یا خیر.

### نکته:

در شرکت‌های بزرگ فناوری مانند گوگل یا مایکروسافت (بینگ)، برای اینکه از این سیگنال‌های ضمنی جهت ارزیابی نهایی و مقایسه

**2** الگوریتم جستجوی مختلف استفاده کنند، از چارچوب‌هایی نظیر آزمون **A/B (A/B Testing)** و درهم‌تنیدگی (**Interleaving**) بهره می‌برند.

در روش **Interleaving**، سیستم در لحظه جستجو، نتایج الگوریتم فعلی (**A**) و الگوریتم جدید (**B**) را به صورت یکی‌درمیان (مانند بر زدن کارت‌ها) در یک لیست واحد به کاربر واقعی نشان می‌دهد. سپس با استفاده از همین سیگنال‌های ضمنی (مثل اینکه کاربر روی اسناد معرفی‌شده توسط کدام الگوریتم کلیک کرده و چقدر در آن‌ها مانده است)، الگوریتم برنده را تعیین می‌کنند. این دقیق‌ترین روش برای ارزیابی «مطلوبیت کاربر در عمل» است.

## پاسخنامه مسئله 8.14: بهینه‌سازی سیستم مستقر (Refining a deployed system)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، رویکرد ما بررسی چرخه حیات یک موتور جستجوی مستقر (**Deployed**) است که چگونه می‌تواند به طور خودکار از تعاملات کاربران بیاموزد و خود را ارتقا دهد. هدف، شناخت نحوه استخراج «سیگنال‌های آموزشی» از داده‌های خام کلیک، و همچنین درک یکی از بزرگ‌ترین خطاهای شناختی نهفته در این داده‌ها به نام «سوگیری موقعیت» است که می‌تواند الگوریتم را به اشتباه بیندازد.

### بخش 2) مفاهیم کلیدی

**لاگ‌های کلیک (Click Logs):** فایل‌های ثبت وقایع که نشان می‌دهند برای هر پرسمان، چه نتایجی به کاربر نمایش داده شده و کاربر روی کدامیک کلیک کرده است.

**ترجیح نسبی (Relative Preference):** استنتاج اینکه کاربر یک سند خاص را به اسنادی که بالاتر از آن بوده‌اند اما کلیک نشده‌اند، ترجیح داده است.

**سوگیری موقعیت (Position Bias / Trust Bias):** تمایل روانی و ناخودآگاه کاربران وب به کلیک بر روی نتایج اولیه لیست، صرفاً به دلیل جایگاه آن‌ها، نه به دلیل ارتباط واقعی محتوا.

### بخش 3) حل گام‌به‌گام

#### گام اول: نحوه استفاده از لاگ‌های کلیک

موتورهای جستجو به جای اینکه کلیک را نشانه «ارتباط مطلق» بدانند، از آن برای درک «ارتباط نسبی» استفاده می‌کنند. اگر کاربری نتایج **1** تا **5** را ببیند و فقط روی نتیجه شماره **4** کلیک کند، سیستم این‌گونه استنتاج می‌کند که: از دید کاربر، سند **4** مرتبط‌تر از اسناد **1**، **2** و **3** بوده است (چون کاربر آن‌ها را دیده، رد کرده و به سند **4** رسیده است). این مقایسه‌های زوجی (**Pairwise**) پایگاه داده‌ای از ترجیحات کاربران می‌سازد.

#### گام دوم: بهینه‌سازی الگوریتم رتبه‌بندی

این زوج‌های استخراج‌شده (مثلاً سند **4** بهتر از سند **1** است) به عنوان داده‌های آموزشی (**Training Data**) به الگوریتم‌های یادگیری ماشین داده می‌شوند. الگوریتم با تحلیل این داده‌ها، وزن پارامترهای رتبه‌بندی خود را طوری تغییر می‌دهد که در جستجوهای بعدی، اسنادی مشابه سند **4** در رتبه‌های بالاتری نسبت به اسنادی مشابه سند **1** قرار گیرند.

#### گام سوم: تحلیل چالش سوگیری موقعیت

چالش اساسی این است که داده‌های کلیک به شدت نويز دارند. کاربران وب تمایل دارند به موتور جستجو «اعتماد» کنند و غالباً

بدون دقت به محتوای واقعی، روی اولین لینک کلیک می‌کنند.

اگر الگوریتم به صورت خام به این کلیک‌ها تکیه کند، یک حلقه بازخورد معیوب (**Feedback Loop**) ایجاد می‌شود: سندی که در رتبه اول است به دلیل جایگاهش کلیک زیادی می‌خورد، الگوریتم به اشتباه تصور می‌کند این سند بسیار مرتبط است، پس آن را در رتبه اول نگه می‌دارد (پدیده «ثروتمند، ثروتمندتر می‌شود»). این سوگیری، ارزیابی واقعی میزان مطلوبیت اسناد رتبه‌های پایین‌تر را به شدت مختل می‌کند.

## بخش 4) پاسخ نهایی

**استفاده از لاگ‌ها برای بهبود مستمر:** سیستم از لاگ‌های کلیک برای استخراج «ترجیحات نسبی زوجی» (مثلاً ترجیح سند کلیک‌شده نسبت به اسناد کلیک‌نشده در رتبه‌های بالاتر) استفاده می‌کند. این ترجیحات سپس به الگوریتم‌های یادگیری ماشین خورانده می‌شوند تا وزن ویژگی‌های رتبه‌بندی را به طور خودکار اصلاح و بهینه کنند.

**چالش سوگیری موقعیت:** تمایل ذاتی کاربران به کلیک بر روی نتایج بالای لیست (به ویژه رتبه اول) صرف نظر از کیفیت واقعی آن اسناد است. این سوگیری باعث می‌شود اسنادی که صرفاً جایگاه بهتری دارند کلیک بیشتری دریافت کنند و الگوریتم را در تشخیص ارتباط واقعی اسناد دچار خطای محاسباتی سازد.

### نکته:

در مهندسی سیستم‌های جستجوی پیشرفته، به این شاخه که الگوریتم‌ها از روی داده‌های کلیک رتبه‌بندی را یاد می‌گیرند، یادگیری رتبه‌بندی (**LTR Learning to Rank**) گفته می‌شود. برای غلبه بر چالش «سوگیری موقعیت»، مهندسان از مدل‌های کلیک (**Click Models**) مانند «مدل آبشاری» (**Cascade Model**) استفاده می‌کنند. در این مدل‌ها فرض بر این است که کاربر نتایج را به صورت متوالی از بالا به پایین بررسی می‌کند؛ بنابراین کلیک روی رتبه اول ارزش محاسباتی بسیار کمتری نسبت به کلیک روی رتبه 5 دارد (زیرا رسیدن به رتبه 5 نشان‌دهنده دقت و جستجوی واقعی کاربر در لیست است، در حالی که کلیک روی رتبه اول ممکن است صرفاً از روی عادت یا اعتماد بی‌مورد باشد).

## پاسخنامه مسئله 8.15: تولید اسنیپت‌های پویا (Dynamic Snippet Generation)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، به مکانیزم مهندسی پنهان در پشت تولید «اسنیپت‌های پویا» (**Dynamic** یا **Query-biased Snippets**) می‌پردازیم. همان‌طور که می‌دانیم، این اسنیپت‌ها در لحظه و بر اساس پرسمان کاربر ساخته می‌شوند. چالش اصلی این است که متن اسناد ممکن است بسیار طولانی باشد و سیستم باید در کسری از ثانیه تشخیص دهد کدام بخش از متن، بهترین بستر معنایی (**Context**) را برای نمایش کلمات جستجو شده فراهم می‌کند. خواسته مسئله، بررسی تکنیک‌ها و الگوریتم‌های متنی مورد استفاده در این فرآیند است.

### بخش 2) مفاهیم کلیدی

**پنجره لغزان (Sliding Window):** تکنیکی که در آن متن به قطعات یا قاب‌های کوچک‌تر (مثلاً دنباله‌ای از کلمات یا جملات) تقسیم می‌شود تا به صورت جداگانه ارزیابی شوند.

**تراکم کلمات (Term Density):** فاصله‌ی بین کلمات کلیدی در داخل یک جمله. هرچه کلمات به یکدیگر نزدیک‌تر باشند، احتمال اینکه آن جمله دقیقاً به موضوع پرسمان اشاره داشته باشد، بیشتر است.

**برجسته‌سازی (Highlighting):** عملیات بصری در واسط کاربری (معمولاً با استفاده از تگ‌های **<b>** یا **<strong>** در HTML) برای جلب توجه چشم کاربر به کلمات کلیدی.

## بخش 3) حل گام به گام

تولید یک اسنippet پویای باکیفیت، معمولاً طی یک الگوریتم 3 مرحله‌ای انجام می‌شود:

### گام اول: بخش‌بندی متن (Text Segmentation)

ابتدا سیستم باید متن خام سند را به واحدهای منطقی کوچک‌تر بشکند. این کار معمولاً به 2 روش انجام می‌شود:

1. **بخش‌بندی مبتنی بر جمله (Sentence-based):** استفاده از علائم نگارشی (مانند نقطه، علامت سؤال و تعجب)

برای استخراج جملات کامل.

2. **بخش‌بندی مبتنی بر پنجره (Window-based):** ایجاد قاب‌هایی با طول ثابت (مثلاً پنجره‌هایی شامل 20 تا 30

کلمه) که کلمات کلیدی پرسمان در مرکز آن‌ها قرار دارند.

### گام دوم: امتیازدهی به قطعات (Heuristic Scoring)

پس از استخراج جملات یا پنجره‌ها، سیستم از یک «تابع امتیازدهی اکتشافی» برای ارزیابی هر قطعه استفاده می‌کند. جمله‌ای بیشترین امتیاز را می‌گیرد که ویژگی‌های زیر را داشته باشد:

1. **تعداد کلمات متمایز (Distinct Query Terms):** قطعه‌ای که شامل کلمات کلیدی بیشتری از پرسمان باشد، امتیاز

بالتری دارد (مثلاً اگر پرسمان "خرید لپ‌تاپ ایسوس" است، جمله‌ای که هر 3 کلمه را داشته باشد بهتر از جمله‌ای است که فقط کلمه "لپ‌تاپ" را دارد).

2. **مجاورت و تراکم (Proximity & Density):** اگر کلمات کلیدی در داخل جمله دقیقاً در کنار هم (به صورت 1 عبارت

دقیق) یا با فاصله بسیار کمی از هم ظاهر شده باشند، امتیاز به شدت افزایش می‌یابد.

3. **موقعیت ساختاری (Structural Position):** به جملاتی که در بخش‌های مهم‌تر سند قرار دارند (مانند تیترها،

پاراگراف اول یا چکیده مقاله) وزن بیشتری داده می‌شود.

4. **خوانایی (Readability):** جملاتی که خیلی کوتاه یا خیلی طولانی باشند، جریمه (Penalize) می‌شوند تا اسنippet

نهایی از نظر بصری برای کاربر مناسب باشد.

### گام سوم: انتخاب، ترکیب و قالب‌بندی (Selection & Formatting)

در نهایت، سیستم قطعه (یا قطعاتی) که بالاترین امتیاز را کسب کرده‌اند انتخاب می‌کند. اگر 1 قطعه برای پر کردن فضای

اسنippet (معمولاً حدود 160 کاراکتر در گوگل) کافی نباشد، سیستم ممکن است 2 قطعه‌ی برتر از بخش‌های مختلف متن را

انتخاب کرده و آن‌ها را با علامت سه‌نقطه (...) به یکدیگر متصل کند. در آخرین مرحله، کلمات کلیدی منطبق با پرسمان، با فونت

ضخیم (Bold) علامت‌گذاری می‌شوند تا به راحتی توسط چشم کاربر اسکن شوند.

## بخش 4) پاسخ نهایی

برای تولید اسنippet‌های پویا، سیستم‌های بازتابی اطلاعات از الگوریتمی 3 مرحله‌ای استفاده می‌کنند:

1. **بخش‌بندی متن با استفاده از تکنیک‌های پردازش زبان طبیعی (مانند تشخیص مرز جملات) یا الگوریتم «پنجره**

لغزان».

2. امتیازدهی اکتشافی (**Heuristic Scoring**) به هر قطعه بر اساس عواملی نظیر: تعداد کلمات کلیدی موجود،

مجاورت کلمات کلیدی با یکدیگر (تراکم کلمات)، و موقعیت مکانی قطعه در ساختار کلی سند.

3. ترکیب و برجسته‌سازی که در آن قطعات برتر انتخاب شده، در صورت نیاز با (...) به هم پیوند می‌خورند و کلمات

پرسمان برای جلب توجه کاربر و نمایش بستر معنایی، پررنگ (**Bold**) می‌شوند.

#### نکته:

از منظر معماری و کارایی سیستم (**System Efficiency**)، تولید اسنپت پویا یک «گلوگاه» (**Bottleneck**) جدی است. از آنجا که این عملیات باید دقیقاً در زمان پرسش (**Query Time**) انجام شود، سیستم فرصت ندارد متن کامل اسناد را از روی دیسک‌های گُند بازخوانی کند. به همین دلیل، موتورهای جستجو مجبورند متن خام (یا حداقل بخش‌های مهمی از متن) تمام اسناد را در حافظه‌های پنهان بسیار سریع (**Document Cache**) نگهداری کنند.

علاوه بر این، در سال‌های اخیر با پیشرفت هوش مصنوعی، موتورهای جستجو مانند گوگل در قابلیت‌هایی نظیر **Featured Snippets** از مدل‌های زبانی بزرگ (**LLM**) برای فهم معنایی پرسمان و استخراج مستقیم پاسخ استفاده می‌کنند که فراتر از الگوریتم‌های تطبیق کلمه کلیدی سنتی عمل می‌کنند.

## پاسخنامه مسئله 8.16: مجموعه تست کرنفیلد (Cranfield test collection)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، سیر تطور و تکامل محیط‌های آزمایشگاهی در سیستم‌های بازیابی اطلاعات مورد بررسی قرار می‌گیرد. هدف، شناخت ریشه‌های تاریخی ارزیابی استاندارد (پارادایم کرنفیلد) و درک محدودیت‌های مقیاس‌پذیری آن است که منجر به ظهور مجموعه‌های مدرن و عظیم مانند **TREC** برای پاسخگویی به نیازهای پیچیده موتورهای جستجوی وب شد.

### بخش 2) مفاهیم کلیدی

**پارادایم کرنفیلد (Cranfield Paradigm):** مدل پایه‌ای ارزیابی سیستم‌های بازیابی اطلاعات که شامل 3 جزء اساسی

است: مجموعه اسناد، پرسمان‌ها و قضاوت‌های ارتباط جامع (**Exhaustive Relevance Judgments**).

**مجموعه TREC:** کنفرانس بازیابی متن (**Text REtrieval Conference**) که توسط موسسه ملی استاندارد و فناوری

آمریکا (**NIST**) پایه‌گذاری شد و ارزیابی در مقیاس بزرگ را ممکن ساخت.

**قضاوت ارتباط جامع در برابر روش تجمیع (Pooling):** تفاوت کلیدی در نحوه یافتن اسناد مرتبط در مجموعه‌های

کوچک (کلاسیک) و مجموعه‌های بزرگ (وب).

### بخش 3) حل گام‌به‌گام

**گام اول: تبیین نقش تاریخی پروژه کرنفیلد**

پروژه کرنفیلد (در دهه‌های 1950 و 1960 میلادی به سرپرستی سیریل کلوردون) سنگ‌بنای ارزیابی علمی در موتورهای جستجو است. این پروژه برای اولین بار رویکرد «ارزیابی مبتنی بر مجموعه آزمون استاندارد» را ابداع کرد و نشان داد که می‌توان عملکرد سیستم‌های مختلف را در یک محیط کنترل‌شده آزمایشگاهی مقایسه کرد. همچنین، معرفی مفاهیم بنیادین «دقت» (**Precision**) و «فراخوانی» (**Recall**) به عنوان معیارهای استاندارد ارزیابی، خروجی مستقیم همین پروژه محسوب می‌شود.

## گام دوم: بررسی ویژگی‌ها و محدودیت‌های کرنفیلد

مجموعه کرنفیلد بسیار کوچک بود (نسخه معروف آن شامل حدود 1400 سند علمی در حوزه هوافضا و 225 پرسمان بود). به دلیل این اندازه کوچک، پژوهشگران می‌توانستند «ارزیابی جامع» (**Exhaustive Evaluation**) انجام دهند؛ یعنی هر پرسمان را با تک‌تک 1400 سند مقایسه کرده و ارتباط آن‌ها را توسط داوران انسانی قضاوت کنند تا حقیقت پایه (**Ground Truth**) کاملاً بی‌نقصی به دست آید.

## گام سوم: تحلیل دلایل گذار به TREC و ارزیابی وب

با ظهور اینترنت و کلان‌داده‌ها، پارادایم سنتی کرنفیلد با شکست مواجه شد. در محیط وب که شامل میلیاردها صفحه است، مقایسه یک پرسمان با تمامی اسناد موجود برای یافتن «کل اسناد مرتبط» کاملاً غیرممکن و خارج از توان انسان است. مجموعه‌های **TREC** (از سال 1992) برای حل این مشکل ایجاد شدند.

**TREC** به جای ارزیابی کل پایگاه داده، از تکنیکی به نام تجمیع (**Pooling**) استفاده می‌کند. در این روش، تنها اسناد برتری (مثلاً 100 سند اول) که توسط موتورهای جستجوی شرکت‌کننده در مسابقه برای یک پرسمان ارزیابی شده‌اند، در یک «استخر» جمع‌آوری می‌شوند. داوران انسانی فقط همین اسناد منتخب را بررسی می‌کنند. این روش، ارزیابی دقیق الگوریتم‌ها را در مقیاس میلیون‌ها سند با هزینه‌ای منطقی ممکن ساخت.

## بخش 4) پاسخ نهایی

**نقش تاریخی کرنفیلد:** این پروژه پایه‌گذار ارزیابی سیستماتیک و کمی در علم ارزیابی اطلاعات بود و معماری

«مجموعه‌های آزمون استاندارد»، و همچنین معیارهای جهانی «دقت» و «فراخوانی» را به دنیای فناوری معرفی کرد.

**دلیل جایگزینی با TREC:** کرنفیلد مبتنی بر مجموعه‌های بسیار کوچک و قضاوت انسانی بر روی تک‌تک اسناد بود که این رویکرد در مقیاس بی‌نهایت وب کاملاً غیرعملی است. مجموعه‌های **TREC** با معرفی تکنیک «تجمیع» (**Pooling**)، نیاز به قضاوت درباره تمام اسناد را حذف کردند و زیرساخت لازم برای ارزیابی پایگاه‌های داده عظیم (**Large-scale**) را فراهم آوردند.

### نکته:

علاوه بر مشکل مقیاس‌پذیری و حجم داده، پایگاه داده کرنفیلد فاقد ویژگی‌های ساختاری مختص به شبکه اینترنت بود. در اسناد کرنفیلد مفاهیمی مانند «لینک‌های فرامتنی» (**Hyperlinks**)، «هرزنامه وب» (**Spam**)، «ساختار تگ‌های **HTML**» و «رفتار کلیک کاربر» وجود نداشت. مجموعه‌های آزمون مدرن (مانند **TREC Web Tracks**) به گونه‌ای طراحی شده‌اند که گراف وب و ساختار لینک‌ها (که پایه و اساس الگوریتم‌های مدرنی مانند **PageRank** هستند) را در خود جای داده‌اند تا شرایط محیط واقعی وب را به بهترین شکل شبیه‌سازی کنند.

## پاسخنامه مسئله 8.17: محاسبه معیار R-Precision (R-Precision Calculation)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، با معیار ارزیابی دیگری به نام «دقت در نقطه **R**» یا **R-Precision** سروکار داریم. بر خلاف معیار **Precision@K** که در آن نقطه برش (**K**) برای تمامی پرسمان‌ها یک عدد ثابت و از پیش‌تعیین‌شده است (مانند بررسی همیشه 10 نتیجه اول)، در این معیار، نقطه برش برای هر پرسمان به صورت کاملاً پویا و بر اساس تعداد کل اسناد مرتبط با همان پرسمان تعیین می‌گردد. هدف مسئله، تعریف این معیار و انجام یک محاسبه ساده برای درک مکانیزم آن است.

**نقطه برش متغیر (Variable Cut-off Point):** در بازیابی اطلاعات، پرسمان‌های مختلف تعداد پاسخ‌های متفاوتی دارند. یک پرسمان ممکن است فقط 3 سند مرتبط در کل مجموعه داشته باشد و پرسمان دیگر 100 سند. ارزیابی هر 2 پرسمان در یک نقطه ثابت (مثلاً  $P@10$ ) از نظر آماری می‌تواند گمراه‌کننده باشد.

**معیار R-Precision:** این معیار نقطه برش لیست نتایج را دقیقاً روی عدد R تنظیم می‌کند، که R برابر است با کل اسناد مرتبط موجود برای آن پرسمان در پایگاه داده (حقیقت پایه).

## بخش 3) حل گام به گام

### گام اول: تعریف رسمی معیار

معیار **R-Precision** به این صورت تعریف می‌شود: اگر برای یک پرسمان خاص، تعداد کل اسناد مرتبط در مجموعه داده برابر با R باشد، مقدار **R-Precision** برابر است با میزان دقت (**Precision**) سیستم دقیقاً در R سند اول بازیابی شده. به بیان ساده‌تر، این معیار بررسی می‌کند که اگر سیستم لیست نتایج را به اندازه تعداد کل جواب‌های موجود برش دهد، چه کسری از این لیست با جواب‌های درست پر شده است.

### گام دوم: استخراج داده‌ها و تعیین نقطه برش

بر اساس صورت مسئله:

کل اسناد مرتبط در پایگاه داده برای این پرسمان (R): 12 سند.

بنابراین، ما باید عملکرد سیستم را فقط تا رتبه دوازدهم ارزیابی کنیم (نقطه برش ما  $K = 12$  است).

تعداد اسناد مرتبطی که سیستم موفق شده در این 12 جایگاه اول قرار دهد: 4 سند.

### گام سوم: محاسبه ریاضی

فرمول محاسبه همان فرمول کلاسیک دقت در نقطه مشخص است:

$$R\text{-Precision} = \frac{|\text{Relevant Documents in Top } R|}{R} = \frac{4}{12} = \frac{1}{3} \approx 0.333$$

## بخش 4) پاسخ نهایی

**تعریف: R-Precision** معیاری است که دقت سیستم جستجو را دقیقاً در نقطه برشی برابر با تعداد کل اسناد مرتبط موجود برای پرسمان (R) اندازه‌گیری می‌کند.

**محاسبه مسئله:** با توجه به اینکه کل اسناد مرتبط 12 عدد است، ما دقت را در 12 نتیجه اول می‌سنجیم. چون 4 سند از این 12 نتیجه مرتبط هستند، مقدار **R-Precision** برابر با کسر  $1/3$  یا 0.333 خواهد بود.

### نکته مهم:

معیار **R-Precision** دارای یک ویژگی ریاضی بسیار جذاب و منحصر به فرد است. اگر در نقطه R فرمول‌های دقت و فراخوانی را بنویسیم، به نتیجه جالبی می‌رسیم:

**فرمول دقت در نقطه R:** (تعداد اسناد مرتبط یافت شده در R رتبه اول) تقسیم بر (کل اسناد بازیابی شده تا این نقطه که برابر با R است).

**فرمول فراخوانی در نقطه R:** (تعداد اسناد مرتبط یافت شده در R رتبه اول) تقسیم بر (کل اسناد مرتبط موجود در مجموعه که آن هم دقیقاً برابر با R است).

همان‌طور که می‌بینید، مخرج هر 2 کسر در این نقطه خاص برابر با R می‌شود. بنابراین، به عنوان یک قانون ثابت ریاضی در علم بازیابی اطلاعات: در نقطه برش R، همواره مقدار **Precision** دقیقاً با مقدار **Recall** برابر است. به همین دلیل، **R-Precision** معیاری است که به تنهایی می‌تواند خلاصه‌ای از هر 2 فاکتور دقت و فراخوانی را به صورت همزمان به ما نشان دهد.

## پاسخنامه مسئله 8.18: توافق داوران در ارتباط (Inter-judge agreement)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، با یکی از چالش‌های عملی و بسیار مهم در فرآیند ساخت مجموعه‌های آزمون (**Test Collections**) مواجه هستیم. حقیقت پایه (**Ground Truth**) باید یک مرجع کاملاً قابل‌اتکا و بدون ابهام باشد تا بتوان عملکرد سیستم‌های مختلف را بر اساس آن سنجید. با این حال، به دلیل ماهیت ذهنی مفهوم «ارتباط»، داوران انسانی ممکن است در قضاوت‌های خود دچار اختلاف شوند. خواسته مسئله، بررسی روش‌های مهندسی و استاندارد است که در زمان بروز اختلافات گسترده میان داوران، برای رسیدن به یک قضاوت واحد و معتبر به کار گرفته می‌شوند.

### بخش 2) مفاهیم کلیدی

**حقیقت پایه (Ground Truth):** مجموعه‌ای از داده‌های برچسب‌گذاری شده که به عنوان معیار قطعی درستی و صحت در نظر گرفته می‌شود.

**دستورالعمل‌های ارزیابی (Annotation Guidelines):** سند مکتوبی که تعاریف، قوانین و مثال‌هایی را برای داوران مشخص می‌کند تا بر اساس یک استاندارد واحد قضاوت کنند.

**رفع اختلاف (Adjudication):** فرآیند سیستماتیک حل و فصل عدم توافق بین 2 یا چند داور انسانی.

### بخش 3) حل گام‌به‌گام

هنگامی که 2 داور بر روی مرتبط بودن یا نبودن اسناد اختلاف نظر زیادی دارند (که در آمار با پایین بودن ضریب کاپای کوهن یا **K** نشان داده می‌شود)، پژوهشگران حوزه بازیابی اطلاعات از 4 راهکار استاندارد زیر استفاده می‌کنند:

#### گام اول: بازبینی و اصلاح دستورالعمل‌ها (Guideline Refinement)

وجود اختلاف نظر زیاد و سیستماتیک، معمولاً نشان‌دهنده خطای انسانی نیست، بلکه نشان‌دهنده ابهام در تعریف مسئله است.

**اقدام:** فرآیند برچسب‌زنی بلافاصله متوقف می‌شود. طراحان مجموعه آزمون باید «دستورالعمل ارزیابی» را بازنویسی کنند، مفاهیم مبهم پرسمان را شفاف کرده و برای حالت‌های مرزی (**Edge Cases**) که باعث اختلاف شده‌اند، مثال‌های دقیقی به سند اضافه کنند. سپس از داوران خواسته می‌شود اسناد مورد اختلاف را مجدداً بر اساس قوانین جدید ارزیابی کنند.

#### گام دوم: برگزاری جلسات اجماع (Consensus Building)

در مواردی که اختلاف نظر محدود به چند سند خاص و پیچیده است، از روش اجماع استفاده می‌شود.

**اقدام: 2** دآوری که اختلاف نظر دارند، در یک جلسه مشترک حضور یافته و استدلال‌های خود را برای انتخاب برچسب «مرتبط» یا «نامرتب» بیان می‌کنند. این تبادل نظر اغلب باعث می‌شود یکی از داوران متوجه نکته‌ای پنهان در سند شود و با تغییر رأی خود، هر **2** به یک توافق منطقی (حقیقت پایه) دست یابند.

### گام سوم: دآوری شخص ثالث یا ارزیاب ارشد (Third-party Adjudication)

اگر داوران نتوانند از طریق بحث به اجماع برسند، گره‌گشایی به یک مرجع بالاتر سپرده می‌شود.

**اقدام:** یک داور سوم که معمولاً تخصص بیشتری دارد (مانند محقق ارشد پروژه یا یک متخصص دامنه یا **Domain Expert**)، اسناد مورد اختلاف را بررسی کرده و رأی نهایی و قطعی (**Tie-breaking vote**) را صادر می‌کند. رأی این شخص به عنوان حقیقت پایه ثبت خواهد شد.

### گام چهارم: گذار از ارتباط دودویی به ارتباط طیفی (Graded Relevance)

گاهی اوقات اختلاف به این دلیل است که سیستم ارزیابی فقط **2** حالت (**0** و **1** / مرتبط و نامرتب) دارد و سند مورد نظر در منطقه خاکستری است.

**اقدام:** به جای اجبار داوران به انتخاب مطلق، مقیاس قضاوت به صورت طیفی (مثلاً از **1** تا **4**: کاملاً مرتبط، نسبتاً مرتبط، حاشیه‌ای، نامرتب) تغییر می‌کند. در این حالت، اختلاف‌های کوچک میان داوران قابل جمع‌بندی است و می‌توان از میانگین نظرات یا ادغام آن‌ها برای محاسبه معیارهای پیشرفته‌تری مانند **NDCG** استفاده کرد.

### بخش 4) پاسخ نهایی

برای رسیدن به یک "حقیقت پایه" در زمان بروز اختلاف نظر شدید میان **2** داور، راهکارهای استاندارد زیر پیشنهاد و اجرا می‌شوند:

**1. اصلاح دستورالعمل‌های ارزیابی:** رفع ابهامات موجود در تعاریف و ارائه مثال‌های شفاف برای حالت‌های مرزی، و سپس ارزیابی مجدد.

**2. جلسات اجماع:** بحث و تبادل استدلال بین **2** داور برای رسیدن به یک درک مشترک و توافق دوجانبه.

**3. دآوری ارشد (شخص ثالث):** ارجاع اسناد مورد اختلاف به یک متخصص ارشد برای صدور رأی نهایی و قطعی (شکستن بن‌بست).

**4. استفاده از مقیاس طیفی:** تغییر سیستم قضاوت از حالت دودویی (بله/خیر) به ارتباط چندسطحی، تا امکان میانگین‌گیری و کاهش خطای مطلق فراهم شود.

#### نکته:

در پروژه‌های مدرن و در مقیاس وب (مانند استفاده از پلتفرم‌های جمع‌سپاری یا **Crowdsourcing** نظیر **Amazon Mechanical Turk**)، معمولاً از ارزیابی توسط **2** نفر استفاده نمی‌شود، بلکه هر پرسمان به **5** الی **7** داور ناشناس سپرده می‌شود. در چنین شرایطی، راهکار حل اختلاف بسیار ساده‌تر و مبتنی بر قانون «رأی اکثریت» (**Majority Voting**) است. سندی که اکثریت داوران (مثلاً **4** نفر از **5** نفر) آن را مرتبط تشخیص دهند، به عنوان مرتبط در حقیقت پایه ثبت شده و نظر اقلیت به عنوان نویز آماری در نظر گرفته شده و حذف می‌گردد. با این حال، در مجموعه‌های تخصصی (پزشکی، حقوقی) همچنان نظارت دقیق و دآوری شخص ثالث (**Adjudication**) بالاترین استاندارد ممکن محسوب می‌شود.

## بخش 1) تحلیل اولیه مسئله

در این مسئله، هدف سنجش میزان پایایی و اعتبار قضاوت‌های انسانی در فرآیند ساخت یک مجموعه آزمون است. زمانی که از ارزیابان متعدد (مانند گروهی متشکل از **100** دانشجو) برای برچسب‌زنی استفاده می‌کنیم، همواره احتمال دارد که آن‌ها به صورت کاملاً تصادفی و شانسی با یکدیگر هم‌نظر شوند. آماره «کاپای کوهن» (**Cohen's Kappa**) یک معیار آماری بسیار قدرتمند است که این «توافقات تصادفی» را از محاسبات حذف می‌کند تا ارزش واقعی و خالص هماهنگی میان **2** ارزیاب را به دست آورد.

## بخش 2) مفاهیم کلیدی

**توافق مشاهده‌شده ( $P_o$ ):** نسبت کل اسنادی که **2** ارزیاب روی آن‌ها دقیقاً نظر یکسانی داشته‌اند (اعم از مرتبط یا نامرتب) به کل اسناد.

**توافق تصادفی ( $P_e$ ):** احتمالی که **2** ارزیاب صرفاً بر اساس توزیع پاسخ‌هایشان و به صورت تصادفی با هم هم‌نظر شوند.

**کاپا ( $\kappa$ ):** نسبتی که نشان می‌دهد چه مقدار از توافقات‌های ممکن فراتر از شانس، واقعاً محقق شده است. بازه این عدد معمولاً بین **0** تا **1** است.

## بخش 3) حل گام‌به‌گام

**گام اول: تشکیل جدول توزیع فراوانی (ماتریس درهم‌ریختگی)**

برای درک بهتر داده‌ها، ابتدا آن‌ها را در یک جدول استاندارد **2** در **2** مرتب کرده و مجموع حاشیه‌ای (**Marginal Totals**) سطرها و ستون‌ها را محاسبه می‌کنیم:

| ارزیاب 2 (مرتبط)    | ارزیاب 2 (نامرتبط) | جمع سطر (ارزیاب 1) |                |
|---------------------|--------------------|--------------------|----------------|
| ارزیاب 1 (مرتبط)    | 15 (توافق)         | 10 (اختلاف)        | 25             |
| ارزیاب 1 (نامرتبط)  | 5 (اختلاف)         | 20 (توافق)         | 25             |
| جمع ستون (ارزیاب 2) | 20                 | 30                 | جمع کل: 50 سند |

**گام دوم: محاسبه توافق مشاهده‌شده ( $P_o$ )**

تعداد اسنادی که هر **2** ارزیاب روی آن‌ها توافق داشته‌اند (**15** سند مرتبط + **20** سند نامرتب) برابر با **35** است.

$$P_o = \frac{15 + 20}{50} = \frac{35}{50} = 0.70$$

(**2**) دانشجو در **70%** پس مواقع با هم هم‌نظر بوده‌اند)

**گام سوم: محاسبه احتمال توافق تصادفی ( $P_e$ )**

برای محاسبه این بخش، باید احتمال اینکه هر **2** ارزیاب به صورت تصادفی «مرتبط» یا هر **2** «نامرتب» را انتخاب کنند، به دست آوریم.

احتمال اینکه ارزیاب **1** سندی را «مرتبط» بداند (**25** مورد از **50**): **25 / 50 = 0.50**

احتمال اینکه ارزیاب **2** سندی را «مرتبط» بداند (**20** مورد از **50**): **20 / 50 = 0.40**

احتمال توافق تصادفی روی «مرتبط» بودن:

$$P(\text{Both Relevant}) = 0.50 \times 0.40 = 0.20$$

احتمال اینکه ارزیاب 1 سندی را «نامرتبط» بداند (25 مورد از 50):  $25 / 50 = 0.50$

احتمال اینکه ارزیاب 2 سندی را «نامرتبط» بداند (30 مورد از 50):  $30 / 50 = 0.60$

احتمال توافق تصادفی روی «نامرتبط» بودن:

$$P(\text{Both Non-Relevant}) = 0.50 \times 0.60 = 0.30$$

مجموع احتمال توافق تصادفی:

$$P_e = 0.20 + 0.30 = 0.50$$

( پس اگر این 2 دانشجو بدون خواندن اسناد و فقط با شیر یا خط سکه تصمیم می‌گرفتند، در 50% مواقع با هم هم‌نظر می‌شدند.)

گام چهارم: محاسبه آماره کاپا ( $\kappa$ )

اکنون مقادیر به دست آمده را در فرمول اصلی کاپای کوهن قرار می‌دهیم:

$$\kappa = \frac{P_o - P_e}{1 - P_e} = \frac{0.70 - 0.50}{1 - 0.50} = \frac{0.20}{0.50} = 0.40$$

بخش 4) پاسخ نهایی

مقدار توافق مشاهده‌شده ( $P_o$ ): 0.70

مقدار توافق تصادفی مورد انتظار ( $P_e$ ): 0.50

مقدار نهایی کاپای کوهن ( $\kappa$ ): 0.40

نکته:

بر اساس مقیاس استاندارد تفسیر کاپا (Landis & Koch)، مقدار  $\kappa = 0.40$  دقیقاً در مرز توافق «ضعیف / نسبی» (Fair) و «متوسط» (Moderate) قرار می‌گیرد.

آیا این سطح از توافق قابل قبول است؟

خیر. در مهندسی بازایی اطلاعات و ساخت مجموعه‌های آزمون مرجع (مانند TREC)، این سطح از توافق برای ایجاد یک «حقیقت پایه» (Ground Truth) غیرقابل قبول و مردود ارزیابی می‌شود. استاندارد طلایی برای یک مجموعه دادگان باکیفیت، مقدار کاپای بالاتر از 0.60 (توافق قابل توجه) و در بهترین حالت بالاتر از 0.80 (توافق عالی) است.

وجود چنین نمره‌ای در نمونه‌گیری 50 سندی نشان می‌دهد که اگر کار برچسب‌زنی را با تمام 100 دانشجو روی کل مجموعه پیش ببرید، دادگان نهایی شما پر از نویز و داده‌های متناقض خواهد بود. برای رفع این مشکل پیش از شروع فاز اصلی ارزیابی، باید:

1. دستورالعمل‌های ارزیابی و تعریف «ارتباط» را بازنویسی و بسیار شفاف‌تر کنید.

2. برای دانشجویان جلسات آموزشی توجیهی بگذارید و مثال‌های نقض و حالت‌های مرزی را برای آن‌ها تحلیل کنید تا به یک درک ذهنی مشترک برسند.

## پاسخنامه مسئله 8.20: سناریوی محاسبه ★ (NDCG Calculation Scenario)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما از ارزیابی‌های دودویی (مرتبط/نامرتب) فاصله گرفته و وارد فضای «ارزیابی طیفی» (Graded Relevance) می‌شویم. در بسیاری از سیستم‌های واقعی مانند موتورهای جستجوی سازمانی یا وب، اسناد دارای درجات مختلفی از ارتباط هستند (مثلاً کاملاً مرتبط، نسبتاً مرتبط، حاشیه‌ای و نامرتب). برای سنجش عملکرد سیستم در این حالت، نیازمند معیاری هستیم که هم به «درجه ارتباط» و هم به «جایگاه (رتبه) سند» حساس باشد. این معیار **NDCG (Normalized Discounted Cumulative Gain)** نام دارد. هدف این مسئله، محاسبه عملی این معیار برای یک لیست 5 تایی از نتایج است.

### بخش 2) مفاهیم کلیدی

**بهره تجمعی (CG):** مجموع ساده نمرات ارتباط اسناد در لیست نتایج، بدون در نظر گرفتن رتبه آن‌ها.

**بهره تجمعی تخفیف‌یافته (DCG):** معیاری که نمره ارتباط هر سند را بر اساس جایگاه آن در لیست کاهش (تخفیف) می‌دهد. منطق آن این است که یک سند بسیار مرتبط اگر در رتبه اول باشد ارزش بسیار بیشتری دارد تا اینکه در رتبه دهم ظاهر شود. این تخفیف معمولاً با استفاده از لگاریتم در مخرج کسر اعمال می‌گردد.

**بهره تجمعی تخفیف‌یافته ایده‌آل (IDCG):** مقدار **DCG** برای حالتی که سیستم بهینه‌ترین عملکرد ممکن را داشته باشد؛ یعنی اسناد را دقیقاً به ترتیب نزولی نمرات ارتباطشان مرتب کند.

### بخش 3) حل گام‌به‌گام

**گام اول: معرفی فرمول استاندارد DCG**

رایج‌ترین فرمول ریاضی برای محاسبه **DCG** در نقطه **p** (در اینجا **p=5**) به شکل زیر است، که در آن **i** نشان‌دهنده رتبه سند و **rel<sub>i</sub>** نشان‌دهنده نمره ارتباط آن سند است:

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

(نکته: در این فرمول از  **$\log_2(i+1)$**  استفاده می‌شود تا در رتبه اول (**i=1**) مخرج کسر برابر با  **$\log_2(2)=1$**  شده و نمره سند اول بدون هیچ تخفیفی محاسبه شود).

**گام دوم: محاسبه DCG@5 برای سیستم فعلی**

لیست نمرات سیستم ما به این صورت است: [2, 0, 3, 1, 3]

رتبه‌ها (**i**) را از 1 تا 5 در نظر گرفته و محاسبات را انجام می‌دهیم:

رتبه 1 (rel=2):

$$\frac{2}{\log_2(1+1)} = \frac{2}{1} = 2.0$$

رتبه 2 (rel=0):

$$\frac{0}{\log_2(2+1)} = \frac{0}{\log_2(3)} = 0.0$$

رتبه 3 (rel=3):

$$\frac{3}{\log_2(3+1)} = \frac{3}{\log_2(4)} = \frac{3}{2} = 1.5$$

رتبه 4 (rel=1):

$$\frac{1}{\log_2(4+1)} = \frac{1}{\log_2(5)} \approx \frac{1}{2.322} \approx 0.431$$

رتبه 5 (rel=3):

$$\frac{3}{\log_2(5+1)} = \frac{3}{\log_2(6)} \approx \frac{3}{2.585} \approx 1.161$$

مجموع این مقادیر برابر با **DCG** سیستم ماست:

$$DCG@5 = 2.0 + 0.0 + 1.5 + 0.431 + 1.161 = 5.092$$

گام سوم: محاسبه **IDCG@5** (حالت ایده‌آل)

طبق صورت مسئله، بهترین چینش ممکن برای این پرسمان **[3, 3, 2, 1, 0]** است (یعنی اسناد با بالاترین نمرات در صدر لیست قرار گیرند). حال **DCG** را برای این لیست ایده‌آل محاسبه می‌کنیم:

رتبه 1 (rel=3):

$$\frac{3}{\log_2(2)} = \frac{3}{1} = 3.0$$

رتبه 2 (rel=3):

$$\frac{3}{\log_2(3)} \approx \frac{3}{1.585} \approx 1.893$$

رتبه 3 (rel=2):

$$\frac{2}{\log_2(4)} = \frac{2}{2} = 1.0$$

$$\frac{1}{\log_2(5)} \approx \frac{1}{2.322} \approx 0.431$$

$$\frac{0}{\log_2(6)} = 0.0$$

مجموع این مقادیر برابر با **IDCG** است:

$$IDCG@5 = 3.0 + 1.893 + 1.0 + 0.431 + 0.0 = 6.324$$

### گام چهارم: نرمال سازی و محاسبه NDCG

برای به دست آوردن **NDCG**، کافی است عملکرد سیستم (**DCG**) را بر عملکرد ایده آل (**IDCG**) تقسیم کنیم تا عددی بین 0 و 1 به دست آید:

$$NDCG@5 = \frac{DCG@5}{IDCG@5} = \frac{5.092}{6.324} \approx 0.805$$

### بخش 4) پاسخ نهایی

بر اساس محاسبات انجام شده:

مقدار **DCG@5** (عملکرد سیستم فعلی) برابر با **5.092** است.

مقدار **IDCG@5** (عملکرد ایده آل) برابر با **6.324** است.

مقدار نهایی **NDCG@5** برابر با **0.805** (یا **80.5%**) می باشد. این عدد نشان می دهد که الگوریتم رتبه بندی سیستم، تا حدود **80** درصد به حالت ایده آل ممکن نزدیک بوده است.

### نکته آموزشی:

شاید بپرسید چرا اصلاً به مرحله «نرمال سازی» (تقسیم بر **IDCG**) نیاز داریم و از همان مقدار **DCG** استفاده نمی کنیم؟ پاسخ در تفاوت ذاتی پرسمان ها نهفته است. در یک موتور جستجو، پرسمان **A** ممکن است کلاً ده ها سند با نمره ارتباط **3** داشته باشد (که باعث می شود **DCG** ایده آل آن بسیار بزرگ و مثلاً حدود **50** شود)، در حالی که پرسمان **B** ممکن است فقط **1** سند مرتبط با نمره **1** داشته باشد (که **DCG** ایده آل آن کلاً **1** است). اگر بخواهیم عملکرد موتور جستجو را روی میانگین این **2** پرسمان ارزیابی کنیم، پرسمان **A** به دلیل داشتن اعداد بزرگ تر، بر میانگین نهایی کاملاً مسلط می شود و خطای محاسباتی ایجاد می کند. عملیات نرمال سازی، امتیاز همه پرسمان ها را به یک بازه استاندارد (بین **0** تا **1**) می آورد تا بتوانیم آن ها را به صورت عادلانه با هم جمع بسته و میانگین گیری کنیم.

## بخش 1) تحلیل اولیه مسئله

در این مسئله، ما از ارزیابی‌های آزمایشگاهی و آفلاین (مانند مجموعه‌های تست **TREC**) خارج شده و وارد فاز ارزیابی برخط (**Online Evaluation**) در محیط عملیاتی می‌شویم. هدف این است که تأثیر یک تغییر الگوریتمی (اصلاح وزن‌دهی **tf-idf**) را بر روی کاربران واقعی موتور جستجوی دانشگاه بسنجیم. برای این کار نیازمند طراحی یک آزمون **A/B** استاندارد هستیم که بدون ایجاد اختلال در تجربه کاربری، داده‌های رفتاری لازم را برای تعیین الگوریتم برتر جمع‌آوری کند.

## بخش 2) مفاهیم کلیدی

**گروه کنترل (Control Group - A):** گروهی از کاربران که نسخه فعلی و دست‌نخورده الگوریتم (**Base tf-idf**) را تجربه می‌کنند.

**گروه آزمایش (Treatment Group - B):** گروهی از کاربران که نسخه جدید و اصلاح‌شده الگوریتم را تجربه می‌کنند.

**تخصیص تصادفی و پایدار (Consistent Random Assignment):** مکانیزمی که اطمینان حاصل می‌کند ترافیک به درستی تقسیم شده و تجربه یک کاربر واحد در طول زمان تغییر نمی‌کند.

## بخش 3) حل گام‌به‌گام

### گام اول: مکانیزم تقسیم‌بندی ترافیک (Traffic Routing)

برای انجام یک تست **A/B** علمی، ترافیک کاربران باید به صورت تصادفی اما «پایدار» تقسیم شود.

- شناسه مبنا:** سیستم باید بر اساس شناسه کاربری (**User ID**) دانشجویان واردشده به پورتال، یا در صورت عدم ورود، بر اساس کوکی مرورگر (**Session ID**) آن‌ها را دسته‌بندی کند. تخصیص بر اساس پرسمان (**Query**) اشتباه است، زیرا یک دانشجو نباید با جستجوی مجدد، ناگهان رفتار متفاوتی از سیستم ببیند.
- تابع درهم‌ساز (Hash Function):** شناسه کاربر به یک تابع درهم‌ساز داده می‌شود تا عددی بین 0 تا 99 تولید کند. اگر عدد کوچکتر از 50 بود، کاربر به گروه **A** (الگوریتم قدیم) و اگر 50 یا بیشتر بود، به گروه **B** (الگوریتم جدید) هدایت می‌شود (تقسیم 50/50).

### گام دوم: تعیین متریک‌های سنجش مطلوبیت کاربر (User Utility Metrics)

برای تعیین برنده، نمی‌توانیم از معیارهایی مانند **Precision** یا **Recall** استفاده کنیم، زیرا حقیقت پایه‌ای (**Ground Truth**) در کار نیست. در عوض، متریک‌های رفتاری زیر پایش می‌شوند:

- نرخ کلیک در رتبه‌های برتر (Top-k CTR):** بررسی می‌شود که در کدام الگوریتم، درصد بیشتری از کاربران روی اولین یا 3 نتیجه اول کلیک کرده‌اند. کلیک سریع‌تر و بالاتر، نشان‌دهنده دقت بهتر رتبه‌بندی است.
- زمان ماندگاری (Dwell Time):** از آنجا که اسناد دانشگاهی نیازمند مطالعه هستند، اگر کاربری روی سندی کلیک کند و زمان طولانی‌تری را در آن صفحه بگذراند، نشان‌دهنده مطلوبیت بالای آن سند است. (الگوریتمی که **Dwell Time** میانگین بالاتری تولید کند برنده است).
- نرخ بازنویسی پرسمان (Query Reformulation Rate):** اگر دانشجو پرسمانی را جستجو کند و بدون کلیک روی هیچ نتیجه‌ای، کلمات خود را تغییر دهد (یا به طور کلی از جستجو منصرف شود)، این یک نمره منفی محسوب می‌شود. کاهش این متریک در گروه **B** نشان‌دهنده موفقیت الگوریتم جدید است.

پس از جمع‌آوری داده‌ها، باید بررسی شود که آیا تفاوت بین متریک‌های گروه **A** و **B** از نظر آماری معنادار است یا صرفاً بر اثر شانس رخ داده است. برای این کار از آزمون‌های فرض آماری مانند آزمون تی (**t-test**) استفاده می‌شود. اگر مقدار  $p\text{-value} < 0.05$  باشد، می‌توانیم با اطمینان **95%** الگوریتم جدید را جایگزین الگوریتم قبلی کنیم.

## بخش 4) پاسخ نهایی

برای طراحی تست **A/B** روی الگوریتم **tf-idf**:

1. **تقسیم ترافیک:** کاربران بر اساس شناسه یکتا (**User ID / Session**) و با استفاده از یک تابع درهم‌ساز (**Hash**) به

صورت مساوی (**50%-50%**) و پایدار بین گروه کنترل (الگوریتم فعلی) و گروه آزمایش (الگوریتم جدید) تقسیم می‌شوند.

2. **متریک‌های تعیین برنده (مبتنی بر مطلوبیت کاربر):**

**افزایش CTR رتبه‌های بالا:** کلیک بیشتر روی نتایج 1 تا 3.

**افزایش Dwell Time:** مدت زمان طولانی‌تر ماندگاری دانشجویان در صفحه اسناد دانشگاهی کلیک‌شده.

**کاهش نرخ بازنویسی پرسمان:** کاهش دفعاتی که دانشجو مجبور می‌شود کلمات جستجوی خود را برای رسیدن به نتیجه مطلوب تغییر دهد.

### نکته:

در محیط‌های دانشگاهی یا پورتال‌های کلاسی، به دلیل محدود بودن جامعه آماری (مثلاً داشتن یک گروه کاربری شامل حدود **100** دانشجوی فعال)، حجم ترافیک و داده‌های کلیک به شدت پایین‌تر از موتورهای جستجوی وب است. در چنین شرایطی، رسیدن به معناداری آماری ( $p\text{-value} < 0.05$ ) در یک تست **A/B** کلاسیک ممکن است به هفته‌ها یا ماه‌ها زمان نیاز داشته باشد.

## پاسخنامه مسئله 8.22: الگوریتم ارزیاب خودکار (Autograder Algorithm)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، هدف گذر از محاسبات دستی و طراحی یک سیستم تصحیح خودکار (**Autograder**) است. توسعه یک اسکریپت کارآمد، به‌ویژه برای ارزیابی پروژه‌های بازیابی اطلاعات یک جامعه آماری بالا (مانند یک کلاس **100** نفره)، نیازمند کدی بهینه، ساختاریافته و بدون ابهام است. از آنجا که ساختار اولیه در صورت سؤال درج نشده بود، یک معماری استاندارد و جامع برای تابع محاسبه میانگین متوسط دقت (**MAP**) ارائه می‌شود. به منظور تطابق با استانداردهای اجرای خودکار و جلوگیری از خطاهای رمزگذاری (**Encoding**)، تمامی متغیرها و مستندات داخل کد منحصراً با کاراکترهای لاتین تدوین شده‌اند.

### بخش 2) مفاهیم کلیدی

**متوسط دقت (AP):** میانگین مقادیر دقت در نقاطی که اسناد مرتبط بازیابی شده‌اند، تقسیم بر کل اسناد مرتبط یک پرسمان.

**میانگین متوسط دقت (MAP):** میانگین حسابی مقادیر **AP** برای مجموعه‌ای از پرسمان‌ها. این معیار، استاندارد طلایی برای ارزیابی کلی یک سیستم بازیابی اطلاعات است.

### بخش 3) حل گام به گام

#### گام اول: طراحی معماری توابع

برای حفظ خوانایی و رعایت اصول مهندسی نرم افزار، مسئله به 2 تابع مجزا شکسته می شود:

1. تابع `calculate_average_precision`: مسئولیت محاسبه **AP** برای یک پرسمان منفرد را بر عهده دارد.

2. تابع `calculate_map`: نتایج دانشجو را برای تمامی پرسمان ها دریافت کرده، تابع اول را به صورت حلقه ای

فراخوانی می کند و در نهایت میانگین نهایی را برمی گرداند.

#### گام دوم: پیاده سازی منطق AP

در تابع محاسبه **AP**، باید روی لیست اسنادِ بازیابی شده پیمایش کنیم. هر بار که یک سندِ مرتبط یافت شد، تعداد کل اسناد مرتبطِ یافت شده (**Hits**) را 1 واحد افزایش داده و دقت (**Precision**) را در همان نقطه محاسبه می کنیم. مجموع این دقت ها در نهایت بر تعداد کل اسناد مرتبط موجود در حقیقت پایه (**Ground Truth**) تقسیم می شود.

#### گام سوم: مدیریت حالت های مرزی (Edge Cases)

اسکرپت تصحیح خودکار نباید با خطای زمان اجرا (**Runtime Error**) متوقف شود. بنابراین، باید شرایطی نظیر مواردی که دانشجو هیچ سندی را بازیابی نکرده، یا پرسمانی که هیچ سند مرتبطی در حقیقت پایه ندارد (جلوگیری از خطای تقسیم بر صفر) به دقت مدیریت شوند.

### بخش 4) پیاده سازی کد

```
mean_average_precision.py

def calculate_average_precision(retrieved_docs, ground_truth_set):
    """
    Calculates Average Precision (AP) for a single query.

    retrieved_docs: List of document IDs retrieved by the student's system.
    ground_truth_set: Set of true relevant document IDs.
    """
    if not ground_truth_set:
        return 0.0

    score = 0.0
    num_hits = 0.0

    for i, doc_id in enumerate(retrieved_docs):
        if doc_id in ground_truth_set:
            num_hits += 1.0
            precision_at_i = num_hits / (i + 1.0)
            score += precision_at_i

    return score / len(ground_truth_set)

def calculate_map(student_results, ground_truth):
    """
    Calculates Mean Average Precision (MAP) across all queries.

    student_results: Dictionary mapping query_id to a list of retrieved doc_ids.
    ground_truth: Dictionary mapping query_id to a set of relevant doc_ids.
    """
    if not ground_truth:
        return 0.0

    total_ap = 0.0
    num_queries = len(ground_truth)

    for query_id, true_relevant in ground_truth.items():
        retrieved = student_results.get(query_id, [])
        ap_score = calculate_average_precision(retrieved, true_relevant)
        total_ap += ap_score

    map_score = total_ap / num_queries
    return map_score
```

در پردازش‌های کلان‌داده، ساختار داده‌ای (**Data Structure**) که برای ذخیره مقادیر انتخاب می‌کنید تأثیر مستقیمی بر کارایی الگوریتم دارد. در کد بالا، متغیر حقیقت پایه (**ground\_truth\_set**) باید الزاماً از نوع **set** در پایتون تعریف شود، نه **list**. استفاده از **set** باعث می‌شود عملیات جستجو (**if doc\_id in ground\_truth\_set**) از پیچیدگی زمانی **O(n)** به **O(1)** کاهش یابد. این ظرافت‌های برنامه‌نویسی در ارزیابی سیستم‌های واقعی موتورهای جستجو که حاوی میلیون‌ها سند هستند، تفاوت بین یک سیستم کارآمد و یک سیستم کند را رقم می‌زنند.

## پاسخنامه مسئله 8.23: بهینه‌سازی سیستم مستقر بر اساس رفتار

### ★ (Refining a deployed system based on behavior)

#### بخش 1) تحلیل اولیه مسئله

در این سناریو، ما با 2 سیگنال رفتاری کاملاً متضاد از سوی کاربر روبرو هستیم: از یک سو، رفتن به صفحات دوم و سوم نشان‌دهنده «نیاز اطلاعاتی شدید» (**High Information Need**) و تلاش کاربر برای یافتن پاسخ است؛ او به سادگی تسلیم نشده است. از سوی دیگر، عدم کلیک و ترک جستجو (**Abandonment**) نشان‌دهنده «شکست سیستم در ارائه محتوای متقاعدکننده» است. پرسمان‌های طولانی (**Long-tail queries**) به دلیل خاص بودن، طولانی بودن و فراوانی پایین در لاگ‌های جستجو، معمولاً پاشنه آشیل موتورهای جستجوی کلاسیک هستند.

#### بخش 2) مفاهیم کلیدی

**پرسمان‌های طولانی (Long-tail Queries):** پرسمان‌هایی بسیار خاص و معمولاً چندکلمه‌ای هستند که حجم جستجوی پایینی دارند، اما نیت کاربر در آن‌ها بسیار شفاف و دقیق است.

**پیمایش عمیق (Deep Pagination):** حرکت کاربر به صفحات بعدی نتایج که در علم بازیابی اطلاعات نشان‌دهنده پایین بودن **Precision@K** (دقت در نتایج اولیه) است.

**ترک بدون کلیک (Abandonment):** خروج کاربر از صفحه نتایج (**SERP**) بدون هیچ‌گونه کلیک روی پیوندها، که معمولاً ناشی از عدم تطابق ظاهری نتایج با نیاز کاربر است.

#### بخش 3) حل گام‌به‌گام

برای یافتن ریشه این مشکل، باید معماری سیستم جستجو را از پایین به بالا (فاز به فاز) عارضه‌یابی کنیم:

#### گام اول: بررسی بخش نمایه‌سازی (Indexing)

آیا مشکل از نمایه‌سازی است؟ به احتمال بسیار زیاد خیر. اگر سیستم هیچ نتیجه‌ای برای نمایش نداشت یا پایگاه داده کاملاً خالی از اطلاعات مرتبط بود، جستجوگر در همان صفحه اول با خطای «نتیجه‌ای یافت نشد» مواجه می‌شد. تولید چندین صفحه نتیجه نشان می‌دهد که سیستم نمایه‌سازی به درستی کار کرده و اسنادی را (هرچند با تطابق جزئی) واکنش داده است.

#### گام دوم: ردیابی مشکل در بخش امتیازدهی و رتبه‌بندی (Scoring)

این بخش مظنون و مقصر اصلی در این سناریوی رفتاری است. در پرسمان‌های طولانی، الگوریتم‌های رتبه‌بندی سنتی مبتنی بر کلمه (مانند **TF-IDF** یا **BM25**) معمولاً دچار خطای محاسباتی می‌شوند:

**چالش تطابق جزئی (Partial Matching):** فرض کنید پرسمان 6 کلمه دارد. سیستم ممکن است سندی را که فقط 2

کلمه از این پرسمان را دارد اما آن 2 کلمه را بارها تکرار کرده است، در رتبه بالاتری نسبت به سندی قرار دهد که هر 6 کلمه را دارد اما طول سند بسیار بلند است.

**نتیجه رفتاری:** سیستم اسنادی را در رتبه‌های بالا (صفحه اول) قرار می‌دهد که از نظر آماری با پرسمان اشتراک دارند، اما از نظر معنایی (**Semantic**) کاملاً نامرتب هستند. کاربر با دیدن نتایج نامرتب، به امید یافتن پاسخ بهتر به صفحات بعد می‌رود، اما چون کل لیست بر اساس همان منطق اشتباه مرتب شده است، در نهایت ناامید شده و صفحه را ترک می‌کند.

### گام سوم: ردیابی مشکل در بخش تولید خلاصه نتایج (Snippet Generation)

این بخش می‌تواند عامل تشدیدکننده و ثانویه باشد. در پرسمان‌های طولانی، کلمات کلیدی معمولاً در متن سند با فاصله زیادی از هم پراکنده هستند (پایین بودن **Term Proximity**).

**چالش پنجره متنی:** الگوریتم تولید اسنیپت پویا در پیدا کردن یک «پنجره متنی منسجم» که تمامی کلمات این پرسمان طولانی را پوشش دهد، ناکام می‌ماند.

**نتیجه رفتاری:** سیستم ممکن است سند کاملاً مرتبطی را در صفحه اول یا دوم آورده باشد، اما به دلیل ضعف در **Snippet Generation**، یک خلاصه مبهم، بریده‌بریده و نامرتب به کاربر نشان می‌دهد. کاربر با خواندن این خلاصه ضعیف، متوجه ارتباط سند نمی‌شود و از کلیک کردن روی آن صرف‌نظر می‌کند.

### بخش 4 پاسخ نهایی

رفتار «پیمایش عمیق و سپس ترک بدون کلیک» در پرسمان‌های طولانی، ریشه در 2 بخش از معماری سیستم دارد:

- بخش Scoring (عامل اصلی):** افت شدید الگوریتم رتبه‌بندی در محاسبه ارتباط معنایی پرسمان‌های چندکلمه‌ای. سیستم نتوانسته است اسناد واقعاً مرتبط را در رتبه‌های بالا قرار دهد و اسناد دارای «تطابق جزئی» را به اشتباه در صدر لیست نمایش داده است.
- بخش Snippet Generation (عامل ثانویه):** ناتوانی سیستم در استخراج یک بستر معنایی (**Context**) مناسب برای نمایش کلمات متعدد پرسمان. این امر باعث افت «مطلوبیت ظاهری» اسناد شده و کاربر را از کلیک روی اسنادی که شاید مرتبط بوده‌اند، منصرف کرده است.

#### نکته آموزشی:

برای حل مشکل الگوریتم‌های سنتی **Scoring** در مواجهه با پرسمان‌های طولانی (**Long-tail**)، در معماری موتورهای جستجوی مدرن از روشی به نام جستجوی معنایی یا متراکم (**Dense Retrieval / Semantic Search**) استفاده می‌شود. در این روش، به جای تکیه بر تطابق دقیق کلمات کلیدی (**Lexical Search**)، از شبکه‌های عصبی عمیق (مانند مدل‌های مبتنی بر **BERT**) برای تبدیل کل جمله کاربر به یک «بردار ریاضی» (**Vector Embedding**) استفاده می‌گردد. در این فضای برداری، حتی اگر سند و پرسمان کلمات مشترک زیادی نداشته باشند، سیستم می‌تواند «نیت» (**Intent**) نهفته در پرسمان طولانی را درک کرده و اسنادی با مفهوم مشابه را در رتبه‌های برتر قرار دهد. این تکنیک، نرخ **Abandonment** را در جستجوهای پیچیده به شدت کاهش می‌دهد.

## پاسخنامه مسئله 8.24: چالش نتایج رتبه‌بندی با اطلاعات ناقص

### ★ (Ranked retrieval evaluation with pooling)

#### بخش 1) تحلیل اولیه مسئله

در این مسئله، با چالش مقیاس‌پذیری (**Scalability**) در ارزیابی سیستم‌های بازیابی اطلاعات مواجه هستیم. در محیط وب یا دادگان‌های عظیمی مانند **TREC**، بررسی تمامی اسناد برای یافتن حقیقت پایه (**Ground Truth**) از توان انسانی خارج است. این مسئله به دنبال معرفی تکنیک «تجمیع» (**Pooling**) به عنوان راهکار این مشکل، و سپس اثبات این موضوع است که چرا خطای ذاتی این روش (یعنی از قلم افتادن برخی اسناد مرتبط)، به اعتبار مقایسه و رتبه‌بندی الگوریتم‌های مختلف آسیبی وارد نمی‌کند.

#### بخش 2) مفاهیم کلیدی

**تجمیع (Pooling):** روشی که در آن به جای کل پایگاه داده، تنها یک زیرمجموعه کوچک از اسناد برتر بازیابی‌شده توسط سیستم‌های مختلف، برای قضاوت به انسان سپرده می‌شود.

**فرض اسناد قضاوت‌نشده (Unjudged Documents Assumption):** در استاندارد **TREC**، هر سندی که در استخراج تجمیع (**Pool**) قرار نگیرد و توسط داوران دیده نشود، به صورت پیش‌فرض «نامرتب» (**Non-relevant**) در نظر گرفته می‌شود.

**پایداری نسبی (Relative Stability):** مفهومی آماری که نشان می‌دهد حتی اگر نمره مطلق (**Absolute Score**) یک معیار مانند **MAP** دارای خطا باشد، فاصله و برتری نسبی سیستم‌ها نسبت به یکدیگر حفظ می‌شود.

#### بخش 3) حل گام‌به‌گام

##### گام اول: مکانیزم عملکرد تکنیک Pooling

هنگامی که پروژه‌های بازیابی ده‌ها الگوریتم مختلف (یا مثلاً خروجی سیستم‌های **100** دانشجو در یک محیط آکادمیک) به صورت همزمان ارزیابی می‌شود، ساختار **Pooling** به این شکل عمل می‌کند:

1. برای یک پرسمان مشخص، سیستم هر یک از شرکت‌کنندگان، لیست رتبه‌بندی شده خود را تولید می‌کند.

2. یک نقطه برش (Cut-off) مثلاً **k = 100** در نظر گرفته می‌شود.

3. تنها **100** سند اول از خروجی هر سیستم استخراج شده و اجتماع (**Union**) آن‌ها محاسبه می‌شود. اگر **n** سیستم داشته باشیم، استخراج اسناد (**P**) به این شکل به دست می‌آید:

$$P = \bigcup_{j=1}^n \text{Top}_k(\text{System}_j)$$

4. داوران انسانی فقط اسناد موجود در مجموعه **P** را ارزیابی می‌کنند.

##### گام دوم: تحلیل مسئله اسناد جا مانده

اگر سند مرتبطی در پایگاه داده وجود داشته باشد که هیچ‌یک از سیستم‌ها نتوانسته باشند آن را در **100** رتبه اول خود قرار دهند، این سند وارد مجموعه **P** نشده و در نتیجه قضاوت نمی‌شود. طبق قانون **TREC**، این سند «نامرتب» فرض خواهد شد. در

نتیجه، تعداد اسناد مرتبط یافت‌شده در استخر ( $|R_{pool}|$ ) همواره کوچکتر یا مساوی تعداد واقعی اسناد مرتبط ( $|R_{true}|$ ) است. این امر باعث می‌شود فراخوانی مطلق (**Absolute Recall**) سیستم‌ها افت کند.

### گام سوم: اثبات ریاضی/منطقی عدم تأثیر مخرب بر مقایسه نسبی (MAP)

با وجود این خطا، مقایسه نسبی 2 موتور جستجوی **A** و **B** معتبر باقی می‌ماند. این استدلال بر 2 پایه استوار است:

#### 1. تأثیر در مخرج کسر AP (یکسان برای همه): در محاسبه متوسط دقت (AP) برای یک پرسمان، فرمول بر تعداد کل

اسناد مرتبط تقسیم می‌شود. در روش **Pooling**، ما به جای  $|R_{true}|$  از  $|R_{pool}|$  استفاده می‌کنیم:

$$AP \approx \frac{1}{|R_{pool}|} \sum_{k=1}^n P@k \cdot rel(k)$$

از آنجا که  $|R_{pool}|$  برای یک پرسمان خاص، یک مقدار ثابت برای تمامی سیستم‌ها است، نمره **AP** هر 2 سیستم **A** و **B** دقیقاً در یک ضریب ثابت یکسان ضرب/تقسیم می‌شود.

#### 2. تأثیر در صورت کسر AP (عدم کسب امتیاز توسط هیچ‌کدام): اگر سند مرتبط **d** از استخر جا مانده باشد، معنایش

این است که نه سیستم **A** و نه سیستم **B** نتوانسته‌اند آن را در رتبه‌های برتر (مثلاً **Top 100**) نمایش دهند.

### بخش 4) پاسخ نهایی

تکنیک **Pooling** با استخراج اجتماع **k** سند برتر از تمامی سیستم‌های مورد ارزیابی و ساخت یک «استخر» محدود از اسناد، نیاز به بررسی کل پایگاه داده را حذف کرده و قضاوت انسانی را امکان‌پذیر می‌سازد.

جا ماندن برخی اسناد مرتبط از این استخر، تأثیر مخربی بر مقایسه نسبی سیستم‌های **A** و **B** در معیار **MAP** ندارد؛ زیرا اولاً، تعداد اسناد مرتبط شناخته‌شده ( $|R_{pool}|$ ) به عنوان مخرج مشترک کسر **AP** برای هر 2 سیستم یکسان است و ثانیاً، سندی که جا مانده، توسط هیچ‌یک از 2 سیستم در رتبه‌های بالا بازایی نشده است، در نتیجه هیچ‌کدام از آن‌ها از این سند پنهان امتیازی دشت نمی‌کنند تا باعث برتری ناعادلانه یکی بر دیگری شود. پژوهش‌های معتبر (مانند **Zobel 1998**) نشان داده‌اند که رتبه‌بندی سیستم‌ها با این روش، به شدت پایدار است.

#### نکته:

در طراحی یک پایگاه آزمون استاندارد، برای اینکه فرآیند **Pooling** عادلانه باشد، «تنوع» سیستم‌های شرکت‌کننده بسیار حیاتی است. اگر استخر اسناد فقط توسط یک خانواده از الگوریتم‌ها (مثلاً فقط بر اساس مدل‌های برداری مانند **TF-IDF**) ساخته شود، اسناد مرتبطی که نیاز به درک معنایی (**Semantic**) دارند ممکن است هرگز وارد استخر نشوند. بنابراین، در مسابقات **TREC**، عمداً از ترکیبی از موتورهای جستجوی بولین، احتمالی، برداری و عصبی استفاده می‌شود تا استخر نهایی، پوشش بسیار همه‌جانبه‌ای از انواع مختلف اسناد مرتبط داشته باشد و سیستم شما (چه در مقیاس دانشگاهی و چه جهانی) به دقیق‌ترین شکل ممکن نمره‌دهی شود.

## پاسخنامه مسئله 8.25: تحلیل خطای اسنیپت‌ها در موتور جستجو (Snippet error analysis) ★

### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما با مشکل «خلاصه‌های نامرتب یا فاقد بستر» (**Context-less Snippets**) مواجه هستیم. هنگامی که یک سند طولانی (مانند یک مقاله 50 صفحه‌ای) فقط به دلیل تکرار پراکنده کلمات کلیدی بازایی می‌شود، اگر سیستم نتواند

متراکم‌ترین بخش سند را که کلمات جستجو شده در آن تجمع دارند پیدا کند، خلاصه‌ای نامرتب تولید می‌کند. نتیجه این امر، کاهش شدید نرخ کلیک (**CTR**) است؛ زیرا کاربر قادر به درک ارتباط سند با پرسمان خود از روی اسنیپت نخواهد بود. برای حل این مشکل، باید الگوریتمی طراحی کنیم که متن را اسکن کرده و زیررشته‌ای (**Substring**) را بیابد که بیشترین کلمات کلیدی را در کوتاه‌ترین فاصله ممکن در خود جای داده است.

## بخش 2) مفاهیم کلیدی

**نمایه موقعیتی (Positional Index):** ساختار داده‌ای که نه تنها حضور یک کلمه در سند، بلکه جایگاه (**Index**) دقیق آن را در متن ذخیره می‌کند تا فاصله بین کلمات قابل محاسبه باشد.

**پنجره لغزان (Sliding Window):** تکنیکی الگوریتمی که در آن یک قاب با اندازه مشخص (یا متغیر) روی متن حرکت می‌کند تا ویژگی‌های مجموعه‌ای از کلمات مجاور را به صورت محلی ارزیابی کند.

**تراکم کلمات (Term Density):** معیاری برای امتیازدهی به یک پنجره متنی، که معمولاً با نسبت «تعداد کلمات کلیدی یافت‌شده» به «طول کل پنجره» محاسبه می‌شود.

## بخش 3) حل گام به گام

### گام اول: موقعیتیابی کلمات پرسمان در متن

ابتدا باید مشخص کنیم کلمات پرسمان کاربر دقیقاً در کجای سند قرار دارند. فرض کنید پرسمان کاربر شامل کلمات زیر است:

$$Q = \{q_1, q_2, \dots, q_k\}$$

سیستم متن مقاله را پیمایش کرده و لیستی از موقعیت‌ها (**Index**) که یکی از کلمات **Q** در آن ظاهر شده است را استخراج می‌کند. مثلاً اگر پرسمان «طراحی الگوریتم وب» باشد، موقعیت حضور هر یک از این 3 کلمه در یک آرایه ذخیره می‌شود.

### گام دوم: تعریف تابع امتیازدهی (Scoring Function)

برای اینکه الگوریتم بتواند بهترین پنجره را انتخاب کند، نیازمند یک فرمول ریاضی برای ارزیابی هر پنجره (**W**) هستیم. یک فرمول استاندارد و کارآمد به شکل زیر است:

$$Score = \frac{|U|^2}{|L|}$$

که در آن:

**|U|:** تعداد کلمات کلیدی منحصربه‌فرد موجود در آن پنجره متنی. (به توان 2 می‌رسد تا به پنجره‌هایی که تنوع بیشتری از کلمات پرسمان را دارند، وزن تصاعدی و بسیار بالاتری داده شود).

**|L|:** طول پنجره متنی (تعداد کل کلمات از اولین کلمه کلیدی تا آخرین کلمه کلیدی در آن پنجره). قرار گرفتن در مخرج باعث می‌شود پنجره‌های کوتاه‌تر و متراکم‌تر امتیاز بیشتری بگیرند.

### گام سوم: پیاده‌سازی الگوریتم پنجره لغزان

در این مرحله، تمام ترکیبات ممکن از پنجره‌هایی که با یک کلمه کلیدی شروع و با یک کلمه کلیدی دیگر ختم می‌شوند را تا سقف

یک طول مجاز (مثلاً حداکثر **30** کلمه برای جا شدن در اسنیپت صفحه نتایج) بررسی می‌کنیم.

```
dynamic_snippet.py

def generate_dynamic_snippet(doc_tokens, query_terms, max_window_size=30):
    best_score = -1.0
    best_window = (0, 0)

    # Extract positions of query terms in the document
    match_positions = [i for i, token in enumerate(doc_tokens) if token in query_terms]

    if not match_positions:
        return ""

    # Sliding window logic over matched positions
    for i in range(len(match_positions)):
        for j in range(i, len(match_positions)):
            start_idx = match_positions[i]
            end_idx = match_positions[j]
            window_length = end_idx - start_idx + 1
            # Stop expanding window if it exceeds maximum snippet length
            if window_length > max_window_size:
                break
            # Extract window tokens and count unique query terms
            window_tokens = doc_tokens[start_idx:end_idx+1]
            unique_terms_matched = len(set(window_tokens).intersection(query_terms))
            # Density calculation: (Unique Matches ^ 2) / Total Window Length
            current_score = (unique_terms_matched ** 2) / float(window_length)
            # Update best window
            if current_score > best_score:
                best_score = current_score
                best_window = (start_idx, end_idx)
    # Format and return the final snippet string
    snippet_tokens = doc_tokens[best_window[0]:best_window[1]+1]
    return " ".join(snippet_tokens)
```

#### گام چهارم: استخراج و قالب‌بندی (Formatting)

پس از اجرای الگوریتم و یافتن **best\_window** (نقطه‌شروع و نقطه‌پایان در متن)، سیستم آن بخش از متن را جدا کرده، در صورت نیاز از **2** طرف به آن علامت سه‌نقطه (...) اضافه می‌کند و کلمات کلیدی پرسمان را با تگ‌های **HTML** مانند **<b>** ضخیم می‌کند تا چشم کاربر به سرعت آن‌ها را تشخیص دهد.

#### بخش 4) پاسخ نهایی

برای تولید اسنیپت‌های پویا و جلوگیری از ریزش کلیک در اسناد طولانی، الگوریتم «پنجره لغزان مبتنی بر تراکم» پیشنهاد می‌شود. این الگوریتم به این صورت عمل می‌کند:

**1.** مکان تمامی کلمات کلیدی پرسمان را در سند پیدا می‌کند.

**2.** با استفاده از یک پنجره متنی متغیر (تا سقف مشخصی از کلمات، مثلاً **30** کلمه)، فواصل مختلف بین این کلمات را اسکن می‌کند.

**3.** به هر پنجره بر اساس فرمول **Score = U<sup>2</sup> / L** (تعداد کلمات کلیدی منحصربه‌فرد به توان **2**، تقسیم بر طول پنجره) امتیاز می‌دهد.

**4.** پنجره‌ای که بالاترین امتیاز را کسب کند، متراکم‌ترین و مرتبط‌ترین بستر معنایی را دارد و به عنوان اسنیپت نهایی به کاربر نمایش داده می‌شود.

#### نکته:

از نظر پیچیدگی زمانی (**Time Complexity**)، اسکن کردن کل سند در زمان پرسش (**Query Time**) برای میلیون‌ها نتیجه غیرعملی است. در موتورهای جستجوی واقعی در مقیاس وب، گام اول الگوریتم بالا (موقعیت‌یابی کلمات) هرگز در زمان جستجو انجام نمی‌شود، بلکه داده‌ها از پیش در فاز نمایه‌سازی داخل نمایه معکوس موقعیتی (**Positional Inverted Index**) ذخیره شده‌اند.

## پاسخنامه مسئله 8.26: مطلوبیت کاربر و مدل‌های کلیک (User utility and Click models) ★

### بخش 1) تحلیل اولیه مسئله

در این مسئله، وارد مبحث پیشرفته‌ای به نام «مدل‌سازی رفتار کاربر» (**User Behavior Modeling**) با استفاده از داده‌های کلیک می‌شویم. هدف این است که درک کنیم کاربران چگونه لیست نتایج موتور جستجو (**SERP**) را اسکن می‌کنند و تصمیم به کلیک می‌گیرند. شناخت این فرضیات رفتاری برای طراحی معیارهای ارزیابی متناسب با سیستم‌های خاص (مانند سیستم‌های جستجوی کدهای برنامه‌نویسی) بسیار حیاتی است.

### بخش 2) مفاهیم کلیدی

**مدل‌های کلیک (Click Models):** چارچوب‌های ریاضی و احتمالی هستند که تلاش می‌کنند رفتار کاربر در برخورد با لیست نتایج را شبیه‌سازی کنند تا بتوانند از روی داده‌های خام کلیک، میزان ارتباط واقعی (**Relevance**) اسناد را استخراج کنند.

**فرض پیمایش (Traversal Assumption):** قانونی در مدل‌های کلیک که مشخص می‌کند کاربر با چه ترتیبی و چگونه به اسناد نگاه می‌کند.

### بخش 3) حل گام‌به‌گام

#### گام اول: تحلیل مدل آبشاری (Cascade Model)

این مدل یکی از کلاسیک‌ترین و پرکاربردترین مدل‌های رفتار کاربر است. فرضیات بنیادین آن به شرح زیر است:

- پیمایش خطی:** کاربر نتایج را دقیقاً و به صورت متوالی از بالا به پایین (از رتبه 1 به سمت پایین) اسکن می‌کند.
- تصمیم‌گیری مرحله‌ای:** کاربر در هر رتبه، خلاصه (اسنیپت) را می‌خواند. اگر آن را مرتبط یافت، روی آن کلیک می‌کند و جستجو در همان نقطه پایان می‌یابد (احتمال پیمایش به رتبه بعدی صفر می‌شود).
- تداوم در صورت عدم رضایت:** اگر سند فعلی نامرتبب تشخیص داده شود، کاربر با احتمال مشخصی به سند رتبه بعدی (پایین‌تر) می‌رود. این مدل فرض می‌کند که جستجوگر به دنبال اطلاعاتی است که در اولین مواجهه با یک نتیجه خوب، نیاز او برطرف می‌شود.

#### گام دوم: تحلیل مدل ناوبری (Navigational Model)

این مدل برای پرسرمان‌هایی طراحی شده است که در آن‌ها کاربر دقیقاً می‌داند به دنبال چه چیزی (چه موجودیت یا وب‌سایتی) است (مانند جستجوی کلمه "**Facebook**" یا "سامانه گلستان"). فرضیات آن عبارتند از:

- هدف قطعی:** کاربر به دنبال یک و فقط یک نتیجه خاص است (یک سوزن در انبار کاه).
- اسکن سریع و هدفمند:** کاربر ممکن است به صورت خطی نتایج را اسکن نکند، بلکه با چشم به دنبال یک کلیدواژه یا آدرس اینترنتی (**URL**) آشنا در میان نتایج بگردد.

**3. نادیده گرفتن سایر نتایج:** تمام نتایجِ دیگر، هر چقدر هم که از نظر متنی مرتبط به نظر برسند، برای کاربر ارزش صفر

دارند. کاربر به محض یافتن تارنمای هدف روی آن کلیک کرده و می‌رود.

### گام سوم: تطبیق با سیستم جستجوی کدهای برنامه‌نویسی

حال باید بررسی کنیم که رفتار یک برنامه‌نویس در جستجوی کد، به کدام مدل نزدیک‌تر است.

جستجوی کد (مانند جستجوی خطاهای کامپایلر یا نحوه پیاده‌سازی یک الگوریتم) ماهیتی اطلاعاتی و مسئله‌محور (**Informational / Problem-solving**) دارد. برنامه‌نویس معمولاً یک آدرس اینترنتی خاص را در ذهن ندارد (مدل ناوبری منتفی است). در عوض، او به دنبال یک راه‌حل است. برنامه‌نویس نتیجه اول را می‌بیند، اسنپیت کد را بررسی می‌کند؛ اگر راهگشا بود، آن را کپی کرده و جستجو پایان می‌یابد. اگر کد مشکل داشت یا با فریم‌ورک او سازگار نبود، به سراغ نتیجه دوم می‌رود. این رفتار بررسی متوالی و توقف پس از یافتن پاسخ، دقیقاً منطبق بر مدل آبشاری است.

### بخش 4) پاسخ نهایی

**تفاوت فرضیات:** مدل آبشاری فرض می‌کند که کاربر نتایج را به صورت متوالی و خطی (از بالا به پایین) اسکن می‌کند و

به محض یافتن اولین سند مرتبط و کلیک روی آن، جستجو را متوقف می‌سازد. اما مدل ناوبری فرض می‌کند کاربر به دنبال یک مقصد از پیش تعیین‌شده و قطعی است؛ او نتایج را صرفاً برای یافتن همان تک‌مورد آشنا اسکن کرده و سایر نتایج (صرف نظر از کیفیت محتوایی آن‌ها) را کاملاً نادیده می‌گیرد.

**کاربرد در جستجوی کد:** برای یک سیستم جستجوی کدهای برنامه‌نویسی، مدل آبشاری (**Cascade Model**) بسیار به

واقعیت نزدیک‌تر است. جستجوی کد ماهیتی اطلاعاتی دارد و کاربر معمولاً نتایج (راه‌حل‌ها) را به ترتیب رتبه اسکن

می‌کند تا به اولین قطعه کدی که مشکل او را حل می‌کند برسد.

#### نکته آموزشی:

اگرچه مدل آبشاری برای جستجوی کد بسیار مناسب‌تر از مدل ناوبری است، اما در واقعیت، رفتار برنامه‌نویسان پیچیده‌تر از یک مدل آبشاری ساده است. برنامه‌نویسان اغلب یک کد را از نتیجه اول باز کرده، آن را در محیط توسعه (**IDE**) خود تست می‌کنند و در صورت دریافت خطا (**Compile Error**)، دکمه **Back** را زده و به لیست نتایج برمی‌گردند تا نتیجه دوم را امتحان کنند. مدل کلاسیک آبشاری، فرض را بر این می‌گذارد که «یک کلیک مساوی با پایان جستجو است»، در حالی که بازگشت کاربر این فرض را نقض می‌کند. برای ارزیابی بسیار دقیقِ چنین سیستم‌هایی، در علم اطلاعات از مدل‌های پیشرفته‌تری به نام مدل شبکه‌های بیزی پویا (**Dynamic Bayesian Network (DBN Click Model)**) استفاده می‌شود که به صراحت پدیده «کلیک، عدم رضایت پس از کلیک، و بازگشت برای پیمایش مجدد» را مدل‌سازی می‌کنند.

## پاسخنامه مسئله 8.27: سناریوی تغییر معماری بر اساس لاگ‌ها

### ★ (Architecture shift based on logs)

### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما با محدودیتی جدی در پارادایم کلاسیک ارزیابی بازیابی اطلاعات (**Ad-hoc Retrieval Evaluation**) مواجه هستیم. سیستم‌های سنتی فرض می‌کنند نیاز اطلاعاتی کاربر ایستا است و در یک پرسمان واحد به طور کامل بیان می‌شود. اما وجود نرخ **50 درصدی** «بازنویسی پرسمان» (**Query Reformulation**) نشان می‌دهد که کاربران در حال طی کردن یک فرآیند

اکتشافی و چندمرحله‌ای هستند. خواسته مسئله این است که نقص این نگاه تک‌مرحله‌ای را تشریح کرده و چارچوب ارزیابی را به سمت «ارزیابی مبتنی بر نشست» (**Session-based Evaluation**) تغییر دهیم.

## بخش 2) مفاهیم کلیدی

**ارزیابی تک‌مرحله‌ای (Single-turn Evaluation):** بررسی موفقیت موتور جستجو فقط بر اساس خروجی همان یک پرسمان، بدون توجه به تاریخچه یا قدم‌های بعدی کاربر.

**نشست جستجو (Search Session):** توالی پیوسته‌ای از پرسمان‌ها و کلیک‌های یک کاربر که همگی با هدف برطرف کردن یک نیاز اطلاعاتی واحد انجام می‌شوند.

**هزینه تعامل (Interaction Cost):** تلاش ذهنی و زمانی که کاربر برای تغییر کلمات کلیدی، اسکن مجدد نتایج و کلیک کردن صرف می‌کند.

## بخش 3) حل گام‌به‌گام

**گام اول: تشریح نقص در سیستم ارزیابی فعلی (مبتنی بر پرسمان مستقل)**

هنگامی که ارزیابی شما فقط بر اساس پرسمان‌های تک‌مرحله‌ای باشد (مثلاً محاسبه **MAP** یا **NDCG** برای هر پرسمان به صورت مجزا)، سیستم ارزیابی نسبت به «سفر کاربر» (**User Journey**) کور خواهد بود. این رویکرد **2** نقص عمده ایجاد می‌کند:

**1. جرمه ناعادلانه سیستم:** فرض کنید کاربر پرسمان مبهم **Q<sub>1</sub>** را وارد می‌کند، نتایج را می‌بیند، متوجه می‌شود کلمات مناسبی انتخاب نکرده است و پرسمان دقیق‌تر **Q<sub>2</sub>** را وارد کرده و به جواب می‌رسد. در ارزیابی تک‌مرحله‌ای، **Q<sub>1</sub>** یک «شکست مطلق» محسوب شده و میانگین عملکرد سیستم شما را به شدت کاهش می‌دهد، در حالی که در واقعیت، **Q<sub>1</sub>** یک گام اکتشافی و ضروری برای رسیدن به **Q<sub>2</sub>** بوده و کاربر در نهایت با رضایت سیستم را ترک کرده است.

**2. نادیده گرفتن زحمت کاربر (User Effort):** اگر کاربری در پرسمان اول (**Q<sub>1</sub>**) به جواب برسد، یا پس از **5** بار تغییر کلمات کلیدی در پرسمان پنجم (**Q<sub>5</sub>**) به همان جواب برسد، ارزیابی تک‌مرحله‌ای هر **2** را یکسان (یک موفقیت) می‌بیند. در صورتی که در حالت دوم، کاربر هزینه تعاملی بالایی پرداخته و مطلوبیت بسیار کمتری را تجربه کرده است.

**گام دوم: پیاده‌سازی ارزیابی مبتنی بر نشست (Session-based Evaluation)**

برای حل این نقص، باید چارچوب ارزیابی را از سطح «پرسمان» به سطح «نشست» ارتقا دهیم. این کار در **2** مرحله انجام می‌شود:

**الف) گروه‌بندی لاگ‌ها (Session Boundary Detection):** ابتدا باید لاگ‌های جستجو را تحلیل کنیم و پرسمان‌های متوالی یک کاربر را به هم متصل کنیم. این اتصال معمولاً بر اساس آستانه زمانی (مثلاً پرسمان‌هایی که با فاصله کمتر از **30** دقیقه از هم زده شده‌اند) و شباهت معنایی (اشتراک کلمات کلیدی بین پرسمان‌ها) انجام می‌شود تا یک نشست جستجو (**Session**) تشکیل گردد.

**ب) استفاده از معیارهای تجمعی با اعمال جرمه زمانی (Discounting):** به جای محاسبه معیار برای یک پرسمان، معیار کل نشست را محاسبه می‌کنیم. معروف‌ترین معیار در این زمینه، **sDCG (Session-based DCG)** است. منطق **sDCG** این است: سندی که در پرسمان اول یافت می‌شود، ارزش کامل دارد. اما اگر سند مرتبطی در پرسمان دوم (پس از یک **Reformulation**) یافت شود، ارزش آن به دلیل زحمتی که کاربر برای تغییر پرسمان کشیده است، شامل یک ضریب تخفیف (**Discount Factor**) می‌شود. هرچه کاربر پرسمان را بیشتر بازنویسی کند، ارزش اسناد یافت‌شده در پرسمان‌های بعدی، کمتر و کمتر محاسبه می‌گردد.

**نقص ارزیابی فعلی:** ارزیابی مبتنی بر پرسمان مستقل، «فرآیند پویای جستجو» را نادیده می‌گیرد. این روش نمی‌تواند تفاوت بین یک شکست واقعی و یک جستجوی اکتشافی را درک کند و همچنین هزینه و زحمت کاربر برای تغییر مداوم کلمات کلیدی (**Interaction Cost**) را در محاسبات لحاظ نمی‌کند.

#### نحوه پیاده‌سازی ارزیابی Session-based:

1. پرسمان‌های متوالی مرتبط با یک موضوع و متعلق به یک کاربر، در قالب یک «نشست» (**Session**) گروه‌بندی می‌شوند.
2. به جای ارزیابی نقطه‌ای، کل فرآیند از پرسمان اول تا آخرین کلیک ارزیابی می‌شود.
3. از معیارهایی مانند **sDCG** استفاده می‌شود که در آن، ارزش اطلاعات به‌دست‌آمده بر اساس «تعداد مراحل و بازنویسی‌های طی‌شده» جریمه و تخفیف داده می‌شود تا هرگونه تأخیر در رسیدن به جواب، در نمره نهایی سیستم بازتاب یابد.

#### نکته:

در معماری موتورهای جستجوی مدرن (مانند معماری‌های مبتنی بر **Transformer**)، مفهوم **Session** فراتر از بحث صرفاً «ارزیابی» رفته و وارد فاز «رتبه‌بندی» (**Ranking**) شده است. به این معنا که الگوریتم، پرسمان‌های قبلی شما در همان نشست را به عنوان «بستر معنایی» (**Context**) به عنوان ورودی دریافت می‌کند. به عنوان مثال، اگر کاربر ابتدا پرسمان "تعمیرات ماشین" و سپس کلمه "جاگوار" را جستجو کند، سیستم به صورت هوشمند متوجه می‌شود که منظور کاربر "خودروی جاگوار" است، نه "حیوان جاگوار" (حتی اگر کلمه خودرو در پرسمان دوم ذکر نشده باشد). این کار نرخ بازنویسی‌های بی‌ثمر را به شدت کاهش می‌دهد.

## پاسخنامه مسئله 8.28: طراحی جامع فریم‌ورک ارزیابی

### ★ (Comprehensive Evaluation Framework Design)

#### بخش 1) تحلیل اولیه مسئله

در این مسئله، ما در جایگاه معماری کلان یک موتور جستجوی مقیاس‌بالا (نظیر یک موتور جستجوی ملی) قرار داریم. در چنین مقیاسی، اتکا به تنها یک روش ارزیابی، منجر به کوری سیستماتیک (**Systematic Blindness**) می‌شود. برای تضمین کیفیت، به یک پروتکل 3 لایه نیاز داریم که سیستم را پیش از مواجهه با کاربر (آفلاین)، در حین تعامل با کاربر واقعی (آنلاین)، و از منظر تجربه بصری (کیفی) بسنجد. خواسته مسئله، تعیین دقیق و تحلیلی یک معیار (**Metric**) کلیدی برای هر یک از این 3 فاز است تا به عنوان قطب‌نمای تصمیم‌گیری در پروژه عمل کند.

#### بخش 2) مفاهیم کلیدی

**پروتکل ارزیابی چندلایه:** رویکردی که در آن الگوریتم‌ها ابتدا در محیط آزمایشگاهی و امن ارزیابی می‌شوند، سپس در مقیاس کوچک روی کاربران واقعی آزمایش شده، و در نهایت جزئیات ظاهری آن‌ها توسط انسان‌ها قضاوت می‌شود.

**کلیک رضایت‌بخش (SAT-Click):** کلیک که به دنبال آن، کاربر زمان قابل‌توجهی را در صفحه هدف سپری کند و سریعاً به موتور جستجو برنگردد.

### گام اول: طراحی فاز 1 - ارزیابی آفلاین (Offline Evaluation)

در این فاز، الگوریتم‌ها روی یک مجموعه آزمون استاندارد (تاییدیه‌های انسانی و حقیقت پایه) تست می‌شوند تا پیش از عرضه عمومی، از فاجعه‌بار نبودن نتایج اطمینان حاصل شود.

**متریک کلیدی انتخاب شده: NDCG@10** (بهره تجمعی تخفیف یافته نرمال شده در 10 نتیجه اول)

**دلیل تحلیلی:** در یک موتور جستجوی ملی، اسناد دارای «درجات مختلفی از اعتبار» هستند (مثلاً سایت رسمی دولت

ارزش بسیار بیشتری از یک وبلاگ شخصی دارد). معیار **NDCG** تنها معیاری است که ارتباط طیفی

**(Graded Relevance)** را به خوبی درک می‌کند. همچنین، از آنجا که رفتار کاربران وب به شدت روی صفحه اول متمرکز

است، اعمال تخفیف لگاریتمی روی رتبه‌های پایین‌تر (بخش **Discounted**) و برش روی 10 نتیجه اول (**@10**)،

دقیق‌ترین شبیه‌سازی آفلاین از مطلوبیت نهایی را به ما می‌دهد.

### گام دوم: طراحی فاز 2 - ارزیابی آنلاین (Online A/B Testing)

در این فاز، الگوریتم تایید شده در فاز آفلاین، روی درصدی از ترافیک کاربران واقعی موتور جستجو فعال می‌شود تا رفتار طبیعی آن‌ها سنجیده شود.

**متریک کلیدی انتخاب شده: SAT-Click Rate** (نرخ کلیک رضایت بخش)

**دلیل تحلیلی:** استفاده از معیار ساده نرخ کلیک (**CTR**) در محیط واقعی به شدت گمراه کننده است، زیرا الگوریتم‌هایی

که از اسنیپت‌های فریبنده یا زرد (**Clickbait**) استفاده می‌کنند، می‌توانند **CTR** را به طور کاذب بالا ببرند. معیار

**SAT-Click** تنها کلیک‌هایی را شمارش می‌کند که «زمان ماندگاری» (**Dwell Time**) آن‌ها از یک آستانه مشخص (مثلاً

30 ثانیه) بیشتر باشد و کاربر پدیده پرش (**Pogo-sticking**) را تجربه نکند. این متریک، معتبرترین نماینده برای

«مطلوبیت کاربر» در عمل است.

### گام سوم: طراحی فاز 3 - ارزیابی کیفی اسنیپت‌ها (Qualitative Snippet Evaluation)

پس از تایید الگوریتم‌های رتبه‌بندی، نوبت به ارزیابی رابط کاربری و نحوه نمایش متن‌های خلاصه در صفحه نتایج می‌رسد.

**متریک کلیدی انتخاب شده: Contextual Accuracy Score** (امتیاز دقت بافتی/معنایی توسط ارزیاب انسانی)

**دلیل تحلیلی:** هیچ الگوریتم خودکاری نمی‌تواند «خوانایی» و «عدم ابهام» را درک کند. در این فاز، گروهی از داوران

متخصص زبان‌شناسی و تجربه کاربری (**UX**)، اسنیپت‌ها را در یک مقیاس طیفی (مثلاً از 1 تا 5) بررسی می‌کنند. آن‌ها

می‌سنجند که آیا اسنیپت تولید شده علاوه بر داشتن کلمات کلیدی، بستر و معنای درست مقاله را نیز به کاربر منتقل

می‌کند یا صرفاً جملاتی بی‌ربط و بریده‌بریده است. نمره میانگین داوران، کیفیت الگوریتم استخراج خلاصه

**(Snippet Generation)** را مشخص می‌سازد.

## بخش 4) پاسخ نهایی

چکیده پروتکل جامع ارزیابی موتور جستجوی ملی به شرح زیر طراحی می‌گردد:

1. فاز آفلاین (آزمایشگاهی):

متریک: NDCG@10

**علت:** توانایی محاسبه ارزش جایگاه سند و پشتیبانی از درجات مختلف ارتباط اسناد (**Graded Relevance**)

پیش از استقرار در محیط واقعی.

2. فاز آنلاین (عملیاتی با A/B تست):

**متریک:** SAT-Click Rate

**علت:** فیلتر کردن نویزهای رفتاری و کلیک‌های فریبنده، و اندازه‌گیری موفقیت واقعی کاربر در برطرف کردن نیاز

اطلاعاتی خود (بر اساس زمان ماندگاری).

3. فاز کیفی (ارزیابی واسط کاربری):

**متریک:** Contextual Accuracy Score

**علت:** سنجش میزان خوانایی، پیوستگی جملات و دقت معنایی اسنیپت‌ها، که تنها توسط شناخت شناختی

انسان‌ها قابل ارزیابی است.

#### نکته:

در مهندسی سیستم‌های جستجوی کلان، این 3 فاز دقیقاً به صورت ترتیبی و متوالی (**Sequential Pipeline**) اجرا می‌شوند. فاز آفلاین نقش «دروازه‌بان» (**Gatekeeper**) را دارد؛ هیچ الگوریتمی وارد **A/B** تست نمی‌شود مگر آنکه ابتدا نمره **NDCG** آن نسبت به الگوریتم پایه بهبود یافته باشد (تا از آسیب رسیدن به تجربه کاربران در محیط زنده جلوگیری شود). پس از موفقیت در فاز آنلاین، فاز کیفی به عنوان «صیقل‌دهنده نهایی» اجرا می‌شود تا مطمئن شویم اطلاعات نه تنها درست رتبه‌بندی شده‌اند، بلکه به شیواترین و کارآمدترین شکل ممکن به چشم کاربر می‌رسند.